

Message Builder Library

Axway Integration Broker

Version 2.1.1



Copyright © Axway Software, 2014
All rights reserved.

This documentation describes the following Axway software:
Axway Integration Broker.

In this document, the name of the Axway Integration Broker software product often appears as "XIB". This usage is not to be construed as an attempt by Axway Software to establish legal title over the mark "XIB", and is only for purposes of clarity, convenience, and brevity in the documentation that describes the Axway Integration Broker product.

No part of this publication may be reproduced, transmitted, stored in a retrieval system, or translated into any human or computer language, in any form or by any means, electronic, mechanical, magnetic, optical, chemical, manual, or otherwise, without the prior written permission of the copyright owner, Axway Software.

This document, provided for informational purposes only, may be subject to significant modification. The descriptions and information in this document may not necessarily accurately represent or reflect the current or planned functionalities of this product. Axway Software may change this publication, the product described herein, or both. These changes will be incorporated in new versions of this document. Axway Software does not warrant that this document is error free.

Axway Software recognizes the rights of the holders of all trademarks used in its publications.

Preface

PURPOSE OF THIS MANUAL

This manual contains documentation of Message Builder functions and statements that do not belong to the core Message Builder language. For a description of the core Message Builder language, refer to the *Message Builder* manual.

This manual contains two parts:

- The *Extension Libraries* part describes C-written extensions to the core language (see the *Extender* manual for a description of how to create such extensions).
- The *Auxiliary Functions and Statements* part describes Message-Builder-written functions and statements.

This document describes:

Chapters	Contains
Part 1. Extension libraries	
The DIR extension	describes routines for handling directories.
The PROCESS extension	describes routines for handling processes.
The RAW extension	describes routines for reading and writing ordinary files.
The REGISTER extension	describes routines for reading or writing data to two or more destinations simultaneously.
The SOCKET extension	describes routines for handling sockets.
The UNIQUE extension	describes routines for handling unique IDs.
The NET extension	describes routines for reading details of hosts or services.
The SL_ARGUMENT module	describes routines for handling command-line arguments.
The SL_ARRAY module	describes routines for handling arrays.
The SL_CHAR module	describes routines for classifying characters.
The SL_DATE module	describes routines for handling dates.
The SL_DEBUG module	describes routines for debugging Message Builder code.
The SL_ENVIRON module	describes routines for expanding environment variables.
The SL_FILE module	describes routines for handling files.
The SL_FILEIO module	describes routines for handling file I/O.
The SL_HASH module	describes routines for handling hash tables.
The SL_PACKED module	describes routines for handling packed decimal numbers.
The SL_PATH module	describes routines for handling file path names.
The SL_SEARCH module	describes routines for searching arrays.
The SL_SORT module	describes routines for sorting arrays.
The SL_STDLIB module	describes routines that are common to all modules in the Standard Library.
The SL_STRING module	describes routines for handling strings.
The SL_TAG module	describes routines for reading and writing tags from and to files.

Chapters	Contains
Part 2. AS/400 extension libraries	
The AS/400 extension libraries	describes miscellaneous routines available.
The ICONV extension	describes the routines for the conversion of data between different AS/400 character sets.
The RFILE extension	describes the routines to read, write, delete, and update records from record files on the AS/400.
The ASEXEC extension	describes the routines for dynamically calling programs on the AS400.

WHO SHOULD READ THIS MANUAL

This manual is intended for Axway Integration Broker (hereafter referred to as “Integration Broker”) developers who write Message Builder code.

ASSUMED KNOWLEDGE

In this guide, we assume that you have a good understanding of the previous versions of XIB.

RELATED DOCUMENTATION

XIB is accompanied by a complete set of documentation, covering all aspects of using the component.

All XIB documentation is available from the Axway Support website: www.support.axway.com.

Table of contents

1	Extension libraries	11
1.1	Presentation	11
1.2	The DIR extension	11
1.2.1	Summary of functions and statements	11
1.2.2	DIR_CHANGE	12
1.2.3	DIR_CURRENT	12
1.2.4	DIR_CLOSE	13
1.2.5	DIR_CREATE	13
1.2.6	DIR_ERROR	14
1.2.7	DIR_HARDLINK	14
1.2.8	DIR_OPEN	15
1.2.9	DIR_READ	15
1.2.10	DIR_REMOVE	16
1.2.11	DIR_RENAME	17
1.2.12	DIR_REWIND	17
1.2.13	DIR_VERSION	18
1.3	The PROCESS extension	18
1.3.1	Summary of functions and statements	19
1.3.2	PROCESS.KILL	19
1.3.3	PROCESS.START	20
1.3.4	PROCESS.STARTCOMMAND	21
1.3.5	PROCESS.GETPID	22
1.3.6	PROCESS.VERSION	22
1.3.7	PROCESS.WAIT	22
1.4	The RAW extension	23
1.4.1	Summary of functions and statements	23
1.4.2	RAW_CLOSE	24
1.4.3	RAW_ERROR	24
1.4.4	RAW_FLUSH	25
1.4.5	RAW_OPEN	25
1.4.6	RAW_READ	26
1.4.7	RAW_SEEK	27
1.4.8	RAW_SYNCHRONIZE	28
1.4.9	RAW_TELL	29
1.4.10	RAW_TEMPORARY	30
1.4.11	RAW_VERSION	30
1.4.12	RAW_WRITE	31
1.5	The REGISTER extension	31
1.5.1	Summary of functions and statements	32
1.5.2	REGISTER.CHECKSOCKET	32
1.5.3	REGISTER.CHILDEXITDISABLE	33
1.5.4	REGISTER.CHILDEXITENABLE	33
1.5.5	REGISTER.ERROR	34
1.5.6	REGISTER.READADD	34
1.5.7	REGISTER.READCLEAR	35
1.5.8	REGISTER.READREMOVE	36

1.5.9	REGISTER.SECONDS	37
1.5.10	REGISTER.TIMEOUTADD	37
1.5.11	REGISTER.TIMEOUTCLEAR	38
1.5.12	REGISTER.TIMEOUTREMOVE	38
1.5.13	REGISTER.VERSION	39
1.5.14	REGISTER.WAIT	40
1.5.15	REGISTER.WRITEADD	42
1.5.16	REGISTER.WRITECLEAR	43
1.5.17	REGISTER.WRITEREMOVE	43
1.6	The SOCKET extension	44
1.6.1	Summary of functions and statements	44
1.6.2	SOCKET.ACCEPT	45
1.6.3	SOCKET.CONNECT	45
1.6.4	SOCKET.NCONNECT	46
1.6.5	SOCKET.DISCONNECT	47
1.6.6	SOCKET.GETSOCKERROR	48
1.6.7	SOCKET.SETNODELAY	48
1.6.8	SOCKET.ERROR	48
1.6.9	SOCKET.GET	49
1.6.10	SOCKET.LISTEN	50
1.6.11	SOCKET.NGET	51
1.6.12	SOCKET.TGET	52
1.6.13	SOCKET.NPUT	53
1.6.14	SOCKET.PUT	54
1.6.15	SOCKET.VERSION	55
1.6.16	SOCKET.GETPEERNAME	55
1.7	The UNIQUE extension	56
1.7.1	Summary of functions and statements	56
1.7.2	UNIQUE.ADD	56
1.7.3	UNIQUE.DELETE	58
1.7.4	UNIQUE.FILECLOSE	59
1.7.5	UNIQUE.FILECREATE	59
1.7.6	UNIQUE.FILEOPEN	60
1.7.7	UNIQUE.GETID	61
1.7.8	UNIQUE.LIST	62
1.7.9	UNIQUE.VERSION	63
1.7.10	UNIQUE.SETID	63
1.8	The NET extension	65
1.8.1	Summary of functions and statements	65
1.8.2	NET.GETHOSTNAME	65
1.8.3	NET.GETHOSTBYNAME	66
1.8.4	NET.GETHOSTBYADDRESS	67
1.8.5	NET.GETSERVICEENTRIES	68
1.8.6	NET.VERSION	69
1.8.7	NET.GETSERVBYNAME	70
1.9	The UTIL extension	70
1.9.1	Summary of functions and statements	71
1.9.2	UTIL.FILESTATUS	71
1.9.3	UTIL.FILETIMES	72
1.9.4	UTIL.FILEPERM	73
1.9.5	UTIL.GETFILEPERM	74
1.9.6	UTIL.RANDOM	75
1.9.7	UTIL.SETRANDOM	76
1.9.8	UTIL.MAXRANDOM	76
1.9.9	UTIL.GETTIME	76
1.9.10	UTIL.GETTIME2	77

1.9.11	UTIL.CONVERTWITHDIFFERENTTZ	77
1.9.12	UTIL.CONVERTFROMDIFFERENTTZ	78
1.10	The MIMELIB extension	80
1.10.1	MIME.GetTimeZoneOffset	80
2	Auxiliary functions and statements	82
2.1	Presentation	82
2.2	The SL_ARGUMENT module	82
2.2.1	Summary of functions and statements	83
2.2.2	SL_ARGUMENT.GetOption	83
2.2.3	SL_ARGUMENT.GetArgument	85
2.2.4	SL_ARGUMENT.GetArgumentCount	85
2.2.5	SL_ARGUMENT.GetInterpreterArgument	86
2.2.6	SL_ARGUMENT.GetInterpreterArgumentCount	87
2.2.7	SL_ARGUMENT.GetInterpreterFiles	87
2.2.8	SL_ARGUMENT.Version	88
2.3	The SL_ARRAY module	88
2.3.1	Summary of functions and statements	88
2.3.2	SL_ARRAY.Insert	89
2.3.3	SL_ARRAY.Append	89
2.3.4	SL_ARRAY.Delete	90
2.3.5	SL_ARRAY.Clear	91
2.3.6	SL_ARRAY.Compare	91
2.3.7	SL_ARRAY.Version	91
2.4	The SL_CHAR module	92
2.4.1	Summary of functions	92
2.4.2	SL_CHAR.IsAlpha	92
2.4.3	SL_CHAR.IsAlphaUnderscore	93
2.4.4	SL_CHAR.IsAlphaDigit	94
2.4.5	SL_CHAR.IsAlphaDigitUnderscore	94
2.4.6	SL_CHAR.IsDigit	95
2.4.7	SL_CHAR.IsHex	96
2.4.8	SL_CHAR.IsLower	96
2.4.9	SL_CHAR.IsPrintable	97
2.4.10	SL_CHAR.IsUpper	97
2.4.11	SL_CHAR.ClassCreate	98
2.4.12	SL_CHAR.ClassAdd	99
2.4.13	SL_CHAR.ClassRemove	100
2.4.14	SL_CHAR.IsInClass	100
2.4.15	SL_CHAR.Version	101
2.5	The SL_DATE module	101
2.5.1	Summary of functions	102
2.5.2	SL_DATE.StringToDate	102
2.5.3	SL_DATE.Version	103
2.6	The SL_DEBUG module	103
2.6.1	Summary of functions and statements	103
2.6.2	SL_DEBUG.Assert	103
2.6.3	SL_DEBUG.Version	104
2.7	The SL_ENVIRON module	105
2.7.1	Summary of functions	105
2.7.2	SL_ENVIRON.ExpandEnvironmentVariables	105
2.7.3	SL_ENVIRON.Version	106

2.8	The SL_FILE module	106
2.8.1	Summary of functions and statements	106
2.8.2	SL_FILE.DeleteDirectoryTree	106
2.8.3	SL_FILE.ReadDirectory	107
2.8.4	SL_FILE.SilentDeleteFile	108
2.8.5	SL_FILE.Version	109
2.9	The SL_FILEIO module	109
2.9.1	Summary of functions and statements	110
2.9.2	SL_FILEIO.OpenBinaryRead	110
2.9.3	SL_FILEIO.OpenTextRead	111
2.9.4	SL_FILEIO.CreateBinaryWrite	112
2.9.5	SL_FILEIO.CreateTextWrite	113
2.9.6	SL_FILEIO.AppendBinaryWrite	114
2.9.7	SL_FILEIO.AppendTextWrite	115
2.9.8	SL_FILEIO.OpenBinaryReadWrite	116
2.9.9	SL_FILEIO.CreateBinaryReadWrite	117
2.9.10	SL_FILEIO.AppendBinaryReadWrite	118
2.9.11	SL_FILEIO.ReadCount	119
2.9.12	SL_FILEIO.ReadUntil	120
2.9.13	SL_FILEIO.Unread	121
2.9.14	SL_FILEIO.Synchronize	122
2.9.15	SL_FILEIO.WriteString	122
2.9.16	SL_FILEIO.Seek	123
2.9.17	SL_FILEIO.Tell	124
2.9.18	SL_FILEIO.Flush	124
2.9.19	SL_FILEIO.CloseFile	125
2.9.20	SL_FILEIO.IsOpen	125
2.9.21	SL_FILEIO.IsReadable	126
2.9.22	SL_FILEIO.IsWritable	127
2.9.23	SL_FILEIO.IsBinary	127
2.9.24	SL_FILEIO.IsText	128
2.9.25	SL_FILEIO.DirectorySeparator	128
2.9.26	SL_FILEIO.Version	129
2.10	The SL_HASH module	129
2.10.1	Summary of functions and statements	129
2.10.2	SL_HASH.Initialize	130
2.10.3	SL_HASH.Put	130
2.10.4	SL_HASH.Get	130
2.10.5	SL_HASH.Delete	131
2.10.6	SL_HASH.Size	132
2.10.7	SL_HASH.EnumerateNext	132
2.10.8	SL_HASH.Clear	133
2.10.9	SL_HASH.Version	133
2.11	The SL_PACKED module	134
2.11.1	Summary of functions	134
2.11.2	SL_PACKED.SapIntegerToInt4	134
2.11.3	SL_PACKED.SapInt4ToInteger	135
2.11.4	SL_PACKED.IbmFloatToPackedDecimal	135
2.11.5	SL_PACKED.IbmPackedDecimalToFloat	136
2.11.6	SL_PACKED.Version	137
2.12	The SL_PATH module	137
2.12.1	Summary of functions and statements	137
2.12.2	SL_PATH.Filter	138
2.12.3	SL_PATH.Compare	138
2.12.4	SL_PATH.CompareFilter	139

2.12.5	SL_PATH.AppendSeparator	140
2.12.6	SL_PATH.DirectorySeparator	140
2.12.7	SL_PATH.ReplaceSlashes	140
2.12.8	SL_PATH.Version	141
2.13	The SL_SEARCH module	141
2.13.1	Summary of functions	141
2.13.2	SL_SEARCH.BinarySearch	142
2.13.3	SL_SEARCH.Version	142
2.14	The SL_SORT module	143
2.14.1	Summary of functions and statements	143
2.14.2	SL_SORT.HeapSort_Any	143
2.14.3	SL_SORT.HeapSort_String	144
2.14.4	SL_SORT.QuickSort_String	145
2.14.5	SL_SORT.QuickSort_Integer	146
2.14.6	SL_SORT.Version	146
2.15	The SL_STDLIB module	147
2.15.1	Summary of functions	147
2.15.2	SL_STDLIB.Version	147
2.16	The SL_STRING module	147
2.16.1	Summary of functions	148
2.16.2	SL_STRING.AsciiToEbcDic	148
2.16.3	SL_STRING.EbcDicToAscii	149
2.16.4	SL_STRING.FieldCount	149
2.16.5	SL_STRING.FieldExtract	150
2.16.6	SL_STRING.FieldExplode	151
2.16.7	SL_STRING.FieldImplode	152
2.16.8	SL_STRING.Append	152
2.16.9	SL_STRING.Remove	153
2.16.10	SL_STRING.Replace	153
2.16.11	SL_STRING.Reverse	154
2.16.12	SL_STRING.ToLower	154
2.16.13	SL_STRING.ToUpper	155
2.16.14	SL_STRING.FindInSet	156
2.16.15	SL_STRING.FindNotInSet	156
2.16.16	SL_STRING.WildcardMatch	157
2.16.17	SL_STRING.Version	157
2.17	The SL_TAG module	158
2.17.1	Format of Tag files	158
2.17.2	Summary of functions and statements	158
2.17.3	SL_TAG.Put	159
2.17.4	SL_TAG.Get	160
2.17.5	SL_TAG.Remove	161
2.17.6	SL_TAG.List	162
2.17.7	SL_TAG.Version	163
3	AS/400 specific extensions	164
3.1	The AS400 extension	164
3.2	Language reference	164
3.2.1	AS400.BINDEC function	164
3.2.2	AS400.DIREINFO statement	165
3.2.3	AS400.FILEINFO statement	166
3.2.4	AS400.PCKDEC function	168
3.2.5	AS400.RCVDTAQ statement	169

3.2.6	AS400.RCVMSG statement	171
3.2.7	AS400.RTVDTAARA statement	172
3.2.8	AS400.SNDDTAQ function	173
3.2.9	AS400.SNDMSG statement	174
3.2.10	AS400.UNBDEC function	175
3.2.11	AS400.UNPDEC function	177
3.2.12	AS400.UNZDEC function	178
3.2.13	AS400.USERINFO statement	179
3.2.14	AS400.ZNDDEC function	180
3.3	The ICONV extension	182
3.4	Language reference	182
3.4.1	ICONV.CCLOSE statement	182
3.4.2	ICONV.CONVERT function	183
3.4.3	ICONV.COPEN function	184
3.5	The RFILE extension	185
3.6	Language reference	185
3.6.1	RFILE.COMMIT statement	185
3.6.2	RFILE.ENDCOMMITSESSION statement	187
3.6.3	RFILE.FORMAT statement	188
3.6.4	RFILE.LOCATE statement	189
3.6.5	RFILE.RCLOSE statement	191
3.6.6	RFILE.RDELETE statement	192
3.6.7	RFILE.ROLLBACK statement	193
3.6.8	RFILE.ROPEN function	194
3.6.9	RFILE.RREAD function	196
3.6.10	RFILE.RUPDATE statement	197
3.6.11	RFILE.RWRITE statement	198
3.6.12	RFILE.STARTCOMMITSESSION statement	199
3.7	The ASEXEC file extension	201
3.8	Example implementation	201
3.9	Language Reference	202
3.9.1	ASEXEC.AddParameter function	202
3.9.2	ASEXEC.ExecuteProgram statement	203
3.9.3	ASEXEC.GetParameter function	203
3.9.4	ASEXEC.RemoveParameter statement	204
3.9.5	ASEXEC.SetParameter statement	204

1 Extension libraries

1.1 Presentation

This part describes the following functions and statements that are implemented as Message Builder extensions (for a description of how to create such extensions, refer to the *Extender* manual):

Function	Used For
The DIR Extension	Reading the contents of directories, creating and removing directories, and changing the current directory.
The PROCESS Extension	Starting, stopping, and supervision of processes.
The RAW Extension	Reading and writing ordinary files.
The REGISTER Extension	Reading and writing data to several destinations simultaneously.
The SOCKET Extension	Handling sockets.
The UNIQUE Extension	Unique ID support.
The NET Extension	Reading details of hosts or services.
The MIMELIB Extension	Included here only for a timezone function.

For a description of the conventions used in this section of the manual, refer to *Message Builder language reference* of the *Message Builder* manual.

1.2 The DIR extension

This chapter describes the Message Builder DIR extension. The DIR extension contains functions and statements that read the contents of directories, create and remove directories, and change the current directory.

The DIR extension generates EXECUTE exceptions when errors occur.

All of the DIR extension functions and statements are declared in the library file *edir.s4*. A number of constants (for example, error codes) can be found in the include file *edir.s4h*.

Since the DIR extension uses the MODULE_INIT feature for initialization, no functions or statements from the DIR extension should be called from other MODULE_INIT statements.

1.2.1 Summary of functions and statements

Function/Statement	Used For
DIR_CHANGE	Changing the current directory.
DIR_CLOSE	Closing a directory.
DIR_CREATE	Creating a directory.

Function/Statement	Used For
DIR_CURRENT	Obtaining the current directory.
DIR_ERROR	Returning the last error message.
DIR_OPEN	Opening a directory.
DIR_READ	Reading the next entry in a directory.
DIR_REMOVE	Removing a directory.
DIR_RENAME	Renaming a directory.
DIR_REWIND	Resetting the position to the beginning of the directory.
DIR_VERSION	Returning version number.

1.2.2 DIR_CHANGE

Name

DIR_CHANGE – Change the current directory.

Syntax

DIR_CHANGE *dir-name*;

Description

The DIR_CHANGE statement changes the current directory. The directory to change to is specified by the *dir-name* parameter. It is an *expression* that is converted to a string.

Errors

The following value can be assigned to \$Error:

```
$Error_DirChangeFail
```

Failed to change the current directory.

Examples

```
DIR_CHANGE "xyz";
```

1.2.3 DIR_CURRENT

Name

DIR_CURRENT - Returns the current directory.

Syntax

DIR_CURRENT()

Description

The DIR_CURRENT function returns the current directory.

Errors

No errors are returned.

Examples

LOG "the current directory is " & DIR_CURRENT() TYPE "INFO";

1.2.4 DIR_CLOSE

Name

DIR_CLOSE – Close a directory.

Syntax

DIR_CLOSE *dir-id*;

Description

The DIR_CLOSE statement closes the open directory specified by *dir-id*. The *dir-id* parameter should be an open directory ID previously returned by the DIR_OPEN function. It is an *expression* that is converted to an integer.

Errors

The following value can be assigned to `$Error`:

`$Error_DirCloseFail`

Failed to close the directory.

Examples

For an example, refer to the DIR_READ function, paragraph [1.2.9](#).

See also

DIR_OPEN, DIR_READ, DIR_REWIND.

1.2.5 DIR_CREATE

Name

DIR_CREATE – Create a directory.

Syntax

DIR_CREATE *dir-name*;

Description

The DIR_CREATE statement creates a directory. The name of the directory is specified by *dir-name*. It is an *expression* that is converted to a string.

Errors

The following value can be assigned to `$Error`:

```
$Error_DirCreateFail
```

Failed to create the directory.

Examples

```
DIR_CREATE "xyz";
```

See also

DIR_REMOVE.

1.2.6 DIR_ERROR

Name

DIR_ERROR – Function that returns the last error generated by the DIR extension.

Syntax

DIR_ERROR().

Description

The DIR_ERROR function returns the error string for the last error generated by the DIR extension. If no error has been generated, an empty string is returned.

1.2.7 DIR_HARDLINK

Name

DIR_HARDLINK. - A statement that creates a hard link between two different files. This is supported on all platforms.

Syntax

DIR_HARDLINK *source* DEST *dest*;

Description

The file *source* can now be accessed also by the name *dest*.

Errors

The following value can be assigned to `$Error`:

```
$Error_DirHardlinkFail
```

Failed to create the hard link.

1.2.8 DIR_OPEN

Name

DIR_OPEN – Function that opens a directory.

Syntax

DIR_OPEN([*dir-name*])

Description

The DIR_OPEN function opens a directory for reading. The *dir-name* argument is the name of the directory to open. It is an *expression* that is converted to a string.

This function returns an integer value that should be used for referring to the directory when performing other operations on it.

If *dir-name* is not specified, the current working directory is opened.

Errors

The following value can be assigned to `$Error`:

```
$Error_DirOpenFail
```

Failed to open the directory.

Examples

For an example, refer to the DIR_READ function, paragraph [1.2.9](#).

See also

DIR_CLOSE, DIR_READ, DIR_REWIND.

1.2.9 DIR_READ

Name

DIR_READ – Function that reads the next entry in a directory.

Syntax

DIR_READ(*dir-id*)

Description

The DIR_READ function reads the next entry from the directory specified by the *dir-id* argument. The *dir-id* argument should be an open directory ID previously returned by the DIR_OPEN function. It is an *expression* that is converted to an integer.

A string value representing the next directory entry is returned. If there are no more entries in the directory, an empty string is returned.

Errors

The following value can be assigned to \$Error:

```
$Error_DirReadFail
```

Failed to read directory.

Examples

The following program opens the current working directory, reads all the entries, writes them to the output file, and, finally, closes the directory.

```
DECLARE VARIABLE $dirId INTEGER;
DECLARE VARIABLE $name STRING;

$dirId = DIR_OPEN();
REPEAT {
    $name = DIR_READ($dirId);
    IF $name <> "" {
        PRINT $name & "\n";
    }
} WHILE $name <> "";
DIR_CLOSE $dirId;
```

See also

DIR_CLOSE, DIR_OPEN, DIR_REWIND.

1.2.10 DIR_REMOVE

Name

DIR_REMOVE – Remove a directory.

Syntax

DIR_REMOVE *dir-name*;

Description

The DIR_REMOVE statement removes a directory. The name of the directory is specified by *dir-name*. It is an *expression* that is converted to a string.

Errors

The following value can be assigned to `$Error`:

```
$Error_DirRemoveFail
```

Failed to remove the directory.

Examples

```
DIR_REMOVE "xyz";
```

See also

DIR_CREATE.

1.2.11 DIR_RENAME

Name

DIR_RENAME – Rename a directory.

Syntax

```
DIR_RENAME old-dir-name TO new-dir-name;
```

Description

The DIR_RENAME statement renames a directory from *old-dir-name* to *new-dir-name*.

old-dir-name and *new-dir-name* are *expressions* that are converted to strings.

new-dir-name must not exist.

Errors

The following value can be assigned to `$Error`:

```
$Error_DirRenameFail
```

Failed to rename the directory.

Examples

```
DIR_RENAME "oldname" TO "newname";
```

See also

DIR_REMOVE.

1.2.12 DIR_REWIND

Name

DIR_REWIND – Reset the position to the beginning of the directory.

Syntax

`DIR_REWIND dir-id;`

Description

The `DIR_REWIND` statement resets the position for an opened directory to the beginning of the directory. The *dir-id* parameter should be an open directory ID previously returned by the `DIR_OPEN` function. It is an *expression* that is converted to an integer.

Errors

The following value can be assigned to `$Error`:

`$Error_DirRewindFail`

Failed to rewind directory.

Examples

```
DIR_REWIND $dirId;
```

See also

`DIR_CLOSE`, `DIR_OPEN`, `DIR_READ`.

1.2.13 DIR_VERSION

Name

`DIR_VERSION` – Function that returns the version number of the DIR extension.

Syntax

`DIR_VERSION()`

Description

The `DIR_VERSION` function returns the version number of the DIR extension as a string. The format of the string is *n1.n2.n3*.

1.3 The PROCESS extension

This chapter describes the Message Builder `PROCESS` extension module. The `PROCESS` extension module contains functions and statements to handle starting, stopping, and supervision of processes.

The error type generated by the `PROCESS` functions and statements is `PROCESS`. The following program fragment demonstrates how to handle `PROCESS` errors:

```
TRY {  
    $pid = PROCESS.START($arguments);  
}
```

```

CATCH $exception
WHEN "PROCESS" {
    PRINT "Error when trying to start process\n";
}

```

All the PROCESS extension module functions and statements are declared in the library file *eprocess.s4*. A number of constants (for example, error codes) can be found in the include file *eprocess.s4h*.

Since the PROCESS extension uses the MODULE_INIT feature for initialization, no functions or statements from the PROCESS extension should be called from other MODULE_INIT statements.

1.3.1 Summary of functions and statements

Function/Statement	Used For
PROCESS.KILL	Killing a process.
PROCESS.START	Starting a process.
PROCESS.STARTCOMMAND	Starting a command.
PROCESS.VERSION	Returning version number.
PROCESS.GETPID	Obtaining the current process id.
PROCESS.WAIT	Waiting for processes to terminate.

1.3.2 PROCESS.KILL

Name

PROCESS.KILL – Kill a process.

Syntax

PROCESS.KILL *pid* [SIGNAL *signal*];

Description

The PROCESS.KILL statement sends a signal to a process or tests if a process does exist.

The *pid* parameter is the process ID for the process to send a signal to or test the existence of. The *signal* parameter is the signal to send. If it is *\$Process_SigTest*, it is tested if the process does exist (if it does not, an error is generated). The following are valid values for *signal* (they are declared in the *eprocess.s4h* file):

```

$Process_SigTest
$Process_SigHup
$Process_SigInt
$Process_SigQuit
$Process_SigKill
$Process_SigAlrm
$Process_SigTerm
$Process_SigUsr1
$Process_SigUsr2

```

Both the *pid* and *signal* parameters are *expressions*, which are converted to integers.

Errors

The following values can be assigned to `$Error`:

- `$Error_ProcessKillBadSignal`
An invalid signal was specified.
- `$Error_ProcessKillNoProcess`
The process specified by *pid* does not exist.

1.3.3 PROCESS.START

Name

PROCESS.START – Function that starts a process.

Syntax

PROCESS.START(*arguments* [, *stdin* [, *stdout* [, *stderr* [, *flags*]]]])

Description

The PROCESS.START function starts a process.

The *arguments* argument is a string array containing the program arguments. The first element in this array should be the program to execute. The *stdin*, *stdout*, and *stderr* arguments can be used to specify the standard input file and the standard output and error output files, respectively. If they are not specified or are set to empty strings, the process will use the same standard input, output, and error output as the Message Builder interpreter. On UNIX, output from the command is appended to the *stdout* and *stderr* files (that is, the process is started as if shell operators `>>` or, respectively, `2>>` were used).

An integer identifying the process is returned.

The *stdin*, *stdout*, and *stderr* arguments are *expressions*, which are converted to strings.

The *flags* argument can contain the following flags, combined with the BOR operator:

- `$Process_ThreadEnableOnOs400`: this is the only flag enabled by default (that is, when the *flags* parameter is not present at all; if the *flags* parameter is present in the call and you want to use this parameter you must add it yourself with BOR). It specifies whether the process will be able to start threads on the IBM OS/400 operating system (now known as System i).
- `Process_WillNotUseProcessDotWait`: this flag must be used by MBCs because they use `MBC_HIERCHMSGENV.Wait` instead of `PROCESS.WAIT`. Using this flag eliminates a memory leak in this case. You must also not call any statements or functions that interrupt the current coroutine, such as `MBC_HIERCHMSGENV.Wait`, or `MBC_HIERCHMSGENV.GetData()`, `DATAIO.Copy()`, `DATAIO.WriteOutput`, `DATAIO.CloseOutput`, between the call for `PROCESS.START` and the corresponding call to `MBC_HIERCHMSGENV.Wait`. In the previous list, `MBC_HIERCHMSGENV.Wait` is allowed only if the process id returned by `PROCESS.START` is in the list of process ids to wait for.

Errors

The following value can be assigned to `$Error`:

```
$Error_ProcessStartNoArguments
```

The *arguments* array is empty.

```
$Error_ProcessStartOpenFail
```

One of the files specified for input or output redirection could not be opened.

```
$Error_ProcessStartStartFail
```

Failed to start the process specified by *arguments*.

Notes

On UNIX and Windows 2000 systems, the PATH environment variable is used to find the program to execute.

On AS/400 systems, the path is used to find the program to execute. The program name used must be a symbolic link that points to a program in an AS/400 library, for example:

```
ADDLNK OBJ(/QSYS.LIB/TRADESYNC.LIB/MYPROG.PGM)
        NEWLNK('bin/myprog') LNKTYPE(*SYMBOLIC)
```

On AS/400 systems, the *stdin*, *stdout*, and *stderr* arguments are ignored.

1.3.4 PROCESS.STARTCOMMAND

Name

PROCESS.STARTCOMMAND – Function that starts a command.

Syntax

```
PROCESS.STARTCOMMAND(command [, stdin [, stdout [, stderr [, flags]]]])
```

Description

The PROCESS.STARTCOMMAND function starts the command specified by the *command* argument.

The *command* argument is the command to start. The *stdin*, *stdout*, and *stderr* arguments can be used to specify the standard input file and the standard output and error output files, respectively. If they are not specified or are set to empty strings, the process will use the same standard input, output, and error output as the Message Builder interpreter. On UNIX, output from the command is appended to the *stdout* and *stderr* files (that is, the process is started as if shell operators >> or, respectively, 2>> were used).

See the previous documentation of PROCESS.START for the values that can be specified in the *flags* parameter.

An integer identifying the process is returned.

All arguments are *expressions*, which are converted to strings.

Errors

The following value can be assigned to `$Error`:

```
$Error_ProcessStartNoArguments
```

command is empty.

`$Error_ProcessStartOpenFail`

One of the files specified for input or output redirection could not be opened.

`$Error_ProcessStartStartFail`

Failed to start the process specified by *command*.

Notes

This function is implemented as a Message Builder written user function. It uses the PROCESS.START function to start the process.

1.3.5 PROCESS.GETPID

Name

PROCESS.GETPID. - Function that returns the process identifier of the current process.

Syntax

PROCESS.GETPID()

Description

This function returns the process id of the current process.

1.3.6 PROCESS.VERSION

Name

PROCESS.VERSION – Function that returns the version number of the PROCESS extension.

Syntax

PROCESS.VERSION()

Description

The PROCESS.VERSION function returns the version number of the PROCESS extension as a string. The format of the string is *n1.n2.n3*.

1.3.7 PROCESS.WAIT

Name

PROCESS.WAIT – Wait for processes to terminate.

Syntax

```
PROCESS.WAIT wait-pids PID pid [EXITSTATUS exit-status]  
[WAIT timeout];
```

Description

The PROCESS.WAIT statement waits for processes to terminate.

The *wait-pids* parameter is an integer vector containing the process identifiers for all the processes to wait for. The *timeout* parameter is the maximum amount of time (in units of seconds) to wait for any processes to terminate. It is an *expression* that is converted to an integer.

On return, the output parameter *pid* contains the process identifier for the terminated process or `$Process_Timeout` if a timeout occurred. The *exit-status* parameter contains the exit status for the process or the negative signal number if the process was killed by a signal. Both *pid* and *exit-status* are integer *variables*.

It is forbidden to call this in an MBC; see PROCESS.Start for details.

Errors

The following value can be assigned to `$Error`:

```
$Error_ProcessWaitNoProcess
```

None of the processes specified by *wait-pids* exists.

1.4 The RAW extension

This chapter describes the Message Builder RAW extension. The RAW extension contains functions and statements to read and write ordinary files. The RAW extension generates EXECUTE exceptions when errors occur.

All the RAW extension functions and statements are declared in the library file *eraw.s4*. A number of constants (for example, error codes) can be found in the include file *eraw.s4h*.

Since the RAW extension uses the MODULE_INIT feature for initialization, no functions or statements from the RAW extension should be called from other MODULE_INIT statements.

1.4.1 Summary of functions and statements

Function/Statement	Used For
RAW_CLOSE	Closing a raw file.
RAW_ERROR	Returning the last error message.
RAW_FLUSH	Flushing the contents of a raw file buffer.
RAW_OPEN	Opening a raw file.
RAW_READ	Reading data from a raw file.
RAW_SEEK	Setting the position for a raw file.
RAW_SYNCHRONIZE	Physically writing updates of a file to disk.
RAW_TELL	Returning the current file position.

Function/Statement	Used For
RAW_TEMPORARY	Returning a unique temporary file name.
RAW_VERSION	Returning version number.
RAW_WRITE	Writing data to a raw file.

1.4.2 RAW_CLOSE

Name

RAW_CLOSE – Close a raw file.

Syntax

RAW_CLOSE *file-id*;

Description

The RAW_CLOSE statement closes the open file specified by *file-id*. The *file-id* parameter should be an open file ID previously returned by the RAW_OPEN function. It is an *expression* that is converted to an integer.

Errors

The following value can be assigned to `$Error`:

```
$Error_RawCloseCloseFail
```

Failed to close the file.

Examples

For an example, refer to the RAW_READ function on paragraph [1.4.6](#).

See also

RAW_FLUSH, RAW_OPEN, RAW_READ, RAW_SEEK, RAW_WRITE.

1.4.3 RAW_ERROR

Name

RAW_ERROR – Function that returns the last error generated by the RAW extension.

Syntax

RAW_ERROR()

Description

The RAW_ERROR function returns the error string for the last error generated by the RAW extension. If no error has been generated, an empty string is returned.

1.4.4 RAW_FLUSH

Name

RAW_FLUSH – Flush a raw file buffer.

Syntax

RAW_FLUSH *file-id*;

Description

The RAW_FLUSH statement flushes the contents of the file buffer for the open file specified by *file-id*. The *file-id* parameter should be an open file ID previously returned by the RAW_OPEN function. It is an *expression* that is converted to an integer.

To increase the performance, when writing data using the RAW_WRITE statement, data is buffered internally in the RAW extension to avoid making small write requests to the operating system. This statement writes the contents of the internal buffer to the file.

Errors

The following value can be assigned to `$Error`:

```
$Error_RawFlushFlushFail
```

Failed to flush.

Examples

```
RAW_FLUSH $fileId;
```

See also

RAW_CLOSE, RAW_OPEN, RAW_READ, RAW_SEEK, RAW_WRITE.

1.4.5 RAW_OPEN

Name

RAW_OPEN – Function that opens a raw file.

Syntax

RAW_OPEN(*file-name*[, *open-type*])

Description

The RAW_OPEN function opens a file for reading, writing, or both. The *file-name* argument is the name of the file to open, and *open-type* specifies the type of open. Both are *expressions* that are converted to strings.

The *open-type* argument should have one of the following values:

Values	Meaning
r	Open for reading.
w	Truncate to zero length or create for writing.
a	Append; open for writing at end of file, or create for writing.
r+	Open for update (reading and writing).
w+	Truncate to zero length or create for update.
a+	Append; open or create for update at end of file.

For Windows 2000 systems, append a `t` or a `b` to the *open-type* argument to specify text or binary mode, respectively. The default is text mode (`t`).

For AS/400 systems, append a `b` to the *open-type* argument to specify binary mode. The default is text mode (`t` is not accepted). The `b` option should be used for opening files in binary mode (CCSID 500), and the absence of this option will open a file in text mode (CCSID 819).

In order to create a binary or text file of the appropriate CCSID, the file must be opened in the correct mode.

If *open-type* is not specified, the file is opened for reading.

Errors

The following value can be assigned to `$Error`:

```
$Error_RawOpenOpenFail
```

Failed to open file.

Examples

For an example, refer to the RAW_READ function on paragraph [1.4.6](#).

See also

RAW_CLOSE, RAW_FLUSH, RAW_READ, RAW_SEEK, RAW_WRITE.

Notes

This function uses the *fopen* C library function, and the *open-type* argument is used as the second argument to this function.

1.4.6 RAW_READ

Name

RAW_READ – Function that reads a raw file.

Syntax

`RAW_READ(file-id, count)`

Description

The `RAW_READ` function reads data from the file specified by the *file-id* argument. It should be an open file ID previously returned by the `RAW_OPEN` function. The maximum amount of data to read is specified by *count*. Both *file-id* and *count* are *expressions* that are converted to integers.

A string containing the read data is returned. Note that the returned string may be shorter than *count*. No error is generated if end of file is reached. Instead, an empty string is returned.

Errors

The following values can be assigned to `$Error`:

- `$Error_RawReadIllCount`
Illegal read count. The read count must be greater than or equal to 0.
 - `$Error_RawReadReadFail`
Failed to read from file.
-

Examples

The following user statement copies a file:

```
DECLARE STATEMENT COPY_FILE $source TO $destination
{
    DECLARE VARIABLE $rfd INTEGER;
    DECLARE VARIABLE $wfd INTEGER;
    DECLARE VARIABLE $data STRING;

    $rfd = RAW_OPEN($source);
    $wfd = RAW_OPEN($destination, "w");
    REPEAT {
        $data = RAW_READ($rfd, 1000);
        RAW_WRITE $wfd DATA $data;
    } WHILE STRLEN($data) > 0;
    RAW_CLOSE $wfd;
    RAW_CLOSE $rfd;

    RETURN;
}
```

See also

`RAW_CLOSE`, `RAW_FLUSH`, `RAW_OPEN`, `RAW_SEEK`, `RAW_WRITE`.

1.4.7 RAW_SEEK

Name

`RAW_SEEK` – Set the position for a raw file.

Syntax

`RAW_SEEK file-id [OFFSET offset] [WHENCE whence];`

Description

The `RAW_SEEK` statement repositions the file position for the file specified by *file-id*. It should be an open file ID previously returned by a call to `RAW_OPEN`.

The new file position is obtained by adding *offset* to the position specified by *whence*. If *whence* is 0, *offset* specifies the new position relative to the beginning of the file. If *whence* is 1, *offset* specifies the new position relative to the current file position. If *whence* is 2, *offset* specifies the new position relative to the end of the file.

The default values for both *offset* and *whence* are 0.

All parameters are *expressions* that are converted to integers.

Errors

The following value can be assigned to `$Error`:

```
$Error_RawSeekSeekFail
```

Seek error.

Examples

Set the file position to the end of the file:

```
RAW_SEEK $fileId WHENCE 2;
```

See also

`RAW_CLOSE`, `RAW_FLUSH`, `RAW_OPEN`, `RAW_READ`, `RAW_WRITE`.

1.4.8 RAW_SYNCHRONIZE

Name

`RAW_SYNCHRONIZE` – Physically write updates of a file to disk.

Syntax

`RAW_SYNCHRONIZE file-id;`

Description

The `RAW_SYNCHRONIZE` statement first flushes the contents of the file buffer for the open file specified by *file-id* and then physically writes the updates of the file to disk. The *file-id* parameter should be an open file ID previously returned by the `RAW_OPEN` function. It is an *expression* that is converted to an integer.

Errors

The following values can be assigned to `$Error`:

- `$Error_RawSynchronizeFlushFail`
Failed to flush.
- `$Error_RawSynchronizeSynchronizeFail`
Failed to physically write the updated file to disk.
- `$Error_RawSynchronizeNoSupport`
The `RAW_SYNCHRONIZE` statement is not supported. All currently supported platforms supports `RAW_SYNCHRONIZE`.

Examples

```
RAW_SYNCHRONIZE $fileId;
```

See also

`RAW_FLUSH`.

1.4.9 RAW_TELL

Name

`RAW_TELL` – Function that returns the current file position.

Syntax

```
RAW_TELL(file-id)
```

Description

The `RAW_TELL` function returns the current file position for the file specified by the *file-id* argument. It should be an open file ID previously returned by the `RAW_OPEN` function.

The *file-id* argument is an *expression* that is converted to an integer. The returned value is of integer type.

Errors

The following value can be assigned to `$Error`:

```
$Error_RawTellTellFail
```

Failed to obtain the file position.

Examples

```
$position = RAW_TELL($fileId);
```

See also

`RAW_CLOSE`, `RAW_FLUSH`, `RAW_OPEN`, `RAW_READ`, `RAW_SEEK`, `RAW_WRITE`.

1.4.10 RAW_TEMPORARY

Name

RAW_TEMPORARY – Function that returns a unique temporary file name.

Syntax

RAW_TEMPORARY([*dir*])

Description

The RAW_TEMPORARY function is used to create unique temporary file names.

The directory prefix for the unique file name is specified by the *dir* argument. The *dir* argument is an *expression* that is converted to a string.

If *dir* is not specified or is empty, a machine-dependent default directory prefix is used.

Errors

The following value can be assigned to `$Error`:

```
$Error_RawTemporaryCreateFail
```

Failed to create a unique temporary file name.

Examples

Create a unique temporary file name using the default directory:

```
$file1 = RAW_TEMPORARY();
```

Specify a directory prefix on a UNIX system:

```
$file2 = RAW_TEMPORARY("/usr/tradesync/tmp");
```

Specify a directory prefix on a Windows 2000 system:

```
$file2 = RAW_TEMPORARY("C:\\users\\tradesync\\tmp");
```

Notes

The functionality of RAW_TEMPORARY is based on the *tempnam* C library function.

The RAW_TEMPORARY function creates file names that are unique in the specified directory. If RAW_TEMPORARY is called multiple times with different directories, some of the returned file names may be equal (if the directory part is removed).

1.4.11 RAW_VERSION

Name

RAW_VERSION – Function that returns the version number of the RAW extension.

Syntax

RAW_VERSION()

Description

The RAW_VERSION function returns the version number of the RAW extension as a string. The format of the string is *n1.n2.n3*.

1.4.12 RAW_WRITE

Name

RAW_WRITE – Write data to a raw file.

Syntax

RAW_WRITE *file-id* DATA *data*;

Description

The RAW_WRITE statement writes the data specified by *data* to the file specified by *file-id*. The *file-id* parameter should be an open file ID previously returned by the RAW_OPEN function.

The *file-id* parameter is an *expression* that is converted to an integer, and *data* is an *expression* that is converted to a string. Note that it is allowed to write empty strings.

Errors

The following values can be assigned to `$Error`:

- `$Error_RawWriteWriteCount`
Failed to write all the data.
- `$Error_RawWriteWriteFail`
Failed to write to file.

Examples

For an example, refer to the RAW_READ function on paragraph [1.4.6](#).

See also

RAW_CLOSE, RAW_FLUSH, RAW_OPEN, RAW_READ, RAW_SEEK.

1.5 The REGISTER extension

This chapter describes the Message Builder REGISTER extension module. The REGISTER extension module should be used when data is to be read from or written to several places simultaneously. This is accomplished by its ability to register interest in reading from a

source or writing to a destination identified by a socket descriptor. The REGISTER module also has support for time-outs.

The REGISTER module is intended to be used with other extension modules when it is necessary to read and write to multiple channels simultaneously. An example is the OFTP extension. The REGISTER module is actually an interface to the *select* operating system call.

The error type generated by the REGISTER functions and statements is REGISTER. The following program fragment demonstrates how to handle REGISTER errors:

```
TRY {
    REGISTER.Wait $type;
}
CATCH $exception
WHEN "REGISTER" {
    PRINT "Error when waiting for an event\n";
}
```

All the REGISTER module extension statements are declared in the library file *ereg.s4*. A number of constants (for example, error codes) that are useful when using the REGISTER module can be found in the include file *ereg.s4h*.

Since the REGISTER extension uses the MODULE_INIT feature for initialization, no functions or statements from the REGISTER extension should be called from other MODULE_INIT statements.

1.5.1 Summary of functions and statements

Function/Statement	Used For
REGISTER.CHECKSOCKET	Checking for a valid socket descriptor.
REGISTER.CHILDEXITDISABLE	Disabling child process exit events.
REGISTER.CHILDEXITENABLE	Enabling child process exit events.
REGISTER.ERROR	Returning the last error message.
REGISTER.READADD	Registering interest in reading from a channel.
REGISTER.READCLEAR	Removing everything registered for reading from a channel.
REGISTER.READREMOVE	Removing something registered for reading from a channel.
REGISTER.SECONDS	Returning the current time.
REGISTER.TIMEOUTADD	Registering a timeout.
REGISTER.TIMEOUTCLEAR	Removing all timeouts.
REGISTER.TIMEOUTREMOVE	Removing one timeout.
REGISTER.VERSION	Returning version number.
REGISTER.WAIT	Waiting for an event.
REGISTER.WRITEADD	Registering interest for writing to a channel.
REGISTER.WRITECLEAR	Removing everything registered for writing to a channel.
REGISTER.WRITEREMOVE	Removing something registered for writing to a channel.

1.5.2 REGISTER.CHECKSOCKET

Name

REGISTER.CHECKSOCKET – Function that checks for a valid socket descriptor.

Syntax

REGISTER.CHECKSOCKET(*sock-desc*)

Description

The REGISTER.CHECKSOCKET function checks that *sock-desc* is a valid socket descriptor. The value 1 is returned if *sock-desc* is a valid socket descriptor. Otherwise, the value 0 is returned.

The *sock-desc* argument is an *expression* that is converted to an integer.

Notes

On UNIX machines, REGISTER.CHECKSOCKET also returns the value 1 when *sock-desc* represents a FIFO file.

1.5.3 REGISTER.CHILDEXITDISABLE

Name

REGISTER.CHILDEXITDISABLE – Disable child exit events.

Syntax

REGISTER.CHILDEXITDISABLE;

Description

The REGISTER.CHILDEXITDISABLE statement is used to disable child process exit events enabled by REGISTER.CHILDEXITENABLE.

See also

REGISTER.CHILDEXITENABLE.

1.5.4 REGISTER.CHILDEXITENABLE

Name

REGISTER.CHILDEXITENABLE – Enable child exit events.

Syntax

REGISTER.CHILDEXITENABLE;

Description

The REGISTER.CHILDEXITENABLE statement is used to enable child process exit events. When a child process has been terminated, the REGISTER.WAIT statement returns the event type `$Register_WaitChildExit`. When an event of this type has been delivered, child exit events are automatically disabled.

See also

REGISTER.CHILDEXITDISABLE.

1.5.5 REGISTER.ERROR

Name

REGISTER.ERROR – Function that returns the last error generated by the REGISTER extension.

Syntax

REGISTER.ERROR()

Description

The REGISTER.ERROR function returns the error string for the last error generated by the REGISTER extension. If no error has been generated, an empty string is returned.

1.5.6 REGISTER.READADD

Name

REGISTER.READADD – Register for reading.

Syntax

REGISTER.READADD *sock-desc* [*EXTRAID extra-id*];

Description

The REGISTER.READADD statement is used to register interest in reading from the channel specified by the socket descriptor *sock-desc*. An extra ID—*extra-id*—can also be supplied. If the argument *extra-id* is not specified, the value 0 is used.

The effect of the REGISTER.READADD statement will remain until one of the REGISTER.READCLEAR or REGISTER.READREMOVE statements is called to undo the REGISTER.READADD call. The REGISTER.WAIT statement returns both the socket descriptor and the extra ID when there is something to read from the channel specified by *sock-desc*.

Both *sock-desc* and *extra-id* are *expressions* that are converted to integers.

Errors

The following value can be assigned to `$Error`:

`$Error_RegisterReadaddIllSockdesc`

The *sock-desc* parameter is an illegal socket descriptor.

Examples

The following example opens two channels to the OFTP server, (OFTP.OpenServer), tells the server that it is interested in received files and EERPs (OFTP.ReceiveItem), and waits for a file, or an EERP, to arrive:

```
INCLUDE "eoftp.s4h";
DECLARE VARIABLE $sd1 INTEGER;
DECLARE VARIABLE $sd2 INTEGER;
DECLARE VARIABLE $recvId INTEGER;
DECLARE VARIABLE $type INTEGER;

$sd1 = OFTP.OpenServer();
$sd2 = OFTP.OpenServer();
OFTP.ReceiveItem $sd1 Type $Oftp_RecvFile;
OFTP.ReceiveItem $sd2 Type $Oftp_RecvEERP;
REGISTER.ReadAdd $sd1;
REGISTER.ReadAdd $sd2;
REGISTER.Wait $type ID $recvId;.
```

See also

REGISTER.READCLEAR, REGISTER.READREMOVE, REGISTER.WAIT

1.5.7 REGISTER.READCLEAR

Name

REGISTER.READCLEAR – Remove everything registered for reading.

Syntax

```
REGISTER.READCLEAR;
```

Description

The REGISTER.READCLEAR statement is used to undo all REGISTER.READADD calls.

Examples

The following example opens one channel to the OFTP server (OFTP.OpenServer), and tells the server that it is interested in received files (OFTP.ReceiveItem). A 10-second time-out is set up before the REGISTER.Wait statement is called. If the time-out expires before any files have been received, the REGISTER.ReadClear statement is called in order to undo the REGISTER.ReadAdd statement.

Otherwise, the REGISTER.TimeoutClear statement is used to remove the time-out.

```
INCLUDE "eoftp.s4h";
DECLARE VARIABLE $sd INTEGER;
DECLARE VARIABLE $id INTEGER;
DECLARE VARIABLE $type INTEGER;

$sd = OFTP.OpenServer();
OFTP.ReceiveItem $sd Type $Oftp_RecvFile;
REGISTER.ReadAdd $sd;
$id = REGISTER.TimeoutAdd(10);
```

```

REGISTER.Wait $type;
IF $type = $Register_WaitTimeout {
    REGISTER.ReadClear;
    .
    .
    .
}
ELSE {
    REGISTER.TimeoutClear;
    .
    .
    .
}
.
.
.

```

See also

REGISTER.READADD, REGISTER.READREMOVE.

1.5.8 REGISTER.READREMOVE

Name

REGISTER.READREMOVE – Remove something that has been registered for reading.

Syntax

REGISTER.READREMOVE *sock-desc*;

Description

The REGISTER.READREMOVE statement is used to remove something registered for reading using the REGISTER.READADD statement. The *sock-desc* parameter should equal the *sock-desc* parameter used when the corresponding REGISTER.READADD statement was called.

The *sock-desc* parameter is an *expression* that is converted to an integer.

Errors

The following values can be assigned to \$Error:

- \$Error_RegisterReadremoveIllSockdesc
The *sock-desc* parameter is an illegal socket descriptor.
- \$Error_RegisterReadremoveUnknownSockdesc
The *sock-desc* parameter is an unknown socket descriptor.

Examples

The following example opens one channel to the OFTP server (OFTP.OpenServer). It tells the server that it is interested in received files (OFTP.ReceiveItem), and waits for a file to arrive. When a file is received, the REGISTER.ReadRemove statement is called to undo the REGISTER.ReadAdd call. Note that in this example, the REGISTER.ReadAdd, REGISTER.Wait and REGISTER.ReadRemove sequences can be removed, because there is only one channel to read from.

```

INCLUDE "eoftp.s4h";
DECLARE VARIABLE $sd INTEGER;

$sd = OFTP.OpenServer();
OFTP.ReceiveItem $sd Type $Oftp_RecvFile;
REGISTER.ReadAdd $sd;
REGISTER.Wait;
REGISTER.ReadRemove $sd;
.
.
.

```

See also

REGISTER.READADD, REGISTER.READCLEAR.

1.5.9 REGISTER.SECONDS

Name

REGISTER.SECONDS – Function that returns the current time.

Syntax

REGISTER.SECONDS()

Description

The REGISTER.SECONDS function returns an integer value representing the number of seconds that have elapsed since some unspecified starting time.

1.5.10 REGISTER.TIMEOUTADD

Name

REGISTER.TIMEOUTADD – Function that registers a time-out.

Syntax

REGISTER.TIMEOUTADD(*timeout*[, *extra-id*]);

Description

The REGISTER.TIMEOUTADD function is used to register a time-out. It returns an ID that is used to identify the time-out. The *timeout* argument specifies the length of the time-out in seconds. An extra ID—*extra-id*—can also be supplied. If the argument *extra-id* is not specified, the value 0 is used.

The effect of the REGISTER.TIMEOUTADD statement remains until the time-out expires, or one of the REGISTER.TIMEOUTCLEAR or REGISTER.TIMEOUTREMOVE statements is used to remove the time-out. When the time-out expires, both the ID returned by this function and the extra ID are returned by the REGISTER.WAIT statement.

Both the *timeout* and the *extra-id* arguments are *expressions* that are converted to integers. The returned ID is an integer.

Examples

Refer to the examples for REGISTER.READCLEAR, REGISTER.TIMEOUTREMOVE, and REGISTER.WAIT.

See also

REGISTER.TIMEOUTCLEAR, REGISTER.TIMEOUTREMOVE, REGISTER.WAIT.

1.5.11 REGISTER.TIMEOUTCLEAR

Name

REGISTER.TIMEOUTCLEAR – Remove all time-outs.

Syntax

REGISTER.TIMEOUTCLEAR;

Description

The REGISTER.TIMEOUTCLEAR statement is used to undo all REGISTER.TIMEOUTADD calls.

Examples

Refer to the REGISTER.READCLEAR example.

See also

REGISTER.TIMEOUTADD, REGISTER.TIMEOUTREMOVE.

1.5.12 REGISTER.TIMEOUTREMOVE

Name

REGISTER.TIMEOUTREMOVE – Remove one time-out.

Syntax

REGISTER.TIMEOUTREMOVE *timeout-id*;

Description

The REGISTER.TIMEOUTREMOVE statement is used to remove a time-out added using the REGISTER.TIMEOUTADD function. The *timeout-id* parameter should equal the value returned from the corresponding REGISTER.TIMEOUTADD function call.

The *timeout-id* parameter is an *expression* that is converted to an integer.

Errors

The following value can be assigned to `$Error`:

```
$Error_RegisterTimeoutremoveUnknownId
```

The *timeout-id* parameter is an unknown time-out identifier.

Examples

The following example opens one channel to the OFTP server (OFTP.OpenServer), and tells the server that it is interested in receiving EERPs (OFTP.ReceiveItem). A 10-second time-out is set up before the REGISTER.Wait statement is called. If the time-out expires before any files have been received, the REGISTER.ReadRemove statement is called to undo the REGISTER.ReadAdd statement. Otherwise, the REGISTER.TimeoutRemove statement is used to remove the time-out.

```
INCLUDE "eoftp.s4h";
DECLARE VARIABLE $sd INTEGER;
DECLARE VARIABLE $id INTEGER;
DECLARE VARIABLE $type INTEGER;

$sd = OFTP.OpenServer();
OFTP.ReceiveItem $sd Type $Oftp_RecvEERP;
REGISTER.ReadAdd $sd;
$id = REGISTER.TimeoutAdd(10);
REGISTER.Wait $type;
IF $type = $Register_WaitTimeout {
    REGISTER.ReadRemove $sd;
    .
    .
    .
}
ELSE {
    REGISTER.TimeoutRemove $id;
    .
    .
    .
}
.
.
.
```

See also

REGISTER.TIMEOUTADD, REGISTER.TIMEOUTCLEAR.

1.5.13 REGISTER.VERSION

Name

REGISTER.VERSION – Function that returns the version number of the REGISTER extension.

Syntax

```
REGISTER.VERSION()
```

Description

The REGISTER.VERSION function returns the version number of the REGISTER extension as a string. The format of the string is *n1.n2.n3*.

1.5.14 REGISTER.WAIT

Name

REGISTER.WAIT – Wait for an event.

Syntax

REGISTER.WAIT [*type* [ID *id*] [EXTRAID *extra-id*]];

Description

The REGISTER.WAIT statement is used to wait for a registered event to occur. When there is something to read, or it is possible to write without being blocked, or a time-out expires, this statement returns.

The *type* parameter is a *variable* that receives the type of event. It has one of the following values (these constants are declared in the *ereg.s4h* file):

<code>\$Register_WaitTimeout</code>	A time-out event.
<code>\$Register_WaitWrite</code>	A write event.
<code>\$Register_WaitRead</code>	A read event.
<code>\$Register_WaitChildExit</code>	A child process exit event.

The *id* parameter is a variable that receives the time-out ID or the write, or read socket descriptor for an event. The *extra-id* parameter is also a variable that receives the extra ID supplied when the time-out, write, or read was registered.

Errors

The following values can be assigned to `$Error`:

- `$Error_RegisterWaitNothing`
Nothing to wait for.
- `$Error_RegisterWaitSelectError`
Error when calling `select`.
- `$Error_RegisterWaitSelectNothing`
The `select` call returned nothing.

Examples

The following example registers one socket descriptor for reading and one for writing, and sets up a time-out of 25 seconds. Then it waits for an event to occur:

```
DECLARE VARIABLE $rsd INTEGER;
DECLARE VARIABLE $wsd INTEGER;
DECLARE VARIABLE $id INTEGER;
DECLARE VARIABLE $type INTEGER;
.
.
.
REGISTER.ReadAdd $rsd;
```



```
REGISTER.WriteAdd $wsd;  
$id = REGISTER.TimeoutAdd(25);  
REGISTER.Wait $type;  
.  
.  
.
```

See also

REGISTER.READADD, REGISTER.TIMEOUTADD, REGISTER.WRITEADD.

Notes

If a timeout of zero is set up and any socket descriptors are registered for reading and/or writing, the timeout is only returned if there is nothing to read or it is not possible to write without being blocked.

1.5.15 REGISTER.WRITEADD

Name

REGISTER.WRITEADD – Register for writing.

Syntax

REGISTER.WRITEADD *sock-desc* [EXTRAID *extra-id*];

Description

The REGISTER.WRITEADD statement is used to register interest in writing to the channel specified by the socket descriptor *sock-desc*. An extra ID—*extra-id*—can also be supplied. If the argument *extra-id* is not specified, the value 0 is used.

The effect of the REGISTER.WRITEADD statement remains until one of the REGISTER.WRITECLEAR or REGISTER.WRITEREMOVE statements is called to undo the REGISTER.WRITEADD call. Both the socket descriptor and the extra ID are returned by the REGISTER.WAIT statement when it is possible to write to the channel specified by *sock-desc* without being blocked.

Both *sock-desc* and *extra-id* are *expressions* that are converted to integers.

Errors

The following value can be assigned to `$Error`:

`$Error_RegisterWriterremoveIllSockdesc`

The *sock-desc* parameter is an illegal socket descriptor.

Examples

```
REGISTER.WriteAdd $sd ExtraId $extraId;
```

See also

REGISTER.WAIT, REGISTER.WRITECLEAR, REGISTER.WRITEREMOVE.

1.5.16 REGISTER.WRITECLEAR

Name

REGISTER.WRITECLEAR – Remove everything registered for writing.

Syntax

REGISTER.WRITECLEAR;

Description

The REGISTER.WRITECLEAR statement is used to undo all REGISTER.WRITEADD calls.

Examples

```
REGISTER.WRITECLEAR;
```

See also

REGISTER.WRITEADD, REGISTER.WRITEREMOVE.

1.5.17 REGISTER.WRITEREMOVE

Name

REGISTER.WRITEREMOVE – Remove something that has been registered for writing.

Syntax

REGISTER.WRITEREMOVE *sock-desc*;

Description

The REGISTER.WRITEREMOVE statement is used to remove something registered for writing using the REGISTER.WRITEADD statement. The *sock-desc* parameter should equal the *sock-desc* parameter used when the REGISTER.WRITEADD statement was called.

The *sock-desc* parameter is an *expression* that is converted to an integer.

Errors

The following values can be assigned to `$Error`;

- `$Error_RegisterWriteremoveIllSockdesc`
The *sock-desc* parameter is an illegal socket descriptor.
- `$Error_RegisterWriteremoveUnknownSockdesc`
The *sock-desc* parameter is an unknown socket descriptor.

Examples

```
REGISTER.WriteRemove $sd;
```

See also

REGISTER.WRITEADD, REGISTER.WRITECLEAR.

1.6 The SOCKET extension

This chapter describes the Message Builder SOCKET extension module. The SOCKET extension module contains functions and statements for handling sockets. It is often convenient to use the REGISTER extension module together with the SOCKET extension module.

The error type generated by the SOCKET functions and statements is SOCKET. The following program fragment demonstrates how to handle SOCKET errors:

```
TRY {
    $socket = SOCKET.CONNECT($host, $service);
}
CATCH $exception
WHEN "SOCKET" {
    PRINT "Error when trying to open socket\n";
}
```

All of the SOCKET extension module functions and statements are declared in the library file *esocket.s4*. A number of constants (for example, error codes) can be found in the include file *esocket.s4h*.

Since the SOCKET extension uses the MODULE_INIT feature for initialization, no functions or statements from the SOCKET extension should be called from other MODULE_INIT statements.

1.6.1 Summary of functions and statements

Function	Used For
SOCKET.ACCEPT	Accepting incoming connection requests.
SOCKET.CONNECT	Connecting to a server.
SOCKET.NCONNECT	Beginning to connect to a server (nonblocking mode).
SOCKET.DISCONNECT	Closing a socket.
SOCKET.GETSOCKERROR	Obtaining the last error for a socket.
SOCKET.ERROR	Returning the last error message.
SOCKET.SETNODELAY	Enabling immediate delivery of written data.
SOCKET.GET	Reading data from a socket.
SOCKET.LISTEN	Listening for connections.
SOCKET.NGET	Reading data from a socket in nonblocking mode.
SOCKET.TGET	Reading data from a socket with a timeout.
SOCKET.NPUT	Writing data to a socket in nonblocking mode.
SOCKET.PUT	Writing data to a socket.
SOCKET.VERSION	Returning version number.
SOCKET.GETPEERNAME	Returning the remote IP address for a connected socket.

1.6.2 SOCKET.ACCEPT

Name

SOCKET.ACCEPT – Accept a connection request.

Syntax

SOCKET.ACCEPT(*listen-socket*);

Description

The SOCKET.ACCEPT function is used to accept incoming connection requests.

The *listen-socket* argument is a socket descriptor previously returned by the SOCKET.LISTEN function.

An integer socket descriptor is returned. This socket descriptor can be used to read data from (SOCKET.GET and SOCKET.NGET) and write data to (SOCKET.PUT and SOCKET.NPUT) the other side. It can also be used to register the connection for reading and/or writing using the REGISTER extension module.

Errors

The following values can be assigned to `$Error`:

- `$Error_SocketAcceptUnknownSocket`
The *listen-socket* argument is not a known socket.
- `$Error_SocketAcceptAcceptFailed`
The *accept* system call failed.

Examples

For an example, refer to the SOCKET.LISTEN function, paragraph [1.4.6](#).

See also

SOCKET.LISTEN.

1.6.3 SOCKET.CONNECT

Name

SOCKET.CONNECT – Function that makes a connection.

Syntax

SOCKET.CONNECT(*host*, *service*[, *bindHost* [, *bindPort*]]);

Description

The SOCKET.CONNECT function makes a connection to the socket specified by the *host* and *service* arguments. Both arguments are *expressions* that are converted to strings. The *host*

argument is the name or the IP address (using dot notation) for the host to connect to, and *service* is a service name or a decimal number specifying the TCP port number to use. The optional *bindHost* and *bindPort* arguments can be used to specify the source host (that is, the network interface through which the connection will be attempted, by specifying the local host's address for that interface) and source port of the connection attempt.

An integer socket descriptor is returned. This socket descriptor can be used to read data from (SOCKET.GET and SOCKET.NGET) and write data to (SOCKET.PUT and SOCKET.NPUT) the other side. It can also be used to register the connection for reading and/or writing using the REGISTER extension module.

Errors

The following values can be assigned to `$Error`:

- `$Error_SocketConnectUnknownHostName`
The *host* argument is not a known host name.
- `$Error_SocketConnectUnknownServiceName`
The *service* argument is not a known service name.
- `$Error_SocketConnectSocketFailed`
The *socket* system call failed.
- `$Error_SocketConnectBindFailed`
The *bind* system call failed.
- `$Error_SocketConnectConnectFailed`
The *connect* system call failed.
- `Error_SocketConnectUnknownBindHostName`
The specified source host name does not exist.
- `Error_SocketConnectUnknownBindServiceName`
The specified source service name does not exist.

Examples

```
$connectSocket = SOCKET.CONNECT("localhost", 2222);
```

See also

SOCKET.DISCONNECT.

1.6.4 SOCKET.NCONNECT

Name

SOCKET.NCONNECT. - A statement that begins a nonblocking connection attempt.

Syntax

```
SOCKET.NCONNECT socket HOST host PORT port [BIND_HOST bindHost] [BIND_PORT  
bindPort] HAS_CONNECTED hasConnected;
```

Description

The SOCKET.NCONNECT statement makes a connection to the socket specified by the *host* and *port* arguments. Both arguments are *expressions* that are converted to strings. The *host*

argument is the name or the IP address (using dot notation) for the host to connect to, and *service* is a service name or a decimal number specifying the TCP port number to use. The optional *bindHost* and *bindPort* arguments can be used to specify the source host (that is, the network interface through which the connection will be attempted, by specifying the local host's address for that interface) and source port of the connection attempt.

The *socket* associated with the connection attempt is returned in the *socket* parameter. If the connection has succeeded immediately, the *hasConnected* parameter will be set to true; otherwise the *socket* must be registered for both write and (on Windows) exception interests; if an exception event was received, Windows indicates that the connection has failed and SOCKET.DISCONNECT must be called on the *socket*; if a write event was received, SOCKET.GETSOCKERROR must be called on non-Windows platforms to determine whether the connection has failed (and SOCKET.DISCONNECT must be called), or whether it has succeeded.

Errors

The same errors as for SOCKET.CONNECT apply here.

1.6.5 SOCKET.DISCONNECT

Name

SOCKET.DISCONNECT – Close a socket.

Syntax

SOCKET.DISCONNECT *socket*;

Description

The SOCKET.DISCONNECT statement is used to close a socket.

The *socket* parameter is a socket descriptor returned by one of the SOCKET.ACCEPT, SOCKET.CONNECT or SOCKET.LISTEN functions.

Errors

The following value can be assigned to `$Error`:

`$Error_SocketDisconnectUnknownSocket`

The *socket* parameter is not a known socket.

Examples

```
SOCKET.DISCONNECT $socket;
```

See also

SOCKET.ACCEPT, SOCKET.CONNECT, SOCKET.LISTEN.

1.6.6 SOCKET.GETSOCKERROR

Name

SOCKET.GETSOCKERROR - Returns the last error that occurred on a socket.

Syntax

SOCKET.GETSOCKERROR(*socket*)

Description

This function returns an error string representing the last operating system error that occurred on the specified *socket*, or an empty string if the last operation has succeeded. The *socket* parameter must be returned by SOCKET.CONNECT, SOCKET.NCONNECT, SOCKET.LISTEN or SOCKET.ACCEPT.

1.6.7 SOCKET.SETNODELAY

Name

SOCKET.SETNODELAY - Enable or disable the TCP_NODELAY option on a socket.

Syntax

SOCKET.SETNODELAY *socket* NODELAY *nodelay*;

Description

If the *nodelay* input parameter is non-zero, the TCP_NODELAY option is enabled for the specified *socket*. Otherwise, it is disabled.

The TCP_NODELAY option is a feature of the operating system TCP/IP stack which causes any amount of data written to a socket to be sent as a TCP packet immediately if possible. By default it is disabled, that is, the operating system implements the Nagle algorithm, delaying small amounts of data to increase network efficiency. RFC 1122 (<http://tools.ietf.org/html/rfc1122>) specifies that the Nagle algorithm delays data until a full packet is locally buffered or until outstanding data is acknowledged by the remote host, whichever comes first.

1.6.8 SOCKET.ERROR

Name

SOCKET.ERROR – Function that returns the last error generated by the SOCKET extension.

Syntax

SOCKET.ERROR()

Description

The `SOCKET.ERROR` function returns the error string for the last error generated by the `SOCKET` extension. If no error has been generated, an empty string is returned.

1.6.9 SOCKET.GET

Name

`SOCKET.GET` – Function that reads data from a socket.

Syntax

`SOCKET.GET(connect-socket, size)`

Description

The `SOCKET.GET` function reads data from a socket. The *connect-socket* argument is a socket descriptor previously returned by one of the `SOCKET.ACCEPT` or `SOCKET.CONNECT` functions. The *size* argument is the maximum number of bytes to read. Both arguments are *expressions* that are converted to integers.

The `SOCKET.GET` function blocks until there is anything to read from the socket. A string containing the read data is returned. Note that the returned string may be shorter than *size*. No error is generated if end of file is reached, that is, the other side has closed down the connection. Instead, an empty string is returned.

Errors

The following values can be assigned to `$Error`:

- `$Error_SocketGetUnknownSocket`
The *connect-socket* argument is not a known socket.
- `$Error_SocketGetBlockingFailed`
Failed to set blocking mode.
- `$Error_SocketGetRecvFailed`
The *recv* system call failed.

Examples

The following example makes a connection and reads data from it until the other side closes it:

```
DECLARE VARIABLE $socket INTEGER;
DECLARE VARIABLE $data STRING;

$socket = SOCKET.CONNECT("host", "port");
REPEAT {
    $data = SOCKET.GET($socket, 256);
    PRINT $data;
} WHILE $data <> "";
SOCKET.DISCONNECT $socket;
```

See also

`SOCKET.NGET`, `SOCKET.NPUT`, `SOCKET.PUT`.

1.6.10 SOCKET.LISTEN

Name

SOCKET.LISTEN – Listen for connections.

Syntax

SOCKET.LISTEN(*service*[, *backlog*])

Description

The SOCKET.LISTEN function creates a socket that can be used to listen for connections; that is, it creates a server socket.

The *service* argument is an *expression* that is converted to a string. It is a service name or a decimal number specifying the TCP port number to use.

The *backlog* argument specifies the desirable queue length for pending connections. The default value is 2.

An integer socket descriptor is returned. This socket descriptor can be used to wait for incoming connections (SOCKET.ACCEPT). It can also be used to register the listen socket for reading using the REGISTER extension module.

Errors

The following values can be assigned to `$Error`:

- `$Error_SocketListenUnknownServiceName`
The *service* argument is not a known service name.
- `$Error_SocketListenSocketFailed`
The *socket* system call failed.
- `$Error_SocketListenBindFailed`
The *bind* system call failed.
- `$Error_SocketListenListenFailed`
The *listen* system call failed.

Examples

The following example creates a listen socket. Then it enters a loop that waits for connections. When a connection is obtained, data is read from it until the other side closes it.

```
DECLARE VARIABLE $listenSocket INTEGER;
DECLARE VARIABLE $socket INTEGER;
DECLARE VARIABLE $data STRING;

$listenSocket = SOCKET.LISTEN("17000");
WHILE 1 {
    $socket = SOCKET.ACCEPT($listenSocket);
    REPEAT {
        $data = SOCKET.GET($socket, 256);
        PRINT $data;
    } WHILE $data <> "";
    SOCKET.DISCONNECT $socket;
}
```

See also

SOCKET.ACCEPT, SOCKET.DISCONNECT.

1.6.11 SOCKET.NGET

Name

SOCKET.NGET – Function that reads data from a socket in nonblocking mode.

Syntax

SOCKET.NGET(*connect-socket*, *size*)

Description

The SOCKET.NGET function reads data from a socket in nonblocking mode. The *connect-socket* argument is a socket descriptor previously returned by one of the SOCKET.ACCEPT or SOCKET.CONNECT functions. The *size* argument is the maximum number of bytes to read. Both arguments are *expressions* that are converted to integers.

A string containing the read data is returned. Note that the returned string may be shorter than *size*. An empty string is returned if there are no data waiting to be read.

Errors

The following values can be assigned to `$Error`:

- `$Error_SocketNgetUnknownSocket`
The *connect-socket* argument is not a known socket.
- `$Error_SocketNgetNonblockingFailed`
Failed to set nonblocking mode.
- `$Error_SocketNgetPeerClosedDown`
The other side has closed down.
- `$Error_SocketNgetRecvFailed`
The *recv* system call failed.

Examples

The following example makes a connection and reads data in nonblocking mode until the other side closes the connection:

```
DECLARE VARIABLE $socket INTEGER;
DECLARE VARIABLE $data STRING;

$socket = SOCKET.CONNECT("host", "port");
WHILE 1 {
    TRY {
        $data = SOCKET.NGET($socket, 256);
    }
    CATCH $exception
    WHEN "SOCKET" {
        IF $Error <> $Error_SocketNgetPeerClosedDown {
            THROW $exception;
        }
        /* the other side has closed the connection /
```

```

        BREAK;
    }
    IF $data = "" {
        /* wait a while before reading again */
        SLEEP 2;
    }
    ELSE {
        PRINT $data;
    }
}
SOCKET.DISCONNECT $socket;

```

The SOCKET.NGET function should be used together with the REGISTER extension. The SLEEP statement has been used here instead in order to simplify the example.

See also

SOCKET.GET, SOCKET.NPUT, SOCKET.PUT.

Notes

The SOCKET.NGET function is intended to be used together with the REGISTER extension.

1.6.12 SOCKET.TGET

Name

SOCKET.TGET – Function that reads data from a socket with a timeout.

Syntax

SOCKET.TGET(*connect-socket*, *size*, *seconds*, *microseconds*)

Description

The SOCKET.TGET function reads data from a socket. If no data is received in the given amount of time (given in the *seconds* and *microseconds* parameters), an exception is thrown.

The *connect-socket* argument is a socket descriptor previously returned by one of the SOCKET.ACCEPT or SOCKET.CONNECT functions. The *size* argument is the maximum number of bytes to read.

All four arguments are *expressions* that are converted to integers.

A string containing the read data is returned. Note that the returned string may be shorter than *size*. This function never returns an empty string.

Errors

The following values can be assigned to `$Error` when the function throws the `$Socket_ExceptionName` exception:

- `$Error_SocketTgetUnknownSocket`
The *connect-socket* argument is not a known socket.
- `$Error_SocketTgetNonblockingFailed`

Failed to set nonblocking mode.

- `$Error_SocketTgetPeerClosedDown`
The other side has closed down.
- `$Error_SocketTgetRecvFailed`
The *recv* system call failed.
- `$Error_SocketTgetSelectFailed`
The *select* system call failed.
- `$Error_SocketTgetWaitExpired`
The given amount of time has passed without a single character being received.
- `$Error_SocketTgetTooManyMicroseconds`
The *microseconds* parameter is larger than 1000000
- `$Error_SocketTgetNegativeTime`
The *seconds* parameter is negative, the *microseconds* parameter is negative, or an integer overflow occurred.

1.6.13 SOCKET.NPUT

Name

SOCKET.NPUT – Function that writes data to a socket in nonblocking mode.

Syntax

SOCKET.NPUT(*connect-socket*, *data*)

Description

The SOCKET.NPUT function writes data to a socket in nonblocking mode. The *connect-socket* argument is a socket descriptor previously returned by one of the SOCKET.ACCEPT or SOCKET.CONNECT functions. The *data* argument is the data to write. Both arguments are *expressions* that are converted to an integer and a string, respectively.

The number of bytes actually written is returned as an integer value.

Note: If it is not possible to write without being blocked, zero is returned.

Errors

The following values can be assigned to `$Error`:

- `$Error_SocketNputUnknownSocket`
The *connect-socket* argument is not a known socket.
- `$Error_SocketNputNonblockingFailed`
Failed to set nonblocking mode.
- `$Error_SocketNputPeerClosedDown`
The other side has closed down.
- `$Error_SocketNputSendFailed`
The *send* system call failed.

Examples

The following example makes a connection and writes some data to it (note that the amount of data written may be zero). Then the connection is closed:

```
DECLARE VARIABLE $socket INTEGER;
DECLARE VARIABLE $count INTEGER;

$socket = SOCKET.CONNECT("host", "port");
$count = SOCKET.NPUT($socket, "Example data\n");
PRINT "Wrote " & $count & " bytes\n";
SOCKET.DISCONNECT $socket;
```

See also

SOCKET.GET, SOCKET.NGET, SOCKET.PUT.

Notes

If the SOCKET.NPUT function is intended to be used together with the REGISTER extension.

1.6.14 SOCKET.PUT

Name

SOCKET.PUT – Function that writes data to a socket.

Syntax

SOCKET.PUT(*connect-socket*, *data*)

Description

The SOCKET.PUT function writes data to a socket. The *connect-socket* argument is a socket descriptor previously returned by one of the SOCKET.ACCEPT or SOCKET.CONNECT functions. The *data* argument is the data to write. Both arguments are *expressions* that are converted to an integer and a string, respectively.

The SOCKET.PUT function blocks until some data can be written to the socket. The number of bytes actually written is returned as an integer value. If the other side has closed down the connection, SOCKET.PUT may return 0.

Errors

The following values can be assigned to `$Error`:

- `$Error_SocketPutUnknownSocket`
The *connect-socket* argument is not a known socket.
- `$Error_SocketPutBlockingFailed`
Failed to set blocking mode.
- `$Error_SocketPutSendFailed`
The *send* system call failed.

Examples

The following example makes a connection and writes some data to it. Then the connection is closed:

```
DECLARE VARIABLE $socket INTEGER;
DECLARE VARIABLE $count INTEGER;
```

```
$socket = SOCKET.CONNECT("host", "port");  
$count = SOCKET.PUT($socket, "Example data\n");  
SOCKET.DISCONNECT $socket;
```

See also

SOCKET.GET, SOCKET.NGET, SOCKET.NPUT.

1.6.15 SOCKET.VERSION

Name

SOCKET.VERSION – Function that returns the version number of the SOCKET extension.

Syntax

SOCKET.VERSION()

Description

The SOCKET.VERSION function returns the version number of the SOCKET extension as a string. The format of the string is *n1.n2.n3*.

1.6.16 SOCKET.GETPEERNAME

Name

SOCKET.GETPEERNAME. - Returns the IP address of the other end of a connection.

Syntax

SOCKET.GETPEERNAME(*socket*)

Description

This function returns a string representing IP address of the other end of a connection (that is, the remote address). The *socket* parameter must be returned by SOCKET.CONNECT, SOCKET.NCONNECT or SOCKET.ACCEPT.

Errors

The following values can be assigned to `$Error`:

- `$Error_SocketGetpeernameUnknownSocket`
The *socket* argument is not a known socket.
- `$Error_SocketGetpeernameFailed`
Failed to obtain remote IP address.

1.7 The UNIQUE extension

This chapter describes the UNIQUE extension. The UNIQUE extension implements a unique ID feature.

The error type generated by the UNIQUE extension is UNIQUE. The following program fragment demonstrates how to handle UNIQUE errors:

```
TRY {  
    $id = UNIQUE.FILEOPEN("uniq");  
}  
CATCH $exception  
WHEN UNIQUE.$ExceptionName {  
    PRINT "Failed to open unique ID file\n";  
}
```

All the UNIQUE extension functions and statements are declared in the library file *eunique.s4*. A number of constants (for example, error codes) which are useful when using the UNIQUE extension can be found in the include file *eunique.s4h* (the `UNIQUE.$ExceptionName` variable is declared in this file).

Since the UNIQUE extension uses the `MODULE_INIT` feature for initialization, no functions or statements from the UNIQUE extension should be called from other `MODULE_INIT` statements.

1.7.1 Summary of functions and statements

Function/ Statement	Used For
UNIQUE.ADD	Adding a new identity type.
UNIQUE.DELETE	Deleting an identity type.
UNIQUE.FILECLOSE	Closing and open unique ID file.
UNIQUE.FILECREATE	Creating an empty unique ID file.
UNIQUE.FILEOPEN	Opening an existing unique ID file.
UNIQUE.GETID	Returning unique Ids.
UNIQUE.LIST	Returning information about a unique ID file.
UNIQUE.VERSION	Returning version number.
UNIQUE.SETID	Changing an identity type.

1.7.2 UNIQUE.ADD

Name

UNIQUE.ADD – Add a new identity type.

Syntax

UNIQUE.ADD *id* TYPE *type* LENGTH *length* [BASE *base*] [INITIAL *initial*];

Description

The UNIQUE.ADD statement adds a new identity type to a unique ID file.

The *id* parameter is an open unique ID file identifier previously returned by the UNIQUE.FILEOPEN statement.

The *type* parameter is the new identity type. It is an *expression* that is converted to a string. The maximum identity type length is 10 characters.

The *length* parameter is the length of a unique ID of the new identity type. It is an *expression* that is converted to an integer. Values in the range 1 to 99 are allowed.

The *base* parameter is the base to be used for unique IDs of the new identity type. It is an *expression* that is converted to an integer. Values in the range 2 to 62 are allowed. The characters "A" to "Z" are used for bases greater than 10 and the characters "a" to "z" are used for bases greater than 36. The default value is 10.

The *initial* parameter is the initial value of the unique ID. It is an *expression* that is converted to a string. It must either be an empty string or a valid unique ID of length *length*. Specifying an empty string is equivalent to specifying a unique ID containing only zeroes. The first unique ID returned by UNIQUE.GETID is one greater than the initial value. The default value is an empty string.

Errors

The following values can be assigned to `$Error`:

- `UNIQUE.$Error_AddBadFileId`
The *id* parameter is not a valid open unique ID file identifier.
- `UNIQUE.$Error_AddIllegalTypeLength`
Illegal identity type length. The identity type length must be less than or equal to 10.
- `UNIQUE.$Error_AddIllegalLength`
Illegal unique ID length. The length must be in the range 1 to 99, inclusive.
- `UNIQUE.$Error_AddIllegalBase`
Illegal base. The base must be in the range 2 to 62.
- `UNIQUE.$Error_AddIllegalInitial`
The *initial* parameter is invalid.
- `UNIQUE.$Error_AddLockError`
Error when trying to lock the unique ID file.
- `UNIQUE.$Error_AddReadError`
Error when reading the unique ID file.
- `UNIQUE.$Error_AddTypeExist`
The identity type does already exist.
- `UNIQUE.$Error_AddWriteError`
Error when writing the updated unique ID file.
- `UNIQUE.$Error_AddUnlockError`
Error when trying to unlock the unique ID file.

Examples

```
DECLARE VARIABLE $id INTEGER;  
$id = UNIQUE.FILEOPEN("file.unq");  
UNIQUE.ADD $id TYPE "test" LENGTH 10;  
UNIQUE.FILECLOSE $id;
```

See also

UNIQUE.DELETE.

1.7.3 UNIQUE.DELETE

Name

UNIQUE.DELETE – Delete an identity type.

Syntax

UNIQUE.DELETE *id* TYPE *type*;

Description

The UNIQUE.DELETE statement deletes an existing identity type from a unique ID file.

The *id* parameter is an open unique ID file identifier previously returned by the UNIQUE.FILEOPEN statement.

The *type* parameter is the identity type to delete. It is an *expression* that is converted to a string.

Errors

The following values can be assigned to `$Error`:

- UNIQUE.\$Error_DeleteBadFileId
The *id* parameter is not a valid open unique ID file identifier.
- UNIQUE.\$Error_DeleteIllegalTypeLength
Illegal identity type length. The identity type length must be less than or equal to 10.
- UNIQUE.\$Error_DeleteLockError
Error when trying to lock the unique ID file.
- UNIQUE.\$Error_DeleteReadError
Error when reading the unique ID file.
- UNIQUE.\$Error_DeleteTypeNotExist
The identity type does not exist.
- UNIQUE.\$Error_DeleteWriteError
Error when writing the updated unique ID file.
- UNIQUE.\$Error_DeleteUnlockError
Error when trying to unlock the unique ID file.

Examples

```
DECLARE VARIABLE $id INTEGER;  
$id = UNIQUE.FILEOPEN("file.unq");  
UNIQUE.DELETE $id TYPE "test";  
UNIQUE.FILECLOSE $id;
```

See also

UNIQUE.ADD.

1.7.4 UNIQUE.FILECLOSE

Name

UNIQUE.FILECLOSE – Close an open unique ID file.

Syntax

UNIQUE.FILECLOSE *id*;

Description

The UNIQUE.FILECLOSE statement closes an open unique ID file.

The *id* parameter is an open unique ID file identifier previously returned by the UNIQUE.FILEOPEN statement.

Errors

The following values can be assigned to `$Error`:

UNIQUE.\$Error_FilecloseBadFileId

The *id* parameter is not a valid open unique ID file identifier.

Examples

```
DECLARE VARIABLE $id INTEGER;
$id = UNIQUE.FILEOPEN("file.unq");
.
.
.
UNIQUE.FILECLOSE $id;
```

See also

UNIQUE.FILEOPEN.

1.7.5 UNIQUE.FILECREATE

Name

UNIQUE.FILECREATE – Create a new empty unique ID file or truncate an old file.

Syntax

UNIQUE.FILECREATE *name*;

Description

The UNIQUE.FILECREATE statement creates a new empty unique ID file or truncates an existing one.

The *name* parameter is the name of the unique ID file. It is an *expression* that is converted to a string.

Errors

The following values can be assigned to `$Error`:

```
UNIQUE.$Error_FilecreateFileCreate
```

Failed to create a file named *name*.

Examples

Create a unique ID file with name *file.unq*:

```
UNIQUE.FILECREATE "file.unq";
```

See also

UNIQUE.FILEOPEN.

Notes

A unique ID file consists of lines containing 4 fields:

```
type id-len num-base used-id
```

The fields follow immediately after each other, no separator or space character separates them.

The *type* field is a 10 character long string containing the identity type. This field is case sensitive. If it contains trailing spaces, these spaces do not have to be given when referring to the identity.

The *id-len* field specifies the length of the *used-id* field.

The *num-base* field specifies the base used in *used-id*. It must be in the range 2 – 62. Capital letters are used to represent digits in the range 10 – 35 and lowercase letters are used to represent digits in the range 36 – 61. It is typically 10 (a decimal number) or 36 (the characters 0 – 9 and A – Z).

Both the *id-len* and *num-base* fields are decimal numbers of length 2 (a leading zero must be present if the value is less than 10).

The *used-id* field is the current unique identity and its initial value is normally zero.

As an example, the `FileName` identity type normally is:

```
FileName 053600000
```

1.7.6 UNIQUE.FILEOPEN

Name

UNIQUE.FILEOPEN – Function that opens an existing unique ID file.

Syntax

UNIQUE.FILEOPEN(*name*)

Description

The UNIQUE.FILEOPEN function opens an existing unique ID file.

The *name* argument is the name of the unique ID file. It is an *expression* that is converted to a string.

An integer ID that is used to refer to the open unique ID file is returned.

Errors

The following values can be assigned to `$Error`:

```
UNIQUE.$Error_FileopenFileOpen
```

Failed to open the file named *name*.

Examples

```
DECLARE VARIABLE $id INTEGER;  
$id = UNIQUE.FILEOPEN("file.unq");  
.  
.  
.  
UNIQUE.FILECLOSE $id;
```

See also

UNIQUE.FILECREATE, UNIQUE.FILECLOSE.

1.7.7 UNIQUE.GETID

Name

UNIQUE.GETID – Function that returns unique IDs.

Syntax

```
UNIQUE.GETID(id, type[, count])
```

Description

The UNIQUE.GETID function returns a string array of *count* number of unique IDs.

The *id* argument is an open unique ID file identifier previously returned by the UNIQUE.FILEOPEN statement.

The *type* argument specifies the identity type for the unique IDs. It is an *expression* that is converted to a string.

The *count* argument is the number of unique IDs to return. It is an *expression* that is converted to an integer and the default value is one.

Errors

The following values can be assigned to `$Error`:

- `UNIQUE.$Error_GetidBadFileId`
The *id* parameter is not a valid open unique ID file identifier.
- `UNIQUE.$Error_GetidIllegalTypeLength`
Illegal identity type length. The identity type length must be less than or equal to 10.
- `UNIQUE.$Error_GetidIllegalCount`
The *count* argument is negative.
- `UNIQUE.$Error_GetidLockError`
Error when trying to lock the unique ID file.
- `UNIQUE.$Error_GetidReadError`
Error when reading the unique ID file.
- `UNIQUE.$Error_GetidTypeNotExist`
The identity type does not exist.
- `UNIQUE.$Error_GetidLineInvalid`
The identity type line is invalid.
- `UNIQUE.$Error_GetidWriteError`
Error when writing the updated unique ID file.
- `UNIQUE.$Error_GetidUnlockError`
Error when trying to unlock the unique ID file.

Examples

```
DECLARE VARIABLE $id INTEGER;
DECLARE VARIABLE $uniqueId[] STRING;
$id = UNIQUE.FILEOPEN("file.unq");
$uniqueId = UNIQUE.GETID($id, "test", 7);
FOR $i = 1 TO 7 {
    PRINT $uniqueId[$i] & "\n";
}
UNIQUE.FILECLOSE $id;
```

1.7.8 UNIQUE.LIST

Name

UNIQUE.LIST – Function that returns information about a unique ID file.

SYNTAX

UNIQUE.LIST(*id*)

Description

The UNIQUE.LIST function returns information about all unique IDs in a unique ID file. An array of records, one for each unique ID, is returned.

The *id* argument is an open unique ID file identifier previously returned by the UNIQUE.FILEOPEN statement.

The returned records are of type `UNIQUE.List`:

```
DECLARE PUBLIC RECORD List {
    DECLARE PUBLIC FIELD $Type STRING;
    DECLARE PUBLIC FIELD $Base INTEGER;
    DECLARE PUBLIC FIELD $Value STRING;
}
```

The `$Type` field contains the identity type, the `$Base` field is the base, and the `$Value` field contains the current value of the unique ID (the next value returned by `UNIQUE.GETID` is one greater than this value).

Errors

The following values can be assigned to `$Error`:

- `UNIQUE.$Error_ListBadFileId`
The *id* parameter is not a valid open unique ID file identifier.
- `UNIQUE.$Error_ListLockError`
Error when trying to lock the unique ID file.
- `UNIQUE.$Error_ListReadError`
Error when reading the unique ID file.
- `UNIQUE.$Error_ListUnlockError`
Error when trying to unlock the unique ID file.
- `UNIQUE.$Error_ListInvalidFile`
The contents of the unique ID file is invalid.

Examples

```
DECLARE VARIABLE $id INTEGER;
DECLARE VARIABLE $list[] RECORD UNIQUE.List;
$id = UNIQUE.FILEOPEN("file.unq");
$list = UNIQUE.LIST($id);
FOR $i = 1 TO ARRAYSIZE($list) {
    PRINT $list[$i].$Type & " " & $list[$i].$Base & " " &
        $list[$i].$Value & "\n";
}
UNIQUE.FILECLOSE $id;
```

1.7.9 UNIQUE.VERSION

Name

`UNIQUE.VERSION` – Function that returns the version number of the unique ID extension.

Syntax

`UNIQUE.VERSION()`

Description

The `UNIQUE.VERSION` function returns the version number of the unique ID extension as a string. The format of the string is *n1.n2.n3*.

1.7.10 UNIQUE.SETID

Name

`UNIQUE.SETID` – Change the characteristics of a unique identity type

Syntax

UNIQUE.SETID *id* TYPE *type* LENGTH *length* [BASE *base*] [INITIAL *initial*];

Description

The UNIQUE.SETID statement changes an identity type in a unique ID file.

The *id* parameter is an open unique ID file identifier previously returned by the UNIQUE.FILEOPEN statement.

The *type* parameter is the name of the identity type. It is an *expression* that is converted to a string.

The *length* parameter is the length of a unique ID of the identity type. It is an *expression* that is converted to an integer. Values in the range **1** to **99** are allowed.

The *base* parameter is the base to be used for unique IDs of the identity type. It is an *expression* that is converted to an integer. Values in the range **2** to **62** are allowed. The characters **A** to **Z** are used for bases greater than **10** and the characters **a** to **z** are used for bases greater than **36**. The default value is **10**.

The *initial* parameter is the initial value of the unique ID. It is an *expression* that is converted to a string. It must either be an empty string or a valid unique ID of length *length*. Specifying an empty string is equivalent to specifying a unique ID containing only zeroes. The first unique ID returned by UNIQUE.GETID is one greater than the initial value. The default value is an empty string.

Errors

The following values can be assigned to \$Error:

Errors

The following values can be assigned to \$Error:

- UNIQUE.\$Error_SetIdadFileId
The *id* parameter is not a valid open unique ID file identifier.
- UNIQUE.\$Error_SetIdIllegalTypeLength
Illegal identity type length. The identity type length must be less than or equal to **10**.
- UNIQUE.\$Error_SetIdIllegalLength
Illegal unique ID length. The length must be in the range **1** to **99**, inclusive.
- UNIQUE.\$Error_SetIdIllegalBase
Illegal base. The base must be in the range **2** to **62**.
- UNIQUE.\$Error_SetIdIllegalInitial
The *initial* parameter is invalid.
- UNIQUE.\$Error_SetIdLockError
Error when trying to lock the unique ID file.
- UNIQUE.\$Error_SetIdReadError
Error when reading the unique ID file.
- UNIQUE.\$Error_SetIdTypeNotExist
The identity type does not exist.
- UNIQUE.\$Error_SetIdWriteError
Error when writing the updated unique ID file.
- UNIQUE.\$Error_SetIdUnlockError

Error when trying to unlock the unique ID file.

1.8 The NET extension

This chapter describes the NET extension. The NET extension implements a number of network routines.

The error type generated by the NET extension is NET. The following program fragment demonstrates how to take care of NET errors:

```
TRY {  
    $name = NET.GETHOSTNAME();  
}  
CATCH $exception  
WHEN NET.$ExceptionName {  
    PRINT "Failed to get host name\n";  
}
```

All the NET extension functions and statements are declared in the library file *enet.s4*.

A number of constants (for example, error codes) which are useful when using the NET extension can be found in the include file *enet.s4h* (the `NET.$ExceptionName` variable is declared in this file).

1.8.1 Summary of functions and statements

Function/ Statement	Used For
NET.GETHOSTNAME	Returning the name of the current host.
NET.GETHOSTBYNAME	Getting host parameters.
NET.GETHOSTBYADDRESS	Getting host parameters.
NET.GETSERVICEENTRIES	Getting all service entries.
NET.GETSERVBYNAME	Getting a service numerical value.
NET.VERSION	Returning the version number for the NET extension.

1.8.2 NET.GETHOSTNAME

Name

NET.GETHOSTNAME – Function that returns the name of the current host.

Syntax

NET.GETHOSTNAME()

Description

The NET.GETHOSTNAME function returns the name of the current host as a string.

Errors

The following values can be assigned to `$Error`:

```
NET.$Error_GethostnameFail
```

Failed to get current host name.

Examples

The following one-line program prints the name of the current host:

```
PRINT NET.GETHOSTNAME() & "\n";
```

1.8.3 NET.GETHOSTBYNAME

Name

NET.GETHOSTBYNAME – Get host parameters.

Syntax

```
NET.GETHOSTBYNAME in-name [NAME name] [ALIASES aliases]  
[ADDRESSES addresses];
```

Description

Given a host name, the NET.GETHOSTBYNAME statement returns the network host parameters for it.

The name of the host to return parameters for is *in-name*. It is an *expression* that is converted to a string. If *in-name* is an empty string, the parameters for the current host are returned.

The *name* parameter is an output *variable* that receives the returned host name. It should be of string type.

The *aliases* parameter is an output *variable* that receives all the alternate names for the host. It should be a one-dimensional array of string type.

The *addresses* parameter is an output *variable* that receives all the IP network addresses for the host. Each address is returned as a string using dot notation.

The *addresses* parameter should be a one-dimensional array of string type.

Errors

The following values can be assigned to `$Error`:

- NET.\$Error_GethostbynameGetHostName
Failed to get current host name.
- NET.\$Error_GethostbynameHostNotFound
The host was not found.
- NET.\$Error_GethostbynameTryAgain
Retry at some later time.
- NET.\$Error_GethostbynameNoRecovery
Nonrecoverable error.
- NET.\$Error_GethostbynameNoAddress

The host does not have an address.

- `NET.$Error_GethostbynameUnknownError`
Unknown error.

Examples

The following program outputs the network parameters for the current host:

```
DECLARE VARIABLE $name STRING;
DECLARE VARIABLE $aliases[] STRING;
DECLARE VARIABLE $addresses[] STRING;

NET.GETHOSTBYNAME ""
    NAME $name
    ALIASES $aliases
    ADDRESSES $addresses;

PRINT "Name = " & $name & "\n";
PRINT "Aliases =";
FOR $i = 1 TO ARRAYSIZE($aliases) {
    PRINT " " & $aliases[$i];
}
PRINT "\n";
PRINT "Addresses =";
FOR $i = 1 TO ARRAYSIZE($addresses) {
    PRINT " " & $addresses[$i];
}
PRINT "\n";
```

See also

`NET.GETHOSTBYADDRESS`.

1.8.4 NET.GETHOSTBYADDRESS

Name

`NET.GETHOSTBYADDRESS` – Get host parameters.

Syntax

```
NET.GETHOSTBYADDRESS in-address [NAME name]
                        [ALIASES aliases]
                        [ADDRESSES addresses];
```

Description

Given an IP address in dot notation, the `NET.GETHOSTBYADDRESS` statement returns the network parameters for the host with the given address.

The IP address for the host to return parameters for is *in-address*. It is an *expression* that is converted to a string and the IP address should be specified using dot notation.

The *name* parameter is an output *variable* that receives the returned host name. It should be of string type.

The *aliases* parameter is an output *variable* that receives all the alternate names for the host. It should be a one-dimensional array of string type.

The *addresses* parameter is an output *variable* that receives all the IP network addresses for the host. Each address is returned as a string using dot notation.
The *addresses* parameter should be a one-dimensional array of string type.

Errors

The following values can be assigned to `$Error`:

- `NET.$Error_GethostbyaddressEmptyAddress`
The *in-address* parameter is an empty string.
- `NET.$Error_GethostbyaddressIllegalAddress`
The *in-address* parameter does not contain a valid address.
- `NET.$Error_GethostbyaddressHostNotFound`
The host was not found.
- `NET.$Error_GethostbyaddressTryAgain`
Retry at some later time.
- `NET.$Error_GethostbyaddressNoRecovery`
Nonrecoverable error.
- `NET.$Error_GethostbyaddressUnknownError`
Unknown error.

Examples

The following program outputs the network parameters for the host with IP address 1.2.3.4:

```
DECLARE VARIABLE $name STRING;
DECLARE VARIABLE $aliases[] STRING;
DECLARE VARIABLE $addresses[] STRING;

NET.GETHOSTBYADDRESS "1.2.3.4"
                        NAME $name
                        ALIASES $aliases
                        ADDRESSES $addresses;

PRINT "Name = " & $name & "\n";
PRINT "Aliases =";
FOR $i = 1 TO ARRAYSIZE($aliases) {
    PRINT " " & $aliases[$i];
}
PRINT "\n";
PRINT "Addresses =";
FOR $i = 1 TO ARRAYSIZE($addresses) {
    PRINT " " & $addresses[$i];
}
PRINT "\n";
```

See also

NET.GETHOSTBYNAME.

1.8.5 NET.GETSERVICEENTRIES

Name

NET.GETSERVICEENTRIES – Get all service entries.

Syntax

NET.GETSERVICEENTRIES *service-entries*;

Description

The NET.GETSERVICEENTRIES statement returns all service entries, that is, all entries in the */etc/services* file on UNIX systems, all entries in the *%SystemRoot%\system32\drivers\etc\services* on a Windows 2000 system, or all Service Table Entries accessible through the command WRKSRVTBLE on AS/400 systems. The service entries are returned in the *service-entries* output parameter, which should be an array of NET.ServiceEntry records.

The NET.ServiceEntry record has the following fields:

```
DECLARE PUBLIC RECORD ServiceEntry {  
  DECLARE PUBLIC FIELD $Name STRING;  
  DECLARE PUBLIC FIELD $Port INTEGER;  
  DECLARE PUBLIC FIELD $Protocol STRING;  
  DECLARE PUBLIC FIELD $Aliases[] STRING;  
}
```

The \$Name field is the official name of the service, the \$Port field is the port number at which the service resides, the \$Protocol field is the name of the protocol to use, and the \$Aliases field contains alternate names for the service.

Errors

This statement can generate no nonfatal errors.

Examples

The following program prints all the service entries:

```
DECLARE VARIABLE $entries[] RECORD NET.ServiceEntry;  
NET.GETSERVICEENTRIES $entries;  
FOR $i = 1 TO ARRAYSIZE($entries) {  
  PRINT "Name = " & $entries[$i].$Name & "\n";  
  PRINT "Port = " & $entries[$i].$Port & "\n";  
  PRINT "Protocol = " & $entries[$i].$Protocol & "\n";  
  PRINT "Aliases =";  
  FOR $j = 1 TO ARRAYSIZE($entries[$i].$Aliases) {  
    PRINT " " & $entries[$i].$Aliases[$j];  
  }  
  PRINT "\n";  
}
```

1.8.6 NET.VERSION

Name

NET.VERSION – Function that returns the version number for the NET extension.

Syntax

NET.VERSION()

Description

The NET.VERSION function returns the version number for the NET extension as a string. The format of the string is *n1.n2.n3*.

1.8.7 NET.GETSERVBYNAME

Name

NET.GETSERVBYNAME – Get service number corresponding to a name

Syntax

NET.GETSERVBYNAME(*servname*);

Description

This function returns the number corresponding to the TCP/IP service name *servname*. Services are stored by the operating system in */etc/services* on UNIX, *%windir%\system32\drivers\etc\services* on Windows, and a table, accessible with WRKSRVTBLE on OS/400.

Errors

The following values can be assigned to \$Error:

- NET.\$Error_GetservbynameUnknownServiceName
The *servname* could not be found.

1.9 The UTIL extension

This chapter describes the UTIL extension. The UTIL extension implements various utility functions and statements.

The error type generated by the UTIL extension is UTIL. The following program fragment demonstrates how to take care of UTIL errors:

```
TRY {  
    $perm = UTIL.GETFILEPERM("/etc/services");  
}  
CATCH $exception  
WHEN UTIL.$ExceptionName {  
    PRINT "Failed to get file permissions\n";  
}
```

All the UTIL extension functions and statements are declared in the library file *util.s4*.

A number of constants (for example, error codes) which are useful when using the UTIL extension can be found in the include file *util.s4h* (the UTIL.\$ExceptionName variable is declared in this file).

1.9.1 Summary of functions and statements

Function/ Statement	Used For
UTIL.FILESTATUS	Checking if a file exists and its type/time/size.
UTIL.FILETIMES	Checking timestamps associated with a file.
UTIL.FILEPERM	Changing permissions on a file.
UTIL.GETFILEPERM	Checking permissions of a file.
UTIL.RANDOM	Obtaining a random value
UTIL.SETRANDOM	Initializing the random number generator.
UTIL.MAXRANDOM	Obtaining the scale of the random number generator.
UTIL.GETTIME	Obtaining the current time in Epoch format.
UTIL.GETTIME2	Obtaining the current time including microseconds.
UTIL.CONVERTWITHDIFFERENTTZ	Converting time to a different timezone.
UTIL.CONVERTFROMDIFFERENTTZ	Converting time from a different timezone.
UTIL.VERSION	Returning the version number for the UTIL extension.

1.9.2 UTIL.FILESTATUS

Name

UTIL.FILESTATUS – Statement that checks the existence of a file and returns information about it

Syntax

UTIL.FILESTATUS *filename* [TYPE *type*] [SIZE *size*] [MTIME *mtime*];

Description

This statement will throw an exception and set the *\$Error* variable if the file or directory designated by *filename* does not exist. Another use for it is to check the properties of the file if it does exist.

The output parameter *type*, of type integer, is set to one of the following constants:

- UTIL.\$Util_FileTypeRegular
Regular file.
- UTIL.\$Util_FileTypeDirectory
The file is a directory.
- UTIL.\$Util_FileTypeBlock, UTIL.\$Util_FileTypeCharacter
The file is a special UNIX or OS/400 device file.
- UTIL.\$Util_FileTypeFifo
The file is a pipe file. The pipe is held in memory, this empty special file being a moniker for the in-memory pipe. Only exists on UNIX or OS/400.
- UTIL.\$Util_FileTypeOther
Unknown file type

The output parameter *size*, of type integer, returns the size in bytes. An error occurs on 32-bit Integrator for UNIX (or OS/400) if the file is larger than **2** GB.

The output parameter *mtime*, of type integer, returns the number of seconds since midnight of January 1st, 1970 GMT and the time the file was last modified. The value returned here can then be assigned to a DATE variable to obtain the local time.

Windows time issue

Since 4edi version 6.1.90 this statement returns correct modification time on Windows when Daylight Saving Time is implemented in your country and file was last modified during a winter and the current time is in summer (or vice-versa), and thus, because of a Windows bug that is present in all versions and probably cannot be removed, this correct modification time will no longer coincide with the modification time displayed incorrectly by Windows explorer and Windows dir command.

If you want the incorrect time on Windows you can call UTIL.FILETIMES instead.

You can observe this Windows bug by checking the dates returned by `ls -l --full-time` in Cygwin and the `dir` command in Windows for the same files from the wrong season. Alternatively, you can create a file any time before the spring or autumn transition, and write down its modification time as seen in Windows explorer or with the `dir` command. After the transition the modification time of the file will be different by one hour in both Windows explorer and the command prompt, though the file has not been touched.

Errors

The following value can be assigned to `$Error`:

- `UTIL.$Error_UtilFilestatusFail`
The file does not exist or another error has occurred. `$LogString` contains more details.

1.9.3 UTIL.FILETIMES

Name

UTIL.FILETIMES – Statement that returns timestamps associated with a file.

Syntax

UTIL.FILETIMES *filename* [CTIME *ctime*] [ATIME *size*] [MTIME *mtime*];

Description

The output parameter *mtime*, of type integer, returns the number of seconds since midnight of January 1st, 1970 GMT and the time the file was last modified.

The output parameter *ctime*, of type integer, returns the number of seconds since midnight of January 1st, 1970 GMT and the time the file was created (on Windows) or its properties were updated (on UNIX).

The output parameter *atime*, of type integer, returns the number of seconds since midnight of January 1st, 1970 GMT and the time the file was accessed; however, on both Windows

and UNIX, the result is usually not updated for file system performance reasons (set through the *noatime* mount option on UNIX and a registry setting on Windows).

The values returned by this statement can then be assigned to a DATE variable to obtain the local time.

This statement returns incorrect values on Windows, only when Daylight Saving Time is implemented in your country, and only for winter dates in summer or summer dates in winter. This is a Windows bug present on all Windows versions because it cannot be removed; if you want the correct modification time call UTIL.FILESTATUS on 4edi versions 6.1.90 and later.

You can observe this Windows bug by checking the dates returned by `ls -l --full-time` in Cygwin and the `dir` command in Windows for the same files from the wrong season. Alternatively, you can create a file any time before the spring or autumn transition, and write down its modification time as seen in Windows explorer or with the `dir` command. After the transition the modification time of the file will be different by one hour in both Windows explorer and the command prompt, though the file has not been touched.

Errors

The following value can be assigned to `$Error`:

- `UTIL.$Error_UtilFiletimesFail`
The file does not exist or another error has occurred. `$LogString` contains more details.

1.9.4 UTIL.FILEPERM

Name

UTIL.FILEPERM – Statement that sets permissions on a file.

Syntax

UTIL.FILEPERM *filename* PERMISSIONS *permissions*;

Description

This statement will change the security settings on the file designated by *filename*.

The *permissions* parameter must be an integer obtained by combining some of the following flags with the bitwise or operator (BOR):

- `UTIL.$Util_FilePermissionOwnerRead`
The owner of the file/directory can read the file/enumerate directory contents.
- `UTIL.$Util_FilePermissionOwnerWrite`
The owner of the file/directory can write to it/change directory contents, including by deleting files and directories inside the directory.
- `UTIL.$Util_FilePermissionOwnerExecute`
The owner of the file/directory can execute it as a program/traverse the directory.
- `UTIL.$Util_FilePermissionGroupRead`
The users in the file/directory's assigned user group can read the file/enumerate directory contents.

- **UTIL.\$Util_FilePermissionGroupWrite**
The users in the file/directory's assigned user group can write to it/change directory contents, including by deleting files and directories inside the directory.
- **UTIL.\$Util_FilePermissionGroupExecute**
The users in the file/directory's assigned user group can execute it as a program/traverse the directory.
- **UTIL.\$Util_FilePermissionOtherRead**
Other users can read the file/enumerate directory contents.
- **UTIL.\$Util_FilePermissionOtherWrite**
Other users can write to it/change directory contents, including by deleting files and directories inside the directory.
- **UTIL.\$Util_FilePermissionOtherExecute**
Other users can execute it as a program/traverse the directory.

Errors

The following value can be assigned to `$Error`:

- **UTIL.\$Error_UtilFilepermIllegalPermissions**
The *permissions* parameter was not created using the method described above.
- **UTIL.\$Error_UtilFilepermFail**
The file does not exist or another error has occurred. `$LogString` contains more details.

1.9.5 UTIL.GETFILEPERM

Name

UTIL.GETFILEPERM – Statement that obtains the security permissions of a file.

Syntax

UTIL.GETFILEPERM(*filename*);

Description

This function will return the security settings of the file designated by *filename*.

It works only on UNIX and OS/400.

The returned value contains some of the following flags, combined with the bitwise or operator (BOR):

- **UTIL.\$Util_FilePermissionOwnerRead**
The owner of the file/directory can read the file/enumerate directory contents.
- **UTIL.\$Util_FilePermissionOwnerWrite**
The owner of the file/directory can write to it/change directory contents, including by deleting files and directories inside the directory.

- **UTIL.\$Util_FilePermissionOwnerExecute**
The owner of the file/directory can execute it as a program/traverse the directory.
- **UTIL.\$Util_FilePermissionGroupRead**
The users in the file/directory's assigned user group can read the file/enumerate directory contents.
- **UTIL.\$Util_FilePermissionGroupWrite**
The users in the file/directory's assigned user group can write to it/change directory contents, including by deleting files and directories inside the directory.
- **UTIL.\$Util_FilePermissionGroupExecute**
The users in the file/directory's assigned user group can execute it as a program/traverse the directory.
- **UTIL.\$Util_FilePermissionOtherRead**
Other users can read the file/enumerate directory contents.
- **UTIL.\$Util_FilePermissionOtherWrite**
Other users can write to it/change directory contents, including by deleting files and directories inside the directory.
- **UTIL.\$Util_FilePermissionOtherExecute**
Other users can execute it as a program/traverse the directory.

Errors

The following value can be assigned to `$Error`:

- **UTIL.\$Error_UtilGetfilepermNotsupported**
This function does not work on Windows.
- **UTIL.\$Error_UtilGetfilepermFail**
The file does not exist or another error has occurred. `$LogString` contains more details.

1.9.6 UTIL.RANDOM

Name

UTIL.RANDOM – Function that returns a random number.

Syntax

UTIL.RANDOM()

Description

This function returns a random integer between **0** and UTIL.MAXRANDOM() inclusive.

Before the first call to this function the random number must be initialized. One way to do this is:

```
DECLARE $now INTEGER;
UTIL.GETTIME $now;
```

```
UTIL.SETRANDOM $now;
```

1.9.7 UTIL.SETRANDOM

Name

UTIL.SETRANDOM – Statement that initializes the random number generator.

Syntax

UTIL.SETRANDOM *seed*;

Description

This statement initializes the random number generator with an integer value. See 1.9.6 UTIL.RANDOM for details.

1.9.8 UTIL.MAXRANDOM

Name

UTIL.MAXRANDOM – Function that returns the maximum number UTIL.RANDOM() can return.

Syntax

UTIL.MAXRANDOM();

Description

This function returns an integer which is the maximum number UTIL.RANDOM() can return.

1.9.9 UTIL.GETTIME

Name

UTIL.GETTIME – Statement to obtain current time in Epoch format plus milliseconds.

Syntax

UTIL.GETTIME *seconds*[, *milliseconds*];

Description

The output parameters *seconds* and *milliseconds*, of type integer, return the time since midnight of January 1st, 1970 GMT. The value returned in the *seconds* output parameter can then be assigned to a DATE variable to obtain the local time.

1.9.10 UTIL.GETTIME2

Name

UTIL.GETTIME2 – Statement to obtain current time in Epoch format plus microseconds.

Syntax

UTIL.GETTIME2 *seconds*[, *microseconds*];

Description

The output parameters *seconds* and *microseconds*, of type integer, return the time since midnight of January 1st, 1970 GMT. The value returned in the *seconds* output parameter can then be assigned to a DATE variable to obtain the local time.

1.9.11 UTIL.CONVERTWITHDIFFERENTTZ

Name

UTIL.CONVERTWITHDIFFERENTTZ – Function to convert time to a different time zone.

Syntax

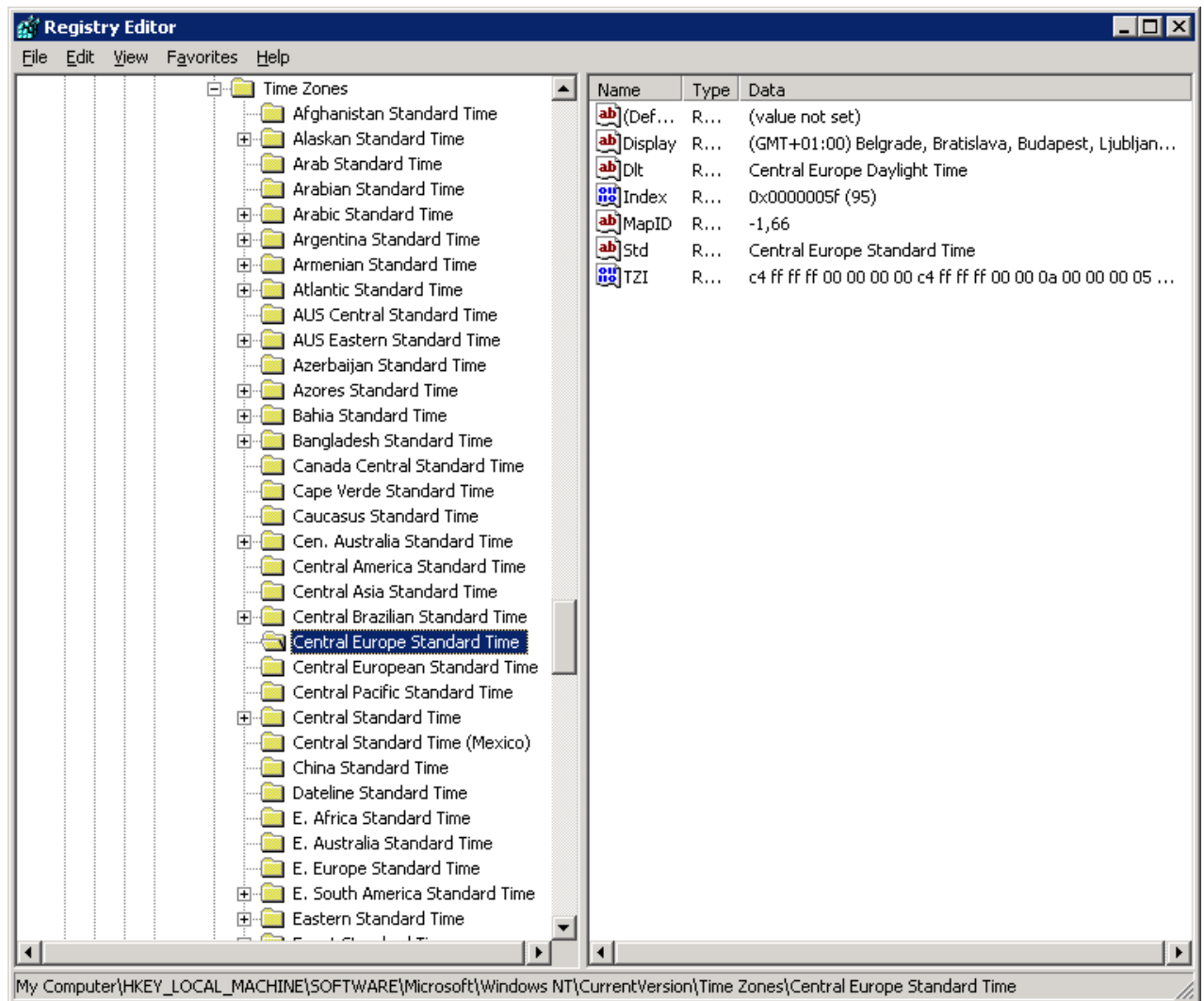
UTIL.CONVERTWITHDIFFERENTTZ(*time*, *timezone*)

Description

This function is implemented only on UNIX and Windows. The result is of type DATE which must not be used to perform date arithmetic; it contains the time converted to the specified time zone.

The *time* parameter can be an integer in Epoch format (UTC) or a variable of type DATE. The function will convert this GMT, or, respectively, local time variable to the remote time zone specified by the next parameter. If you want to convert the current time in a stable fashion around Daylight Saving Time transitions use the result of UTIL.GETTIME not CURRENTDATE() for the value of this parameter.

The *timezone* parameter must be an operating system specific string. For example, Solaris 10 and later accept Olsen specifications such as "Europe/Moscow"; see /usr/share/lib/zoneinfo for details and note that Solaris does not return an error if the specification is incorrect. On Windows, you must use the name of a registry key, without the path, that is a subkey of HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Time Zones; for example, from the image below you can choose "Central Europe Standard Time" as the value for the *timezone* parameter. Windows returns an error if the specification does not exist in Registry.



Errors

The following values can be assigned to `$Error`:

- `UTIL.$Error_UtilConvertwithdifferenttzNotsupported`
This function is not supported on this operating system.
- `UTIL.$Error_UtilConvertwithdifferenttzError`
An error occurred during conversion. For example, a date less than January 1st, 1970 was specified.

1.9.12 UTIL.CONVERTFROMDIFFERENTTZ

Name

UTIL.CONVERTFROMDIFFERENTTZ – Function to convert time from a different time zone.

Syntax

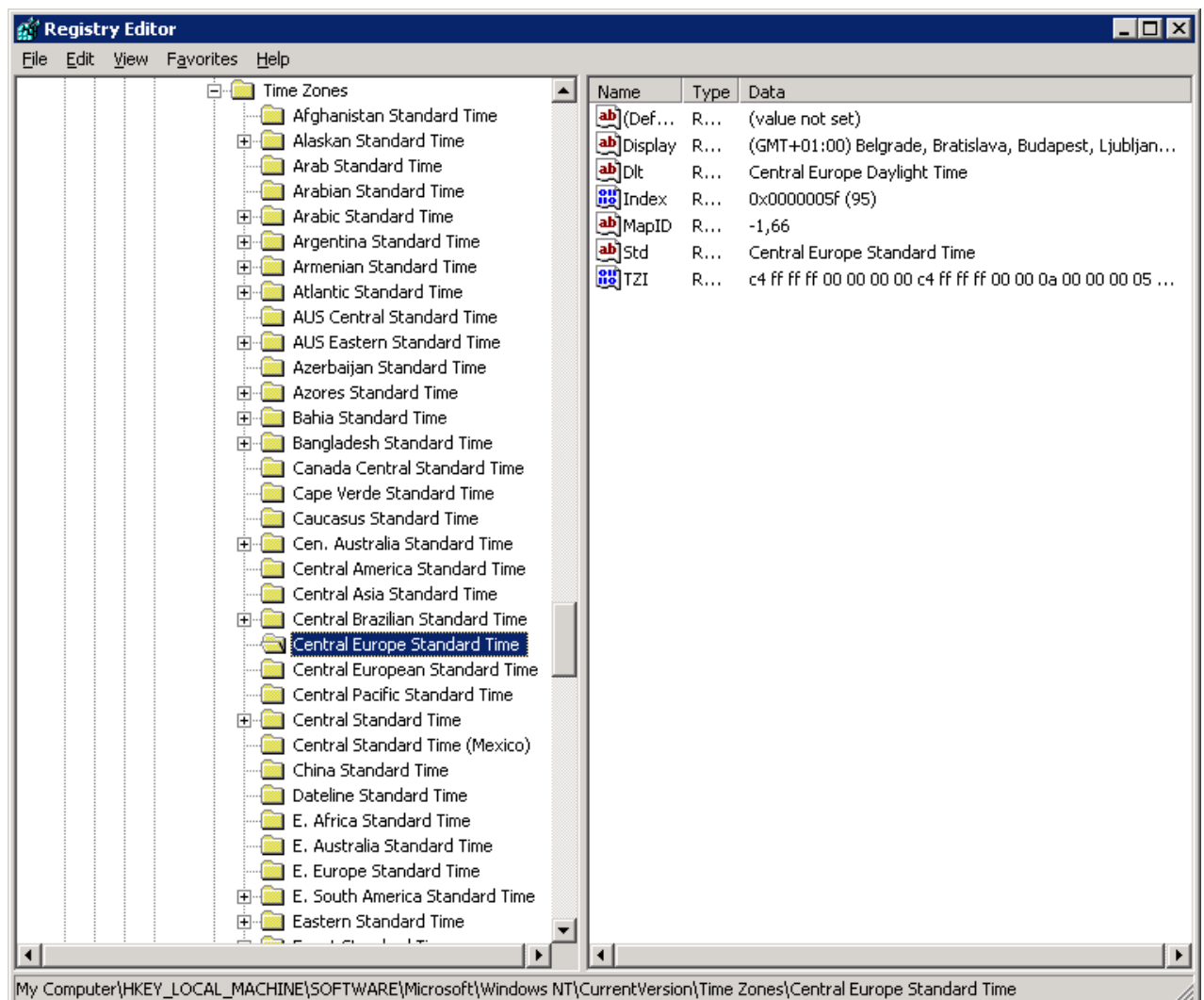
UTIL.CONVERTFROMDIFFERENTTZ *time* TIMEZONE *timezone* RESULT *result* [FRACTION *fraction*];

Description

This function is implemented only on UNIX and Windows.

The input parameter *time* is the time in the different (remote) timezone (specified by the next parameter) to convert to the system timezone (or to UTC Epoch time) and it must be of type DATE. It must not be the result of date arithmetic, or the result of conversion from an integer variable (i.e. the result of conversion from UTC time) unless that conversion was performed on a different machine in the different timezone. The correct way to obtain this variable is to use SETDATE for all components of a DATE variable; or to use UNSERIALIZE.

The *timezone* input parameter must be an operating system specific string. For example, Solaris 10 and later accept Olsen specifications such as "Europe/Moscow"; see /usr/share/lib/zoneinfo for details and note that Solaris does not return an error if the specification is incorrect. On Windows, you must use the name of a registry key, without the path, that is a subkey of HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Time Zones; for example, from the image below you can choose "Central Europe Standard Time" as the value for the *timezone* parameter. Windows returns an error if the specification does not exist in Registry.



The *result* output parameter can be an integer (which will be set to the time in UTC Epoch format) or a variable of type DATE (which will be set to the local time corresponding to the time in the different time zone).

The optional *fraction* parameter contains the fraction part of the input parameter *time*. It is required because the *result* parameter will not preserve fractions of a second, which need to be restored with SETDATE if required after calling this statement.

Errors

The following values can be assigned to `$Error`:

- UTIL.\$Error_UtilConvertfromdifferenttzNotsupported
This statement is not supported on this operating system.
- UTIL.\$Error_UtilConvertfromdifferenttzError
An error occurred during conversion. For example, a date less than January 1st, 1970 was specified.

1.10 The MIMELIB extension

This chapter describes one of the functions in the Message Builder MIMELIB extension. It is included here because of its similarity with the two timezone functions in the previous chapter.

This MIMELIB extension function is declared in the library file *mime.s4m*. A number of constants (for example, error codes) can be found in the include file *mime.s4h*.

1.10.1 MIME.GetTimeZoneOffset

Name

MIME.GetTimeZoneOffset – Function that returns the current offset of local time from UTC time.

Syntax

MIME.GETTIMEZONEOFFSET(*time*)

Description

This function is implemented on all platforms. The result is of type STRING and it starts with a plus or minus sign. The following two characters represent the hours part of the difference; the last two represent the minutes part of the difference (for example, India is +0530; of course for most other countries the minute difference will be 00).

The *time* parameter should be the *seconds* result of UTIL.GETTIME. Or, if it is desired to obtain the offset from UTC at a different local time (the offset can vary due to Daylight Saving Time) a different time can be specified here.

This function does not result in errors.

2 Auxiliary functions and statements

2.1 Presentation

This part describes the Message Builder Standard Library, a collection of Message-Builder-written convenience and utility functions. The Standard Library is divided into a number of modules, each containing related routines. The following table lists the different modules:

Module	Used For
SL_ARGUMENT	Handling command-line arguments.
SL_ARRAY	Handling arrays.
SL_CHAR	Classifying characters.
SL_DATE	Handling dates.
SL_DEBUG	Debugging MB code.
SL_ENVIRON	Expanding environment variables.
SL_FILE	Handling files.
SL_FILEIO	Handling file I/O.
SL_HASH	Handling hash tables.
SL_PACKED	Handling packed decimals.
SL_PATH	Handling path names.
SL_SEARCH	Searching arrays.
SL_SORT	Sorting arrays.
SL_STDLIB	Common routines for the Standard Library.
SL_STRING	Handling strings.
SL_TAG	Reading and writing tags from and to files.

For a description of the conventions used in this part of the manual, refer to chapter *Message builder language reference* of the *Message Builder* manual.

2.2 The SL_ARGUMENT module

This chapter describes the SL_ARGUMENT module. The SL_ARGUMENT module contains routines for handling command-line arguments.

The error type generated by the SL_ARGUMENT module functions and statements is SL_ARGUMENT. The following program fragment demonstrates how to handle SL_ARGUMENT errors:

```
INCLUDE "sl_argument.s4h";
TRY {
```

```

        SL_ARGUMENT.GetOption "ab" OPTION $option;
    }
    CATCH $exception
    WHEN SL_ARGUMENT.$ExceptionName {
        PRINT "Invalid option\n";
    }

```

All of the SL_ARGUMENT module functions and statements are declared in the library file *sl_argument.s4m*. A number of constants (for example, SL_ARGUMENT.\$ExceptionName and error codes) that are useful when using the SL_ARGUMENT module can be found in the include file *sl_argument.s4h*.

Since the SL_ARGUMENT module uses the MODULE_INIT feature for initialization, no functions or statements from the SL_ARGUMENT module should be called from other MODULE_INIT statements.

2.2.1 Summary of functions and statements

Function/Statement	Used For
SL_ARGUMENT.GetArgument	Getting non-option command arguments.
SL_ARGUMENT.GetArgumentCount	Getting the number of non-option command arguments.
SL_ARGUMENT.GetInterpreterArgument	Getting interpreter option arguments.
SL_ARGUMENT.GetInterpreterArgumentCount	Getting the number of interpreter option arguments.
SL_ARGUMENT.GetInterpreterFiles	Getting interpreter file option arguments.
SL_ARGUMENT.GetOption	Getting the next command option.
SL_ARGUMENT.Version	Returning the version number for the module.

2.2.2 SL_ARGUMENT.GetOption

Name

SL_ARGUMENT.GetOption – Get the next command line option.

Syntax

SL_ARGUMENT.GetOption *format* OPTION *option* [ARGUMENT *argument*];

Description

The SL_ARGUMENT.GetOption statement returns the next command-line option that matches a letter in the *format* parameter. A command-line option consists of a minus or, on NT machines, a slash character followed by the option character and an optional option-argument string.

The *format* parameter is a string specifying all valid option characters. If a character in *format* is followed by a colon, the option takes an argument that may or may not be separated from it by white space. A *format* that specifies three options (**a**, **b** and **c**), where **b** takes an argument, looks as follows:

```
"ab:c"
```

The next option is returned as a one-character string in the *option* output parameter. If the option takes an argument, the *argument* output parameter is set to the

argument. If there are no more options on the command line, *option* is set to an empty string and if an invalid option is found on the command line, an exception is thrown.

Errors

The following values can be assigned to `$Error` (the error type is `SL_ARGUMENT`):

- `SL_ARGUMENT.$Error_GetOption_IllegalOption`
Invalid option on the command line.
- `SL_ARGUMENT.$Error_GetOption_MissingOptionArgument`
Command option without required option argument.

Examples

```
INCLUDE "sl_argument.s4h";

DECLARE VARIABLE $opt STRING;
DECLARE VARIABLE $argument STRING;

WHILE 1 {
    TRY {
        SL_ARGUMENT.GetOption "Va:dfp:" OPTION $opt ARGUMENT $argument;
        CASE $opt
        WHEN "" {
            BREAK;      /* no more options */
        }
        WHEN "V" {
            PRINT "Got V option\n";
        }
        WHEN "a" {
            PRINT "Got a option, argument = " & $argument & "\n";
        }
        WHEN "d" {
            PRINT "Got d option\n";
        }
        WHEN "f" {
            PRINT "Got f option\n";
        }
        WHEN "p" {
            PRINT "Got p option, argument = " & $argument & "\n";
        }
    }
    CATCH $exception
    WHEN SL_ARGUMENT.$ExceptionName {
        IF $Error = SL_ARGUMENT.$Error_GetOption_IllegalOption {
            PRINTERR "found illegal option\n";
        }
        ELSE {
            PRINTERR "missing option argument\n";
        }
        EXIT 1;
    }
}

/* take care of non-option command arguments */
FOR $i = 1 TO SL_ARGUMENT.GetArgumentCount() {
    PRINT "non-option argument = " & SL_ARGUMENT.GetArgument($i) &
"\n";
}
```

See also

SL_ARGUMENT.GetArgument, SL_ARGUMENT.GetArgumentCount.

2.2.3 SL_ARGUMENT.GetArgument

Name

SL_ARGUMENT.GetArgument – Function that returns a non-option command argument.

Syntax

SL_ARGUMENT.GetArgument(*argument-number*)

Description

When all options have been parsed using SL_ARGUMENT.GetOption, the SL_ARGUMENT.GetArgument function is used to retrieve the remaining non-option command arguments.

The *argument-number* argument should be of integer type and it specifies the command argument to retrieve. The first non-option command argument has number one and the number for the last is the same as the value returned by SL_ARGUMENT.GetArgumentCount.

The specified non-option command argument is returned as a string.

Errors

The following value can be assigned to `$Error` (the error type is SL_ARGUMENT):

```
SL_ARGUMENT.$Error_GetArgument_NoSuchArgument
```

The *argument-number* argument is illegal. It is either less than one or greater than the number of non-option command arguments.

Examples

Refer to the example for the SL_ARGUMENT.GetOption statement.

See also

SL_ARGUMENT.GetOption, SL_ARGUMENT.GetArgumentCount.

2.2.4 SL_ARGUMENT.GetArgumentCount

Syntax

SL_ARGUMENT.GetArgumentCount()

Description

When all options have been parsed using `SL_ARGUMENT.GetOption`, the `SL_ARGUMENT.GetArgumentCount` function is used to obtain the number of non-option command arguments. The return value is an integer.

Examples

Refer to the example for the `SL_ARGUMENT.GetOption` statement.

See also

`SL_ARGUMENT.GetOption`, `SL_ARGUMENT.GetArgument`.

2.2.5 `SL_ARGUMENT.GetInterpreterArgument`

Name

`SL_ARGUMENT.GetInterpreterArgument` – Function that returns an interpreter option argument.

Syntax

`SL_ARGUMENT.GetInterpreterArgument(argument-number)`

Description

The `SL_ARGUMENT.GetInterpreterArgument` function is used to retrieve an interpreter option argument (that is, the arguments given between the interpreter command name and the Message Builder program name). The number for the first interpreter option argument is one and the number of the last is the same as the value returned by the `SL_ARGUMENT.GetInterpreterArgumentCount` function.

The number of the interpreter option argument to retrieve is *argument-number*. It should be of integer type. The specified interpreter option argument is returned as a string.

Errors

The following value can be assigned to `$Error` (the error type is `SL_ARGUMENT`):

```
SL_ARGUMENT.$Error_GetInterpreterArgument_NoSuchArgument
```

The *argument-number* argument is illegal. It is either less than one or greater than the number of interpreter option arguments.

Examples

The following program prints all the interpreter option arguments:

```
FOR $i = 1 TO SL_ARGUMENT.GetInterpreterArgumentCount() {  
    PRINT SL_ARGUMENT.GetInterpreterArgument($i) & "\n";  
}
```

See also

SL_ARGUMENT.GetInterpreterArgumentCount.

2.2.6 SL_ARGUMENT.GetInterpreterArgumentCount

Name

SL_ARGUMENT.GetInterpreterArgumentCount – Function that returns the number of interpreter option arguments.

Syntax

SL_ARGUMENT.GetInterpreterArgumentCount()

Description

The SL_ARGUMENT.GetInterpreterArgumentCount function returns the number of interpreter option arguments. The return value is an integer.

Examples

Refer to the example for the SL_ARGUMENT.GetInterpreterArgument function.

See also

SL_ARGUMENT.GetInterpreterArgument.

2.2.7 SL_ARGUMENT.GetInterpreterFiles

Name

SL_ARGUMENT.GetInterpreterFiles – Function that returns an array of interpreter file option arguments.

Syntax

SL_ARGUMENT.GetInterpreterFiles()

Description

The SL_ARGUMENT.GetInterpreterFiles function returns a string array of size 4 containing the files specified by the **0**, **1**, **2** and **3** interpreter options. The first element of this array contains the **0** option, the second the **1** option, the third the **2** option, and the fourth the **3** option. An empty string indicates not specified options. Each element corresponding to a specified option has the option prefixed in front of the file name (see the example).

Examples

If the following program:

```
DECLARE VARIABLE $fileOptions[] STRING;

$fileOptions = SL_ARGUMENT.GetInterpreterFiles();
PRINT $fileOptions[1] & "\n";
PRINT $fileOptions[2] & "\n";
PRINT $fileOptions[3] & "\n";
PRINT $fileOptions[4] & "\n";
```

is run using the following command:

```
r4edi -0 file0 -1file1 -2 file2 -3file3 prog.x4;
```

the following is output (to the *file1* file):

```
-0file0
-1file1
-2file2
-3file3
```

2.2.8 SL_ARGUMENT.Version

Name

SL_ARGUMENT.Version – Function that returns the version number for the SL_ARGUMENT module.

Syntax

SL_ARGUMENT.Version()

Description

The SL_ARGUMENT.Version function returns the version number for the SL_ARGUMENT module as a string. The format of the string is *n1.n2.n3*.

2.3 The SL_ARRAY module

This chapter describes the SL_ARRAY module. The SL_ARRAY module contains array-handling routines.

All of the SL_ARRAY module functions and statements are declared in the library file *sl_array.s4m*.

2.3.1 Summary of functions and statements

Function/Statement	Used For
SL_ARRAY.Append	Appending an element to an array.
SL_ARRAY.Delete	Deleting an element from an array.
SL_ARRAY.Insert	Inserting an element into an array.
SL_ARRAY.Clear	Deleting all elements from an array.

Function/Statement	Used For
SL_ARRAY.Compare	Comparing two arrays with any number of dimensions.
SL_ARRAY.Version	Returning the version number for the module.

2.3.2 SL_ARRAY.Insert

Name

SL_ARRAY.Insert – Insert an element into an array.

Syntax

SL_ARRAY.Insert *array* POSITION *index* VALUE *element*;

Description

The SL_ARRAY.Insert statement inserts an element into an array. The *array* parameter is the array to insert the element into, the *index* parameter is the position at which to insert the element, and *element* is the element to insert. The *index* parameter can be any integer ≥ 1 and *element* should be of the same type as the elements of *array*.

If *index* is less than or equal to the size of *array*, all elements that have an index greater than or equal to *index* are moved up one position, that is, their index is incremented by 1. If *index* is greater than the size of *array*, no elements are moved up.

Examples

If the following program is executed:

```
DECLARE VARIABLE $array[] = { 1, 2, 3, 4 } INTEGER;

SL_ARRAY.Insert $array POSITION 3 VALUE 54;
FOR $i = 1 TO ARRAYSIZE($array) {
    PRINT $array[$i] & "\n";
}
```

the following is written to the output file:

```
1
2
54
3
4
```

2.3.3 SL_ARRAY.Append

Name

SL_ARRAY.Append – Append an element to the end of an array.

Syntax

SL_ARRAY.Append *array* VALUE *element*;

Description

The `SL_ARRAY.Append` statement appends an element to the end of an array. The *array* parameter is the array to append the element to and *element* is the element to append. The *element* parameter should be of the same type as the elements of *array*.

Examples

If the following program is executed:

```
DECLARE VARIABLE $array[] = { 1, 2, 3, 4 } INTEGER;

SL_ARRAY.Append $array VALUE 54;
FOR $i = 1 TO ARRAYSIZE($array) {
    PRINT $array[$i] & "\n";
}
```

the following is written to the output file:

```
1
2
3
4
54
```

2.3.4 SL_ARRAY.Delete

Name

`SL_ARRAY.Delete` – Delete an element from an array.

Syntax

```
SL_ARRAY.Delete array POSITION index;
```

Description

The `SL_ARRAY.Delete` statement deletes an element from an array. The *array* parameter is the array to delete the element from and *index* is the index position of the element to delete. The *index* parameter should be an integer.

All elements that have an index greater than *index* are moved down one position, that is, their index is decremented by 1.

Examples

If the following program is executed:

```
DECLARE VARIABLE $array[] = { 1, 2, 3, 4 } INTEGER;

SL_ARRAY.Delete $array POSITION 3;
FOR $i = 1 TO ARRAYSIZE($array) {
    PRINT $array[$i] & "\n";
}
```

the following is written to the output file:

```
1
2
4
```

2.3.5 SL_ARRAY.Clear

Name

SL_ARRAY.Clear – Delete all elements from an array.

Syntax

SL_ARRAY.Clear *array*;

Description

The SL_ARRAY.Clear statement deletes all elements from an array.

2.3.6 SL_ARRAY.Compare

Name

SL_ARRAY.Compare – Compare two arrays

Syntax

SL_ARRAY.Compare(*array1*, *array2*)

Description

This function returns non-zero if the two arrays are identical. The arrays may have any number of dimensions but must not contain records.

Examples

If the following program is executed:

```
DECLARE VARIABLE $array1[] = { 1, 2, 3, 4 } INTEGER;  
DECLARE VARIABLE $array2[] = { 1, 2, 3, 4 } INTEGER;  
  
PRINT SL_ARRAY.Compare($array1, $array2) & "\n";
```

the following is written to the output file:

1

2.3.7 SL_ARRAY.Version

Name

SL_ARRAY.Version. – Function that returns the version number for the SL_ARRAY module.

Syntax

SL_ARRAY.Version()

Description

The `SL_ARRAY.Version` function returns the version number for the `SL_ARRAY` module as a string. The format of the string is *n1.n2.n3*.

2.4 The `SL_CHAR` module

This chapter describes the `SL_CHAR` module. The `SL_CHAR` module contains character classification routines.

All of the `SL_CHAR` module functions are declared in the library file *sl_char.s4m*.

Since the `SL_CHAR` module uses the `MODULE_INIT` feature for initialization, no functions or statements from the `SL_CHAR` module should be called from other `MODULE_INIT` statements.

2.4.1 Summary of functions

Function	Used For
<code>SL_CHAR.ClassAdd</code>	Adding characters to a character class.
<code>SL_CHAR.ClassCreate</code>	Creating a new character class.
<code>SL_CHAR.ClassRemove</code>	Removing characters from a character class.
<code>SL_CHAR.IsAlpha</code>	Checking if upper- or lowercase-letter character.
<code>SL_CHAR.IsAlphaDigit</code>	Checking if alphabetic letter or decimal-digit character.
<code>SL_CHAR.IsAlphaDigitUnderscore</code>	Checking if alphabetic letter, decimal-digit character, or underscore character.
<code>SL_CHAR.IsAlphaUnderscore</code>	Checking if alphabetic letter or an underscore character.
<code>SL_CHAR.IsDigit</code>	Checking if decimal-digit character.
<code>SL_CHAR.IsHex</code>	Checking if hexadecimal-digit character.
<code>SL_CHAR.IsInClass</code>	Checking if character is in character class.
<code>SL_CHAR.IsLower</code>	Checking if lowercase-letter character.
<code>SL_CHAR.IsPrintable</code>	Checking if printable character.
<code>SL_CHAR.IsUpper</code>	Checking if uppercase-letter character.
<code>SL_CHAR.Version</code>	Returning the version number for the module.

2.4.2 `SL_CHAR.IsAlpha`

Name

`SL_CHAR.IsAlpha` – Function that checks for an upper- or lowercase letter character.

Syntax

`SL_CHAR.IsAlpha(character)`

Description

The SL_CHAR.IsAlpha function returns nonzero if the first character in the *character* string is a lower- or uppercase letter. Otherwise, it returns zero. The *character* argument should be of string type.

Examples

The following program:

```
DECLARE VARIABLE $string = "4Ab!_\n" STRING;

FOR $i = 1 TO STRLEN($string) {
    PRINT SL_CHAR.IsAlpha($string[$i,$i]) & "\n";
}
```

prints the following:

```
0
1
1
0
0
0
```

2.4.3 SL_CHAR.IsAlphaUnderscore

Name

SL_CHAR.IsAlphaUnderscore – Function that checks for an upper- or lowercase letter or underscore character.

Syntax

SL_CHAR.IsAlphaUnderscore(*character*)

Description

The SL_CHAR.IsAlphaUnderscore function returns nonzero if the first character in the *character* string argument is a lower- or uppercase letter or an underscore. Otherwise, it returns zero. The *character* argument should be of string type.

Examples

The following program:

```
DECLARE VARIABLE $string = "4Ab!_\n" STRING;

FOR $i = 1 TO STRLEN($string) {
    PRINT SL_CHAR.IsAlphaUnderscore($string[$i,$i]) & "\n";
}
```

prints the following:

```
0
1
1
0
```

1
0

2.4.4 SL_CHAR.IsAlphaDigit

Name

SL_CHAR.IsAlphaDigit – Function that checks for an alphabetic letter or decimal-digit character.

Syntax

SL_CHAR.IsAlphaDigit(*character*)

Description

The SL_CHAR.IsAlphaDigit function returns nonzero if the first character in the argument *character* is a lower- or uppercase letter or a decimal digit. Otherwise, it returns zero. The *character* argument should be of string type.

Examples

The following program:

```
DECLARE VARIABLE $string = "4Ab!\n" STRING;  
  
FOR $i = 1 TO STRLEN($string) {  
    PRINT SL_CHAR.IsAlphaDigit($string[$i,$i]) & "\n";  
}
```

prints the following:

1
1
1
0
0
0

2.4.5 SL_CHAR.IsAlphaDigitUnderscore

Name

SL_CHAR.IsAlphaDigitUnderscore – Function that checks for an alphabetic letter, decimal-digit or underscore character.

Syntax

SL_CHAR.IsAlphaDigitUnderscore(*character*)

Description

The SL_CHAR.IsAlphaDigitUnderscore function returns nonzero if the first character in the argument *character* is a lower- or uppercase letter, a decimal digit or an

underscore. Otherwise, it returns zero.
The *character* argument should be of string type.

Examples

The following program:

```
DECLARE VARIABLE $string = "4Ab!\n" STRING;

FOR $i = 1 TO STRLEN($string) {
    PRINT SL_CHAR.IsAlphaDigitUnderscore($string[$i,$i]) & "\n";
}
```

prints the following:

```
1
1
1
0
1
0
```

2.4.6 SL_CHAR.IsDigit

Name

SL_CHAR.IsDigit – Function that checks for a decimal-digit character.

Syntax

SL_CHAR.IsDigit(*character*)

Description

The SL_CHAR.IsDigit function returns nonzero if the first character in the *character* string is a decimal digit. Otherwise, it returns zero. The *character* argument should be of string type.

Examples

The following program:

```
DECLARE VARIABLE $string = "4Ab!\n" STRING;

FOR $i = 1 TO STRLEN($string) {
    PRINT SL_CHAR.IsDigit($string[$i,$i]) & "\n";
}
```

prints the following:

```
1
0
0
0
0
0
```

2.4.7 SL_CHAR.IsHex

Name

SL_CHAR.IsHex – Function that checks for a hexadecimal-digit character.

Syntax

SL_CHAR.IsHex(*character*)

Description

The SL_CHAR.IsHex function returns nonzero if the first character in the *character* string is a hexadecimal digit. Otherwise, it returns zero. The *character* argument should be of string type.

Examples

The following program:

```
DECLARE VARIABLE $string = "4Ab!\n" STRING;

FOR $i = 1 TO STRLEN($string) {
    PRINT SL_CHAR.IsHex($string[$i,$i]) & "\n";
}
```

prints the following:

```
1
1
1
0
0
0
```

2.4.8 SL_CHAR.IsLower

Name

SL_CHAR.IsLower – Function that checks for a lowercase-letter character.

SYNTAX

SL_CHAR.IsLower(*character*)

Description

The SL_CHAR.IsLower function returns nonzero if the first character in the *character* string is a lowercase letter. Otherwise, it returns zero. The *character* argument should be of string type.

Examples

The following program:

```
DECLARE VARIABLE $string = "4Ab!\n" STRING;

FOR $i = 1 TO STRLEN($string) {
    PRINT SL_CHAR.IsLower($string[$i,$i]) & "\n";
}
```

prints the following:

```
0
0
1
0
0
0
```

2.4.9 SL_CHAR.IsPrintable

Name

SL_CHAR.IsPrintable – Function that checks for a printable character.

Syntax

SL_CHAR.IsPrintable(*character*)

Description

The SL_CHAR.IsPrintable function returns nonzero if the first character in the argument *character* is printable (space is a printable character). Otherwise, it returns zero. The *character* argument should be of string type.

Examples

The following program:

```
DECLARE VARIABLE $string = "4Ab!\n" STRING;

FOR $i = 1 TO STRLEN($string) {
    PRINT SL_CHAR.IsPrintable($string[$i,$i]) & "\n";
}
```

prints the following:

```
1
1
1
1
1
0
```

2.4.10 SL_CHAR.IsUpper

Name

SL_CHAR.IsUpper – Function that checks for an uppercase-letter character.

Syntax

SL_CHAR.IsUpper(*character*)

Description

The SL_CHAR.IsUpper function returns nonzero if the first character in the *character* string is an uppercase letter. Otherwise, it returns zero. The *character* argument should be of string type.

Examples

The following program:

```
DECLARE VARIABLE $string = "4Ab!\n" STRING;

FOR $i = 1 TO STRLEN($string) {
    PRINT SL_CHAR.IsUpper($string[$i,$i]) & "\n";
}
```

prints the following:

```
0
1
0
0
0
0
0
```

2.4.11 SL_CHAR.ClassCreate

Name

SL_CHAR.ClassCreate – Function that creates a new character class.

Syntax

SL_CHAR.ClassCreate(*characters*)

Description

The SL_CHAR.ClassCreate function creates and returns a new character class. The *characters* argument should be of string type and it should contain the characters that initially shall be included in the character class. A string representing the new character class is returned.

The SL_CHAR.IsInClass function can be used to check if a character is included in a character class.

Examples

The following program:

```
DECLARE VARIABLE $string = "4Ab!\n" STRING;
DECLARE VARIABLE $class STRING;

$class = SL_CHAR.ClassCreate("4!");
```

```

FOR $i = 1 TO STRLEN($string) {
    PRINT SL_CHAR.IsInClass($string[$i,$i], $class) & "\n";
}

```

prints the following:

```

1
0
0
1
0
0

```

See also

SL_CHAR.ClassAdd, SL_CHAR.ClassRemove, SL_CHAR.IsInClass.

2.4.12 SL_CHAR.ClassAdd

Name

SL_CHAR.ClassAdd – Function that adds characters to a character class.

Syntax

SL_CHAR.ClassAdd(*characters*, *class*)

Description

The SL_CHAR.ClassAdd function adds characters to an existing character class. The *characters* argument should be of string type and contain the characters to add. The *class* argument is the character class to add characters to. A string representing the modified character class is returned.

Examples

The following program:

```

DECLARE VARIABLE $string = "4Ab!_\n" STRING;
DECLARE VARIABLE $class STRING;

$class = SL_CHAR.ClassCreate("4!");
$class = SL_CHAR.ClassAdd("Ab");
FOR $i = 1 TO STRLEN($string) {
    PRINT SL_CHAR.IsInClass($string[$i,$i], $class) & "\n";
}

```

prints the following:

```

1
1
1
1
0
0

```

See also

SL_CHAR.ClassCreate, SL_CHAR.ClassRemove, SL_CHAR.IsInClass.

2.4.13 SL_CHAR.ClassRemove

Name

SL_CHAR.ClassRemove – Function that removes characters from a character class.

Syntax

SL_CHAR.ClassRemove(*characters*, *class*)

Description

The SL_CHAR.ClassRemove function removes characters from an existing character class. The *characters* argument should be of string type and contain the characters to remove. The *class* argument is the character class to remove characters from. A string representing the modified character class is returned.

Examples

The following program:

```
DECLARE VARIABLE $string = "4Ab!\n" STRING;
DECLARE VARIABLE $class STRING;

$class = SL_CHAR.ClassCreate("4!");
$class = SL_CHAR.ClassRemove("4");
FOR $i = 1 TO STRLEN($string) {
    PRINT SL_CHAR.IsInClass($string[$i,$i], $class) & "\n";
}
```

prints the following:

```
0
0
0
1
0
0
```

See also

SL_CHAR.ClassCreate, SL_CHAR.ClassAdd, SL_CHAR.IsInClass.

2.4.14 SL_CHAR.IsInClass

Name

SL_CHAR.IsInClass – Function that checks if a character is included in a character class.

Syntax

SL_CHAR.IsInClass(*character*, *class*)

Description

The SL_CHAR.IsInClass function returns nonzero if the first character in the *character* string is included in the character class specified by *class*. Otherwise, it returns zero. Both the *character* and *class* arguments should be of string type. The *class* argument should previously have been returned from one of the SL_CHAR.ClassCreate, SL_CHAR.ClassAdd or SL_CHAR.ClassRemove functions.

Examples

Refer to the examples for the SL_CHAR.ClassCreate, SL_CHAR.ClassAdd and SL_CHAR.ClassRemove functions.

See also

SL_CHAR.ClassCreate, SL_CHAR.ClassAdd, SL_CHAR.ClassRemove.

2.4.15 SL_CHAR.Version

Name

SL_CHAR.Version. – Function that returns the version number for the SL_CHAR module.

Syntax

SL_CHAR.Version()

Description

The SL_CHAR.Version function returns the version number for the SL_CHAR module as a string. The format of the string is *n1.n2.n3*.

2.5 The SL_DATE module

This chapter describes the SL_DATE module. The SL_DATE module contains date handling routines.

The error type generated by the SL_DATE module functions is SL_DATE. The following program fragment demonstrates how to handle SL_DATE errors:

```
INCLUDE "sl_date.s4h";
TRY {
    $date = SL_DATE.StringToDate($string, "%Y.%m.%d");
}
CATCH $exception
WHEN SL_DATE.$ExceptionName {
    PRINT "Invalid date string\n";
}
```

All of the SL_DATE module functions are declared in the library file *sl_date.s4m*. A number of constants (for example, `SL_DATE.$ExceptionName` and error codes) that are useful when using the SL_DATE module can be found in the include file *sl_date.s4h*.

2.5.1 Summary of functions

Function	Used For
SL_DATE.StringToDate	Converting a string to a date value.
SL_DATE.Version	Returning the version number for the module.

2.5.2 SL_DATE.StringToDate

Name

SL_DATE.StringToDate – Function that converts a string of specified format to a DATE value.

Syntax

SL_DATE.StringToDate(*string*, *format*)

Description

The SL_DATE.StringToDate function converts a string containing a date of a specified format to a date value. The *string* argument is the string to convert to a date value and *format* is the format string. Both arguments should be of string type. The date value is returned.

The format of the *format* argument should be the same as for the `$DateFormat` reserved variable (for details of this variable, refer to the *Message Builder* manual).

Errors

The following value can be assigned to `$Error` (the error type is SL_DATE):

```
SL_DATE.$Error_StringToDate_Convert
```

The contents of *string* are not compatible with *format*.

Examples

The following program:

```
DECLARE VARIABLE $date DATE;

$DateFormat = "%Y-%b-%d";
$date = SL_DATE.StringToDate("2000.01.13", "%Y.%m.%d");
PRINT $date & "\n";
```

prints the following:

```
2000-Jan-13
```

2.5.3 SL_DATE.Version

Name

SL_DATE.Version. – Function that returns the version number for the SL_DATE module.

Syntax

SL_DATE.Version()

Description

The SL_DATE.Version function returns the version number for the SL_DATE module as a string. The format of the string is *n1.n2.n3*.

2.6 The SL_DEBUG module

This chapter describes the SL_DEBUG module. The SL_DEBUG module contains debug utility routines.

All of the SL_DEBUG module functions and statements are declared in the library file *sl_debug.s4m*.

2.6.1 Summary of functions and statements

Function/Statement	Used For
SL_DEBUG.Assert	Asserting that a condition is fulfilled.
SL_DEBUG.Version	Returning the version number for the module.

2.6.2 SL_DEBUG.Assert

Name

SL_DEBUG.Assert – Assert that a condition is fulfilled.

Syntax

SL_DEBUG.Assert *condition* FILE *file* LINE *line*;

Description

The SL_DEBUG.Assert statement is used to assert that the condition specified by *condition* is true (nonzero). If *condition* is true, SL_DEBUG.Assert does nothing. If *condition* is false (zero), SL_DEBUG.Assert will log an assert error (with log type ASSERT) and throw the ASSERT exception. The *file* parameter should be the name of the source code file where the SL_DEBUG.Assert statement is located (that is, SOURCEFILE()) and *line* should be the line number (that is, SOURCELINE()).

All `SL_DEBUG.Assert` calls are by default removed when a program is compiled. However, if the following line is specified somewhere in the program (not inside a statement block or as a local variable) or on the compiler command line:

```
DECLARE VARIABLE $SL_DEBUG_AssertEnabled = 1 ONCE INTEGER;
```

the `SL_DEBUG.Assert` calls will take effect.

Examples

The `SL_DEBUG.Assert` statement can be used to check that arguments to functions and parameters to statements contain valid values. The following function demonstrates this:

```
DECLARE VARIABLE $SL_DEBUG_AssertEnabled = 1 ONCE INTEGER;

DECLARE FUNCTION
ComputeSomething($arg1 PRIMITIVE, $arg2 PRIMITIVE) PRIMITIVE
{
    SL_DEBUG.Assert $arg1 > 0 FILE SOURCEFILE() LINE SOURCELINE();
    SL_DEBUG.Assert $arg2 >= 0 FILE SOURCEFILE() LINE SOURCELINE();
    .
    .
    .
    RETURN $result;
}
```

Notes

The reason that `SL_DEBUG.Assert` statements by default are removed is as follows. The default value of `$SL_DEBUG_AssertEnabled` is zero. When it is zero the optimizer will remove the body of the `SL_DEBUG.Assert` statement because the code is not reachable (because of an IF statement that always evaluates to false). This leaves an empty `SL_DEBUG.Assert` statement. All calls to empty statements are also removed by the optimizer. Finally, the empty `SL_DEBUG.Assert` statement is removed because no one calls it.

2.6.3 SL_DEBUG.Version

Name

`SL_DEBUG.Version`. – Function that returns the version number for the `SL_DEBUG` module.

Syntax

`SL_DEBUG.Version()`

Description

The `SL_DEBUG.Version` function returns the version number for the `SL_DEBUG` module as a string. The format of the string is *n1.n2.n3*.

2.7 The SL_ENVIRON module

This chapter describes the SL_ENVIRON module. The SL_ENVIRON module contains routines for expanding environment variables.

All of the SL_ENVIRON module functions are declared in the library file *sl_environ.s4m*.

2.7.1 Summary of functions

Function	Used For
SL_ENVIRON.ExpandEnvironmentVariables	Expanding environment variables.
SL_ENVIRON.Version	Returning the version number for the module.

2.7.2 SL_ENVIRON.ExpandEnvironmentVariables

Name

SL_ENVIRON.ExpandEnvironmentVariables – Function that expands environment variables.

Syntax

SL_ENVIRON.ExpandEnvironmentVariables(*string*)

Description

The SL_ENVIRON.ExpandEnvironmentVariables function expands all references to environment variables in *string* and returns the expanded string.

On Windows 2000 machines, an environment variable reference is of the format %NAME%.

On UNIX and AS/400 machines, environment variables can be referenced in the following ways:

```
$NAME
${NAME}
${NAME+word}
${NAME-word}
```

Examples

If the HOME environment variable is set to /home/tradesync and the following one-line program is run on a UNIX machine:

```
PRINT SL_ENVIRON.ExpandEnvironmentVariables("$HOME/.profile\n");
```

The following is output:

```
/home/tradesync/.profile
```

2.7.3 SL_ENVIRON.Version

Name

SL_ENVIRON.Version. – Function that returns the version number for the SL_ENVIRON module.

Syntax

SL_ENVIRON.Version()

Description

The SL_ENVIRON.Version function returns the version number for the SL_ENVIRON module as a string. The format of the string is *n1.n2.n3*.

2.8 The SL_FILE module

This chapter describes the SL_FILE module. The SL_FILE module contains file handling routines.

All of the SL_FILE module functions and statements are declared in the library file *sl_file.s4m*. A number of constants (for example, `SL_FILE.$ExceptionName` and error codes) that are useful when using the SL_FILE module can be found in the include file *sl_file.s4h*.

Since the SL_FILE module uses the MODULE_INIT feature for initialization, no functions or statements from the SL_FILE module should be called from other MODULE_INIT statements.

2.8.1 Summary of functions and statements

Function/Statement	Used For
SL_FILE.DeleteDirectoryTree	Deleting a directory tree.
SL_FILE.ReadDirectory	Reading the contents of a directory.
SL_FILE.SilentDeleteFile	Deleting a file without throwing an exception.
SL_FILE.Version	Returning the version number for the module.

2.8.2 SL_FILE.DeleteDirectoryTree

Name

SL_FILE.DeleteDirectoryTree – Delete a directory tree.

Syntax

SL_FILE.DeleteDirectoryTree *path* [DELETE_ROOT *deleteRoot*];

Description

The `SL_FILE.DeleteDirectoryTree` statement deletes a directory including all the files and directories in it. The *path* parameter is the path name for the directory to delete. It should be of string type.

On UNIX and OS/400, if pipes or device files are present in the directory or subdirectories an exception is thrown with `$Error = SL_FILE.$Error_DeleteDirectoryTreeRemoveDirectory`.

By default, the given directory is removed. The optional *deleteRoot* parameter can be specified as 0, to delete only the contents of the given directory without removing the directory itself.

Errors

The following values can be assigned to `$Error` (the error type is `SL_FILE`):

- `SL_FILE.$Error_DeleteDirectoryTreeStatDirectory`
Failed to get the status for a file.
- `SL_FILE.$Error_DeleteDirectoryTreeNotDirectory`
The file specified by the *path* parameter is not a directory.
- `SL_FILE.$Error_DeleteDirectoryTreeOpenDirectory`
Failed to open a directory.
- `SL_FILE.$Error_DeleteDirectoryTreeReadDirectory`
Failed to read the next entry from a directory.
- `SL_FILE.$Error_DeleteDirectoryTreeRemoveDirectory`
Failed to remove a directory file.
- `SL_FILE.$Error_DeleteDirectoryTreeDeleteFile`
Failed to delete a regular file.

Examples

The following program deletes the directory named "abc":

```
SL_FILE.DeleteDirectoryTree "abc";
```

2.8.3 SL_FILE.ReadDirectory

Name

`SL_FILE.ReadDirectory` – Read the contents of a directory.

Syntax

```
SL_FILE.ReadDirectory path FILES files;
```

Description

The `SL_FILE.ReadDirectory` statement reads the contents of a directory. The *path* parameter specifies the name of the directory to read the contents of. An array of `SL_FILE.FileInfo` records containing the names, types and sizes of all the files is returned in the *files* output parameter. The special directories "." and ".." are not included among the returned files.

The *path* input parameter should be of string type and the *files* output parameter should be an array of SL_FILE.FileInfo records:

```
DECLARE PUBLIC RECORD FileInfo {
    DECLARE PUBLIC FIELD $Name STRING;
    DECLARE PUBLIC FIELD $Type INTEGER;
    DECLARE PUBLIC FIELD $Size INTEGER;
}
```

The *\$Name* field contains the name of the file, the *\$Type* field contains a file type code and *\$Size* is the file size. The type code field can contain any of the following values:

- SL_FILE.\$FileType_Regular : Regular file.
- SL_FILE.\$FileType_Directory : Directory file.
- SL_FILE.\$FileType_Fifo : FIFO file.
- SL_FILE.\$FileType_Block : Block special file.
- SL_FILE.\$FileType_Character : Character special file.
- SL_FILE.\$FileType_Other : Unknown file type.

Errors

The following values can be assigned to *\$Error* (the error type is SL_FILE):

- SL_FILE.\$Error_ReadDirectoryNotDirectory
The *path* parameter does not specify a directory.
- SL_FILE.\$Error_ReadDirectoryOpenDirectory
Failed to open the directory specified by *path*.
- SL_FILE.\$Error_ReadDirectoryReadDirectory
Error when reading the directory.
- SL_FILE.\$Error_ReadDirectoryStatEntry
Failed to get the status for one of the files in the *path* directory.

Examples

The following program outputs the name and size of all the regular files in the "dir" directory:

```
DECLARE VARIABLE $files[] RECORD SL_FILE.FileInfo;

SL_FILE.ReadDirectory "dir" FILES $files;
FOR $i = 1 TO ARRAYSIZE($files) {
    IF $files[$i] = SL_FILE.$FileType_Regular {
        PRINT $files[$i].$Name & ": ";
        PRINT $files[$i].$Size & "\n";
    }
}
```

2.8.4 SL_FILE.SilentDeleteFile

Name

SL_FILE.SilentDeleteFile – Delete a file that may not exist

Syntax

`SL_FILE.SilentDeleteFile filename;`

Description

Deletes the file specified by *filename* if possible. No error is printed in the Trace Viewer and no exception is thrown in any case. *\$Error* may be modified.

2.8.5 SL_FILE.Version

Name

`SL_FILE.Version.` – Function that returns the version number for the `SL_FILE` module.

Syntax

`SL_FILE.Version()`

Description

The `SL_FILE.Version` function returns the version number for the `SL_FILE` module as a string. The format of the string is *n1.n2.n3*.

2.9 The SL_FILEIO module

This chapter describes the `SL_FILEIO` module. The `SL_FILEIO` module contains routines for handling file I/O.

The error type generated by the `SL_FILEIO` module functions and statements is `SL_FILEIO`. The following program fragment demonstrates how to handle `SL_FILEIO` errors:

```
INCLUDE "sl_fileio.s4h";
TRY {
    $handle = SL_FILEIO.OpenBinaryRead("binfile");
}
CATCH $exception
WHEN SL_FILEIO.$ExceptionName {
    PRINT "Failed to open file\n";
}
```

All of the `SL_FILEIO` module functions and statements are declared in the library file *sl_fileio.s4m*. A number of constants (for example, `SL_FILEIO.$ExceptionName` and error codes) that are useful when using the `SL_FILEIO` module can be found in the include file *sl_fileio.s4h*.

Since the `SL_FILEIO` module uses the `MODULE_INIT` feature for initialization, no functions or statements from the `SL_FILEIO` module should be called from other `MODULE_INIT` statements.

2.9.1 Summary of functions and statements

Function/Statement	Used For
SL_FILEIO.AppendBinaryReadWrite	Opening or create a binary file for reading and writing.
SL_FILEIO.AppendBinaryWrite	Opening a binary file for writing.
SL_FILEIO.AppendTextWrite	Opening a text file for writing.
SL_FILEIO.CloseFile	Closing an open file.
SL_FILEIO.CreateBinaryReadWrite	Creating an empty binary file and open it for reading and writing.
SL_FILEIO.CreateBinaryWrite	Creating an empty binary file and open it for writing.
SL_FILEIO.CreateTextWrite	Creating an empty text file and open it for writing.
SL_FILEIO.DirectorySeparator	Returning the directory separator character.
SL_FILEIO.Flush	Flushing data to disk.
SL_FILEIO.IsBinary	Testing if a file is open in binary mode.
SL_FILEIO.IsOpen	Testing if a file is open.
SL_FILEIO.IsReadable	Testing if a file is open and readable.
SL_FILEIO.IsText	Testing if a file is open in text mode.
SL_FILEIO.IsWritable	Testing if a file is open and writable.
SL_FILEIO.OpenBinaryRead	Opening a binary file for reading.
SL_FILEIO.OpenBinaryReadWrite	Opening a binary file for reading and writing.
SL_FILEIO.OpenTextRead	Opening a text file for reading.
SL_FILEIO.ReadCount	Reading the specified number of bytes from a file.
SL_FILEIO.ReadUntil	Reading from a file until a string is found.
SL_FILEIO.Synchronize	Synchronizing to disk the contents of a file.
SL_FILEIO.Seek	Setting the current file position
SL_FILEIO.Tell	Getting the current file position.
SL_FILEIO.Unread	Putting back read data.
SL_FILEIO.Version	Returning the version number for the module.
SL_FILEIO.WriteString	Writing a string to a file.

2.9.2 SL_FILEIO.OpenBinaryRead

Name

SL_FILEIO.OpenBinaryRead – Function that opens a binary file for reading.

Syntax

SL_FILEIO.OpenBinaryRead(*pathname*)

Description

The SL_FILEIO.OpenBinaryRead function opens a binary file for reading and returns a handle identifying the open file. The handle is an anonymous record of type SL_FILEIO.FileHandle. It is used to refer to the open file.

The *pathname* argument is the name of the file to open. It should be of string type.

Errors

The following value can be assigned to `$Error` (the error type is `SL_FILEIO`):

```
SL_FILEIO.$Error_OpenBinaryRead_Open
```

Failed to open the file specified by *pathname*.

Examples

The following program opens the binary file `binfile`, reads data from the file in some unspecified way, and finally closes the file:

```
INCLUDE "sl_fileio.s4h";
DECLARE VARIABLE $handle RECORD SL_FILEIO.FileHandle;

$handle = SL_FILEIO.OpenBinaryRead("binfile");
.
.
.
SL_FILEIO.CloseFile $handle;
```

See also

`SL_FILEIO.OpenTextRead`, `SL_FILEIO.CreateBinaryWrite`, `SL_FILEIO.CreateTextWrite`,
`SL_FILEIO.AppendBinaryWrite`, `SL_FILEIO.AppendTextWrite`,
`SL_FILEIO.OpenBinaryReadWrite`, `SL_FILEIO.CreateBinaryReadWrite`,
`SL_FILEIO.AppendBinaryReadWrite`.

2.9.3 SL_FILEIO.OpenTextRead

Name

`SL_FILEIO.OpenTextRead` – Function that opens a text file for reading.

Syntax

`SL_FILEIO.OpenTextRead(pathname)`

Description

The `SL_FILEIO.OpenTextRead` function opens a text file for reading and returns a handle identifying the open file. The handle is an anonymous record of type `SL_FILEIO.FileHandle`. It is used to refer to the open file.

The *pathname* argument is the name of the file to open. It should be of string type.

Errors

The following value can be assigned to `$Error` (the error type is `SL_FILEIO`):

```
SL_FILEIO.$Error_OpenTextRead_Open
```

Failed to open the file specified by *pathname*.

Examples

The following program opens the text file `textfile`, reads data from the file in some unspecified way, and finally closes the file:

```
INCLUDE "sl_fileio.s4h";
DECLARE VARIABLE $handle RECORD SL_FILEIO.FileHandle;

$handle = SL_FILEIO.OpenTextRead("textfile");
.
.
.
SL_FILEIO.CloseFile $handle;
```

See also

SL_FILEIO.OpenBinaryRead, SL_FILEIO.CreateBinaryWrite, SL_FILEIO.CreateTextWrite, SL_FILEIO.AppendBinaryWrite, SL_FILEIO.AppendTextWrite, SL_FILEIO.OpenBinaryReadWrite, SL_FILEIO.CreateBinaryReadWrite, SL_FILEIO.AppendBinaryReadWrite.

2.9.4 SL_FILEIO.CreateBinaryWrite

Name

SL_FILEIO.CreateBinaryWrite – Function that creates an empty binary file and opens it for writing.

Syntax

SL_FILEIO.CreateBinaryWrite(*pathname*)

Description

The SL_FILEIO.CreateBinaryWrite function creates an empty binary file, opens it for writing, and returns a handle identifying the open file. The handle is an anonymous record of type SL_FILEIO.FileHandle. It is used to refer to the open file.

The *pathname* argument is the name of the file to create. It should be of string type.

Errors

The following value can be assigned to `$Error` (the error type is SL_FILEIO):

```
SL_FILEIO.$Error_CreateBinaryWrite_Open
```

Failed to create the file specified by *pathname*.

Examples

The following program creates the binary file `binfile`, writes data to the file in some unspecified way, and finally closes the file:

```
INCLUDE "sl_fileio.s4h";
DECLARE VARIABLE $handle RECORD SL_FILEIO.FileHandle;

$handle = SL_FILEIO.CreateBinaryWrite("binfile");
```



```
.  
. .  
SL_FILEIO.CloseFile $handle;
```

See also

SL_FILEIO.OpenBinaryRead, SL_FILEIO.OpenTextRead, SL_FILEIO.CreateTextWrite, SL_FILEIO.AppendBinaryWrite, SL_FILEIO.AppendTextWrite, SL_FILEIO.OpenBinaryReadWrite, SL_FILEIO.CreateBinaryReadWrite, SL_FILEIO.AppendBinaryReadWrite.

2.9.5 SL_FILEIO.CreateTextWrite

Name

SL_FILEIO.CreateTextWrite – Function that creates an empty text file and opens it for writing.

Syntax

SL_FILEIO.CreateTextWrite(*pathname*)

Description

The SL_FILEIO.CreateTextWrite function creates an empty text file, opens it for writing, and returns a handle identifying the open file. The handle is an anonymous record of type SL_FILEIO.FileHandle. It is used to refer to the open file.

The *pathname* argument is the name of the file to create. It should be of string type.

Errors

The following value can be assigned to \$Error (the error type is SL_FILEIO):

```
SL_FILEIO.$Error_CreateTextWrite_Open
```

Failed to create the file specified by *pathname*.

Examples

The following program creates the text file `textfile`, writes data to the file in some unspecified way, and finally closes the file:

```
INCLUDE "sl_fileio.s4h";  
DECLARE VARIABLE $handle RECORD SL_FILEIO.FileHandle;  
  
$handle = SL_FILEIO.CreateTextWrite("textfile");  
. .  
. .  
SL_FILEIO.CloseFile $handle;
```

See also

SL_FILEIO.OpenBinaryRead, SL_FILEIO.OpenTextRead, SL_FILEIO.CreateBinaryWrite, SL_FILEIO.AppendBinaryWrite, SL_FILEIO.AppendTextWrite, SL_FILEIO.OpenBinaryReadWrite, SL_FILEIO.CreateBinaryReadWrite, SL_FILEIO.AppendBinaryReadWrite.

2.9.6 SL_FILEIO.AppendBinaryWrite

Name

SL_FILEIO.AppendBinaryWrite – Function that opens a binary file for writing.

Syntax

SL_FILEIO.AppendBinaryWrite(*pathname*)

Description

The SL_FILEIO.AppendBinaryWrite function opens a binary file for writing and returns a handle identifying the open file. The handle is an anonymous record of type SL_FILEIO.FileHandle. It is used to refer to the open file. The current file position is initially set to the end of the file.

The *pathname* argument is the name of the file to open. It should be of string type.

Errors

The following value can be assigned to \$Error (the error type is SL_FILEIO):

```
SL_FILEIO.$Error_AppendBinaryWrite_Open
```

Failed to open the file specified by *pathname*.

Examples

The following program opens the binary file `binfile`, writes data to the file in some unspecified way, and finally closes the file:

```
INCLUDE "sl_fileio.s4h";
DECLARE VARIABLE $handle RECORD SL_FILEIO.FileHandle;

$handle = SL_FILEIO.AppendBinaryWrite("binfile");
.
.
.
SL_FILEIO.CloseFile $handle;
```

See also

SL_FILEIO.OpenBinaryRead, SL_FILEIO.OpenTextRead, SL_FILEIO.CreateBinaryWrite, SL_FILEIO.CreateTextWrite, SL_FILEIO.AppendTextWrite, SL_FILEIO.OpenBinaryReadWrite, SL_FILEIO.CreateBinaryReadWrite, SL_FILEIO.AppendBinaryReadWrite.

2.9.7 SL_FILEIO.AppendTextWrite

Name

SL_FILEIO.AppendTextWrite – Function that opens a text file for writing.

Syntax

SL_FILEIO.AppendTextWrite(*pathname*)

Description

The SL_FILEIO.AppendTextWrite function opens a text file for writing and returns a handle identifying the open file. The handle is an anonymous record of type SL_FILEIO.FileHandle. It is used to refer to the open file. The current file position is initially set to the end of the file.

The *pathname* argument is the name of the file to open. It should be of string type.

Errors

The following value can be assigned to `$Error` (the error type is SL_FILEIO):

```
SL_FILEIO.$Error_AppendTextWrite_Open
```

Failed to open the file specified by *pathname*.

Examples

The following program opens the text file `textfile`, writes data to the file in some unspecified way, and finally closes the file:

```
INCLUDE "sl_fileio.s4h";
DECLARE VARIABLE $handle RECORD SL_FILEIO.FileHandle;

$handle = SL_FILEIO.AppendTextWrite("textfile");
.
.
.
SL_FILEIO.CloseFile $handle;
```

See also

SL_FILEIO.OpenBinaryRead, SL_FILEIO.OpenTextRead, SL_FILEIO.CreateBinaryWrite, SL_FILEIO.CreateTextWrite, SL_FILEIO.AppendBinaryWrite, SL_FILEIO.OpenBinaryReadWrite, SL_FILEIO.CreateBinaryReadWrite, SL_FILEIO.AppendBinaryReadWrite.

2.9.8 SL_FILEIO.OpenBinaryReadWrite

Name

SL_FILEIO.OpenBinaryReadWrite – Function that opens a binary file for reading and writing.

Syntax

SL_FILEIO.OpenBinaryReadWrite(*pathname*)

Description

The SL_FILEIO.OpenBinaryReadWrite function opens a binary file for reading and writing and returns a handle identifying the open file. The handle is an anonymous record of type SL_FILEIO.FileHandle. It is used to refer to the open file.

The *pathname* argument is the name of the file to open. It should be of string type.

Note that write operations on the opened file cannot be followed by read operations without an intervening flush or seek operation and read operations cannot be followed by write operations without an intervening seek operation.

Errors

The following value can be assigned to `$Error` (the error type is SL_FILEIO):

```
SL_FILEIO.$Error_OpenBinaryReadWrite_Open
```

Failed to open the file specified by *pathname*.

Examples

The following program opens the binary file `binfile`, processes the file in some unspecified way, and finally closes the file:

```
INCLUDE "sl_fileio.s4h";
DECLARE VARIABLE $handle RECORD SL_FILEIO.FileHandle;

$handle = SL_FILEIO.OpenBinaryReadWrite("binfile");
.
.
.
SL_FILEIO.CloseFile $handle;
```

See also

SL_FILEIO.OpenBinaryRead, SL_FILEIO.OpenTextRead, SL_FILEIO.CreateBinaryWrite, SL_FILEIO.CreateTextWrite, SL_FILEIO.AppendBinaryWrite, SL_FILEIO.AppendTextWrite, SL_FILEIO.CreateBinaryReadWrite, SL_FILEIO.AppendBinaryReadWrite.

2.9.9 SL_FILEIO.CreateBinaryReadWrite

Name

SL_FILEIO.CreateBinaryReadWrite – Function that creates an empty binary file and opens it for reading and writing.

Syntax

SL_FILEIO.CreateBinaryReadWrite(*pathname*)

Description

The SL_FILEIO.CreateBinaryReadWrite function creates an empty binary file, opens it for reading and writing, and returns a handle identifying the open file. The handle is an anonymous record of type SL_FILEIO.FileHandle. It is used to refer to the open file.

The *pathname* argument is the name of the file to create. It should be of string type.

Note that write operations on the opened file cannot be followed by read operations without an intervening flush or seek operation and read operations cannot be followed by write operations without an intervening seek operation.

Errors

The following value can be assigned to `$Error` (the error type is SL_FILEIO):

SL_FILEIO.\$Error_CreateBinaryReadWrite_Open

Failed to create the file specified by *pathname*.

Examples

The following program creates the binary file `binfile`, processes the file in some unspecified way, and finally closes the file:

```
INCLUDE "sl_fileio.s4h";
DECLARE VARIABLE $handle RECORD SL_FILEIO.FileHandle;

$handle = SL_FILEIO.CreateBinaryReadWrite("binfile");
.
.
.
SL_FILEIO.CloseFile $handle;
```

See also

SL_FILEIO.OpenBinaryRead, SL_FILEIO.OpenTextRead, SL_FILEIO.CreateBinaryWrite, SL_FILEIO.CreateTextWrite, SL_FILEIO.AppendBinaryWrite, SL_FILEIO.AppendTextWrite, SL_FILEIO.OpenBinaryReadWrite, SL_FILEIO.AppendBinaryReadWrite.

2.9.10 SL_FILEIO.AppendBinaryReadWrite

Name

SL_FILEIO.AppendBinaryReadWrite – Function that opens or creates a binary file for reading and writing.

Syntax

SL_FILEIO.AppendBinaryReadWrite(*pathname*)

Description

The SL_FILEIO.AppendBinaryReadWrite function opens or, if the file does not exist, creates a binary file for reading and writing and returns a handle identifying the open file. The handle is an anonymous record of type SL_FILEIO.FileHandle. It is used to refer to the open file. The current file position is initially set to the end of the file.

The *pathname* argument is the name of the file to open or create. It should be of string type.

Note that write operations on the opened file cannot be followed by read operations without an intervening flush or seek operation and read operations cannot be followed by write operations without an intervening seek operation.

Errors

The following value can be assigned to `$Error` (the error type is SL_FILEIO):

```
SL_FILEIO.$Error_AppendBinaryReadWrite_Open
```

Failed to open or create the file specified by *pathname*.

Examples

The following program opens the binary file `binfile`, processes the file in some unspecified way, and finally closes the file:

```
INCLUDE "sl_fileio.s4h";
DECLARE VARIABLE $handle RECORD SL_FILEIO.FileHandle;

$handle = SL_FILEIO.AppendBinaryReadWrite("binfile");
.
.
.
SL_FILEIO.CloseFile $handle;
```

See also

SL_FILEIO.OpenBinaryRead, SL_FILEIO.OpenTextRead, SL_FILEIO.CreateBinaryWrite, SL_FILEIO.CreateTextWrite, SL_FILEIO.AppendBinaryWrite, SL_FILEIO.AppendTextWrite, SL_FILEIO.OpenBinaryReadWrite, SL_FILEIO.CreateBinaryReadWrite.

2.9.11 SL_FILEIO.ReadCount

Name

SL_FILEIO.ReadCount – Read the specified number of bytes from a file.

Syntax

SL_FILEIO.ReadCount *filehandle* COUNT *count* DATA *data*;

Description

The SL_FILEIO.ReadCount statement reads the specified number of bytes from a file.

The *filehandle* parameter is a handle identifying the file to read from. It has previously been returned by one of the file open functions in the SL_FILEIO module.

The *count* parameter should be of integer type and it specifies the number of bytes to read. If the end of file is reached, fewer than *count* bytes will be returned.

The read data is returned in the *data* output parameter. It should be a *variable* of string type. If end of file has been reached prior to calling this function, an empty string is returned in *data*.

Errors

The following values can be assigned to `$Error` (the error type is SL_FILEIO):

- SL_FILEIO.\$Error_ReadCount_NotOpen
The *filehandle* parameter does not specify an open file.
- SL_FILEIO.\$Error_ReadCount_MustSeek
A seek or flush operation has to be performed before it is possible to read data from this file. This error is obtained if the previous operation on the file was a write.
- SL_FILEIO.\$Error_ReadCount_Read
Error when reading from the file specified by *filehandle*.

Examples

The following program copies the binary file specified by the first command argument to a file specified by the second command argument:

```
INCLUDE "sl_fileio.s4h";
DECLARE VARIABLE $inHandle RECORD SL_FILEIO.FileHandle;
DECLARE VARIABLE $outHandle RECORD SL_FILEIO.FileHandle;
DECLARE VARIABLE $data STRING;

$inHandle = SL_FILEIO.OpenBinaryRead(ARGUMENT(1));
$outHandle = SL_FILEIO.CreateBinaryWrite(ARGUMENT(2));
REPEAT {
    SL_FILEIO.ReadCount $inHandle COUNT 1024 DATA $data;
    SL_FILEIO.WriteString $outHandle DATA $data;
} WHILE $data <> "";
SL_FILEIO.CloseFile $inHandle;
SL_FILEIO.CloseFile $outHandle;
```

See also

SL_FILEIO.ReadUntil.

2.9.12 SL_FILEIO.ReadUntil

Name

SL_FILEIO.ReadUntil – Function that reads from a file until a string is found.

Syntax

SL_FILEIO.ReadUntil *filehandle* UNTIL *until* [LIMIT *limit*] DATA *data*;

Description

The SL_FILEIO.ReadUntil statement reads data from a file until the specified string is found or the specified maximum number of bytes has been read.

The *filehandle* parameter is a handle identifying the file to read from. It has previously been returned by one of the file open functions in the SL_FILEIO module.

Data is read until the string *until* is found, until *limit* number of bytes have been read, or the end of file has been reached, whichever comes first. The *until* parameter should be of string type and be nonempty and *limit* should be of integer type. If *limit* is not specified or is zero, data will be read until the *until* string is found or the end of file is reached.

The read data is returned in the *data* output parameter. It should be a *variable* of string type. If end of file has been reached prior to calling this function, an empty string is returned in *data*.

Errors

The following values can be assigned to `$Error` (the error type is SL_FILEIO):

- SL_FILEIO.\$Error_ReadUntil_NotOpen
The *filehandle* parameter does not specify an open file.
- SL_FILEIO.\$Error_ReadUntil_EmptyUntil
The *until* parameter is an empty string.
- SL_FILEIO.\$Error_ReadUntil_MustSeek
A seek or flush operation has to be performed before it is possible to read data from this file. This error is obtained if the previous operation on the file was a write.
- SL_FILEIO.\$Error_ReadUntil_Read
Error when reading from the file specified by *filehandle*.

Examples

The following function reads the next line from the specified file:

```
INCLUDE "sl_fileio.s4h";

DECLARE FUNCTION
GetLine($handle RECORD SL_FILEIO.FileHandle) RECORD
SL_FILEIO.FileHandle
{
```



```

        DECLARE VARIABLE $data STRING;
        SL_FILEIO.ReadUntil $handle UNTIL "\n" DATA $data;
        RETURN $data;
    }

```

See also

SL_FILEIO.ReadCount.

2.9.13 SL_FILEIO.Unread

Name

SL_FILEIO.Unread – Put back read data.

Syntax

SL_FILEIO.Unread *filehandle* DATA *data*;

Description

When too much data has been read from a file, the SL_FILEIO.Unread statement can be used to put data back. A later call to one of the SL_FILEIO.ReadCount or SL_FILEIO.ReadUntil statements will return the unread data.

The *filehandle* parameter is a handle identifying the file to put back data to. It has previously been returned by one of the file open functions in the SL_FILEIO module.

The *data* parameter contains the data to unread. It should be of string type.

Errors

The following values can be assigned to \$Error (the error type is SL_FILEIO):

- SL_FILEIO.\$Error_Unread_NotOpen
The *filehandle* parameter does not specify an open file.
- SL_FILEIO.\$Error_Unread_MustSeek
A seek or flush operation has to be performed before it is possible to put back data to this file. This error is obtained if the previous operation on the file was a write.

Examples

The following function reads characters from the specified file until a non-digit character is encountered:

```

INCLUDE "sl_fileio.s4h";

DECLARE FUNCTION
ReadDigits($handle RECORD SL_FILEIO.FileHandle) PRIMITIVE
{
    DECLARE VARIABLE $digits STRING;

    $digits = "";
    WHILE 1 {
        SL_FILEIO.ReadCount $handle COUNT 1 DATA $data;
        IF $data = "" {
            BREAK;      /* end of file */
        }
    }
}

```

```

    }
    IF SL_CHAR.IsDigit($data) = 0 {
        SL_FILEIO.Unread $handle DATA $data;
        BREAK;
    }
    $digits = $digits & $data;
}
RETURN $digits;
}

```

2.9.14 SL_FILEIO.Synchronize

Name

SL_FILEIO.Synchronize – Synchronize to disk the current contents of a file.

Syntax

SL_FILEIO.Synchronize *filehandle*;

Description

The *filehandle* parameter is a handle identifying the file to synchronize to disk. It has previously been returned by one of the file open functions in the SL_FILEIO module.

Errors

The following values can be assigned to \$Error (the error type is SL_FILEIO):

- SL_FILEIO.\$Error_Synchronize_NotOpen
The *filehandle* parameter does not specify an open file.
- SL_FILEIO.\$Error_Synchronize_Synchronize
The file could not be synchronized.

2.9.15 SL_FILEIO.WriteString

Name

SL_FILEIO.WriteString – Write a string of data to a file.

Syntax

SL_FILEIO.WriteString *filehandle* DATA *data*;

Description

The SL_FILEIO.WriteString statement is used to write data to a file.

The *filehandle* parameter is a handle identifying the file to write to. It has previously been returned by one of the file open functions in the SL_FILEIO module.

The *data* parameter contains the data to write. It should be of string type.

Errors

The following values can be assigned to `$Error` (the error type is `SL_FILEIO`):

- `SL_FILEIO.$Error_WriteString_NotOpen`
The *filehandle* parameter does not specify an open file.
- `SL_FILEIO.$Error_WriteString_MustSeek`
A seek operation has to be performed before it is possible to write data to this file. This error is obtained if the previous operation on the file was a read.
- `SL_FILEIO.$Error_WriteString_Write`
Error when writing data to the file.

Examples

Refer to the example for the `SL_FILEIO.ReadCount` statement.

2.9.16 SL_FILEIO.Seek

Name

`SL_FILEIO.Seek` – Set the current file position.

Syntax

`SL_FILEIO.Seek filehandle [OFFSET offset] [WHENCE whence];`

Description

The `SL_FILEIO.Seek` statement sets the current file position to the position specified by the *offset* and *whence* parameters.

The *filehandle* parameter is a handle identifying the file. It has previously been returned by one of the file open functions in the `SL_FILEIO` module.

The *offset* parameter specifies the offset from the position specified by the *whence* parameter. Both are *expressions* that are converted to integers. Valid values for *whence* are:

- `SL_FILEIO.$Seek_WhenceBegin` : *offset* is relative the beginning of the file.
- `SL_FILEIO.$Seek_WhenceCurrent` : *offset* is relative the current file position.
- `SL_FILEIO.$Seek_WhenceEnd` : *offset* is relative the end of the file.

Note: `SL_FILEIO.Seek` can only be used for files that have been opened in binary mode.

Errors

The following values can be assigned to `$Error` (the error type is `SL_FILEIO`):

- `SL_FILEIO.$Error_Seek_NotOpen`
The *filehandle* parameter does not specify an open file.
- `SL_FILEIO.$Error_Seek_InvalidWhence`
The *whence* parameter contains an invalid value.
- `SL_FILEIO.$Error_Seek_TextFile`
The file is opened in text mode.

- `SL_FILEIO.$Error_Seek_Seek`
Error when trying to move the file position.

2.9.17 SL_FILEIO.Tell

Name

SL_FILEIO.Tell – Get the current file position.

Syntax

SL_FILEIO.Tell *filehandle* POSITION *position*;

Description

The SL_FILEIO.Tell statement returns the current file position in the *position* output parameter.

The *filehandle* parameter is a handle identifying the file. It has previously been returned by one of the file open functions in the SL_FILEIO module. The *position* output parameter should be of integer type.

Note that SL_FILEIO.Tell can only be used for files that have been opened in binary mode.

Errors

The following values can be assigned to `$Error` (the error type is SL_FILEIO):

- `SL_FILEIO.$Error_Tell_NotOpen`
The *filehandle* parameter does not specify an open file.
- `SL_FILEIO.$Error_Tell_TextFile`
The file is opened in text mode.
- `SL_FILEIO.$Error_Tell_Tell`
Error when trying to get the file position.

2.9.18 SL_FILEIO.Flush

Name

SL_FILEIO.Flush – Flush data to disk.

Syntax

SL_FILEIO.Flush *filehandle*;

Description

The SL_FILEIO.Flush statement flushes the contents of the file buffer for the open file specified by *filehandle*. To increase the performance when writing data, the SL_FILEIO module internally buffers data to avoid issuing small write requests to the operating system. This statement forces the contents of the internal buffer to be written to the file.

The *filehandle* parameter is a handle identifying the file to flush. It has previously been returned by one of the file open functions in the SL_FILEIO module.

Errors

The following values can be assigned to `$Error` (the error type is SL_FILEIO):

- `SL_FILEIO.$Error_Flush_NotOpen`
The *filehandle* parameter does not specify an open file.
- `SL_FILEIO.$Error_Flush_Flush`
Error when trying to write the internal buffer to file.

2.9.19 SL_FILEIO.CloseFile

Name

SL_FILEIO.CloseFile – Close an open file.

Syntax

SL_FILEIO.CloseFile *filehandle*;

Description

The SL_FILEIO.CloseFile statement closes an open file.

The *filehandle* parameter is a handle identifying the file to close. It has previously been returned by one of the file open functions in the SL_FILEIO module.

Errors

The following values can be assigned to `$Error` (the error type is SL_FILEIO):

- `SL_FILEIO.$Error_CloseFile_NotOpen`
The *filehandle* parameter does not specify an open file.
- `SL_FILEIO.$Error_CloseFile_Close`
Error when trying to close the file.

Examples

Refer to the documentation for the SL_FILEIO.ReadCount statement.

2.9.20 SL_FILEIO.IsOpen

Name

SL_FILEIO.IsOpen – Function that tests if a file is open.

Syntax

SL_FILEIO.IsOpen(*filehandle*)

Description

The `SL_FILEIO.IsOpen` function returns `SL_FILEIO.$True` if the file specified by *filehandle* is open. Otherwise `SL_FILEIO.$False` is returned.

Examples

The following code fragment opens a file only if it is not already open:

```
INCLUDE "sl_fileio.s4h";
DECLARE VARIABLE $handle RECORD SL_FILEIO.FileHandle;
.
.
.
IF SL_FILEIO.IsOpen($handle) = SL_FILEIO.$False {
    $handle = SL_FILEIO.OpenBinaryRead("binfile");
}
.
.
.
```

2.9.21 SL_FILEIO.IsReadable

Name

`SL_FILEIO.IsReadable` – Function that tests if a file is open and readable.

Syntax

`SL_FILEIO.IsReadable(filehandle)`

Description

The `SL_FILEIO.IsReadable` function returns `SL_FILEIO.$True` if the file specified by *filehandle* is open for reading. Otherwise `SL_FILEIO.$False` is returned.

Examples

The following code fragment reads from the file only if it is open for reading:

```
INCLUDE "sl_fileio.s4h";
DECLARE VARIABLE $handle RECORD SL_FILEIO.FileHandle;
.
.
.
IF SL_FILEIO.IsReadable($handle) = SL_FILEIO.$True {
    $handle = SL_FILEIO.ReadCount $handle COUNT 1024 DATA $data;
}
.
.
.
```

See also

`SL_FILEIO.IsWritable`.

2.9.22 SL_FILEIO.IsWritable

Name

SL_FILEIO.IsWritable – Function that tests if a file is open and writable.

Syntax

SL_FILEIO.IsWritable(*filehandle*)

Description

The SL_FILEIO.IsWritable function returns SL_FILEIO.\$True if the file specified by *filehandle* is open for writing. Otherwise SL_FILEIO.\$False is returned.

Examples

The following code fragment writes to the file only if it is open for writing:

```
INCLUDE "sl_fileio.s4h";
DECLARE VARIABLE $handle RECORD SL_FILEIO.FileHandle;
.
.
.
IF SL_FILEIO.IsWritable($handle) = SL_FILEIO.$True {
    $handle = SL_FILEIO.WriteString $handle DATA $data;
}
.
.
.
```

See also

SL_FILEIO.IsReadable.

2.9.23 SL_FILEIO.IsBinary

Name

SL_FILEIO.IsBinary – Function that tests if a file is open in binary mode.

Syntax

SL_FILEIO.IsBinary(*filehandle*)

Description

The SL_FILEIO.IsBinary function returns SL_FILEIO.\$True if the file specified by *filehandle* is open as a binary file. Otherwise SL_FILEIO.\$False is returned.

See also

SL_FILEIO.IsText.

2.9.24 SL_FILEIO.IsText

Name

SL_FILEIO.IsText – Function that tests if a file is open in text mode.

Syntax

SL_FILEIO.IsText(*filehandle*)

Description

The SL_FILEIO.IsText function returns `SL_FILEIO.$True` if the file specified by *filehandle* is open as a text file. Otherwise `SL_FILEIO.$False` is returned.

See also

SL_FILEIO.IsBinary.

2.9.25 SL_FILEIO.DirectorySeparator

Name

SL_FILEIO.DirectorySeparator – Function that returns the directory separator character.

Syntax

SL_FILEIO.DirectorySeparator()

Description

The SL_FILEIO.DirectorySeparator function returns, as a one-character string, the character used to separate directory components in a file/path name.

On UNIX and AS/400 machines, the returned separator is a slash (/) character and on NT machines it is a backslash (\) character

Examples

The following fragment opens a file named *file* in the directory *directory* (this code works on UNIX, Windows 2000, and AS/400 machines):

```
INCLUDE "sl_fileio.s4h";
DECLARE VARIABLE $handle RECORD SL_FILEIO.FileHandle;
DECLARE VARIABLE $path STRING;

$path = "directory" & SL_FILEIO.DirectorySeparator() & "file";
$handle = SL_FILEIO.OpenBinaryRead($path);
.
.
.
```


2.9.26 SL_FILEIO.Version

Name

SL_FILEIO.Version. – Function that returns the version number for the SL_FILEIO module.

Syntax

SL_FILEIO.Version()

Description

The SL_FILEIO.Version function returns the version number for the SL_FILEIO module as a string. The format of the string is *n1.n2.n3*.

2.10 The SL_HASH module

This chapter describes the SL_HASH module. The SL_HASH module contains hash table routines.

The error type generated by the SL_HASH module functions and statements is SL_HASH. The following program fragment demonstrates how to handle SL_HASH errors:

```
INCLUDE "sl_hash.s4h";
TRY {
    SL_HASH.Get $handle KEY "key1" VALUE $value;
}
CATCH $exception
WHEN SL_HASH.$ExceptionName {
    PRINT "Key does not exist\n";
}
```

All of the SL_HASH module functions and statements are declared in the library file *sl_hash.s4m*. A number of constants (for example, SL_HASH.\$ExceptionName and error codes) that are useful when using the SL_HASH module can be found in the include file *sl_hash.s4h*.

2.10.1 Summary of functions and statements

Function/Statement	Used For
SL_HASH.Get	Retrieving a value from a hash table.
SL_HASH.Initialize	Creating a new hash table.
SL_HASH.Put	Inserting a value into a hash table.
SL_HASH.Delete	Deleting a value from a hash table.
SL_HASH.Size	Obtaining the number of keys in a hash table.
SL_HASH.EnumerateNext	Enumerating the keys and values in a hash table.
SL_HASH.Clear	Re-initializing a hash table.
SL_HASH.Version	Returning the version number for the module.

2.10.2 SL_HASH.Initialize

Name

SL_HASH.Initialize. – Function that initializes a new hash table.

Syntax

SL_HASH.Initialize()

Description

The SL_HASH.Initialize function initializes a new hash table and returns a handle to it. The handle is an anonymous record of type SL_HASH.Handle. It is needed when inserting new values into and retrieving values from the hash table.

Examples

Refer to the example for the SL_HASH.Get statement.

2.10.3 SL_HASH.Put

Name

SL_HASH.Put – Insert a value into a hash table.

Syntax

SL_HASH.Put *handle* KEY *key* VALUE *value*;

Description

The SL_HASH.Put statement inserts a new value *value* into the hash table specified by *handle*. The *handle* parameter is a record of type SL_HASH.Handle previously returned by the SL_HASH.Initialize function, the *key* parameter is the key corresponding to the value, and *value* is the value itself. Both *key* and *value* should be of string types.

If the specified key already exists in the hash table, the old value is replaced by *value*.

Examples

Refer to the example for the SL_HASH.Get statement.

2.10.4 SL_HASH.Get

Name

SL_HASH.Get – Retrieve a value from a hash table.

Syntax

SL_HASH.Get *handle* KEY *key* VALUE *value*;

Description

The SL_HASH.Get statement retrieves the value corresponding to the key *key* from the hash table specified by *handle*. The *handle* parameter is a record of type SL_HASH.Handle previously returned by the SL_HASH.Initialize function, the *key* parameter is the key, and *value* is a *variable* that the value corresponding to *key* is assigned to.

Errors

The following value can be assigned to `$Error` (the error type is SL_HASH):

```
SL_HASH.$Error_Get_KeyNotFound
```

The hash table contains no value with *key* as a key.

Examples

If the following program is executed:

```
INCLUDE "sl_hash.s4h";
DECLARE VARIABLE $handle RECORD SL_HASH.Handle;
DECLARE VARIABLE $value STRING;

$handle = SL_HASH.Initialize();
SL_HASH.Put $handle KEY "key1" VALUE "value1";
SL_HASH.Put $handle KEY "key2" VALUE "value2";
SL_HASH.Put $handle KEY "key3" VALUE "value3";
SL_HASH.Put $handle KEY "key2" VALUE "new value2";
SL_HASH.Get $handle KEY "key1" VALUE $value;
PRINT $value & "\n";
SL_HASH.Get $handle KEY "key2" VALUE $value;
PRINT $value & "\n";
SL_HASH.Get $handle KEY "key3" VALUE $value;
PRINT $value & "\n";
```

the following is written to the output file:

```
value1
new value2
value3
```

2.10.5 SL_HASH.Delete

Name

SL_HASH.Delete – Remove a key from a hash table.

Syntax

SL_HASH.Delete *handle* KEY *key*;

Description

The `SL_HASH.Delete` statement removes the key *key* and its associated value from the hash table specified by *handle*. The *handle* parameter is a record of type `SL_HASH.Handle` previously returned by the `SL_HASH.Initialize` function and the *key* parameter is the key.

Errors

The following value can be assigned to `$Error` (the error type is `SL_HASH`):

```
SL_HASH.$Error_Delete_KeyNotFound
```

The hash table contains no value with *key* as a key.

Examples

```
SL_HASH.Delete $handle KEY "key2";
```

2.10.6 SL_HASH.Size

Name

`SL_HASH.Size`. - Function that returns the number of key-value pairs in a hash table.

Syntax

`SL_HASH.Size(handle)`

Description

This function returns the number of key-value pairs from the hash table specified by *handle*.

Example

```
LOG "this hash table contains " & SL_HASH.Size($handle) & " elements"
TYPE "INFO";
```

2.10.7 SL_HASH.EnumerateNext

Name

`SL_HASH.EnumerateNext`. - Statement that can be used to enumerate all key-value pairs in a hash table.

Syntax

`SL_HASH.EnumerateNext handle Iterator iterator Key key Value value EndOfHash endofhash;`

Description

The EnumerateNext statement returns the next key-value pair from the hash table specified by *handle*. The *iterator* statement must be initialized with SL_HASH.\$InitialIterator to obtain the first key-value pair from a hash table. Unspecified behavior results if the *handle* hash table is modified and the *iterator* is not reinitialized as SL_HASH.\$InitialIterator. The *endofhash* parameter will be set to true if the statement cannot return any more key-value pairs from this hash table, otherwise the *key* parameter will receive the key, and the *value* parameter will receive the value.

Example

```
DECLARE $iterator RECORD SL_HASH.Iterator;
DECLARE $endOfHash INTEGER;
DECLARE $key STRING;
DECLARE $value STRING;

$iterator = SL_HASH.$InitialIterator;
WHILE 1 {
    SL_HASH.EnumerateNext $handle ITERATOR $iterator KEY $key VALUE
    $value ENDOFHASH $endOfHash;
    IF $endOfHash {
        BREAK;
    }
    LOG "this hash maps " & $key & " to " & $value TYPE "INFO";
}
```

2.10.8 SL_HASH.Clear

Name

SL_HASH.Clear. - Statement that releases the memory allocated for a hash table.

Syntax

SL_HASH.Clear *handle*;

Description

This statement removes all the key-value pairs from a hash table. It also releases the allocated memory.

Example

```
SL_HASH.Clear $handle;
```

2.10.9 SL_HASH.Version

Name

SL_HASH.Version. - Function that returns the version number for the SL_HASH module.

Syntax

SL_HASH.Version()

Description

The SL_HASH.Version function returns the version number for the SL_HASH module as a string. The format of the string is *n1.n2.n3*.

2.11 The SL_PACKED module

This chapter describes the SL_PACKED module. The SL_PACKED module contains routines to convert integers and floats to packed decimal numbers and vice versa.

The error type generated by the SL_PACKED module functions is SL_PACKED. The following program fragment demonstrates how to handle SL_PACKED errors:

```
INCLUDE "sl_packed.s4h";
TRY {
    PRINT SL_PACKED.IbmPackedDecimalToFloat($packed, 2);
}
CATCH $exception
WHEN SL_PACKED.$ExceptionName {
    PRINT "Failed to convert packed number to float\n";
}
```

All of the SL_PACKED module functions are declared in the library file *sl_packed.s4m*. A number of constants (for example, SL_PACKED.\$ExceptionName and error codes) that are useful when using the SL_PACKED module can be found in the include file *sl_packed.s4h*.

2.11.1 Summary of functions

Function	Used For
SL_PACKED.IbmFloatToPackedDecimal	Converting an MB float to a packed decimal.
SL_PACKED.IbmPackedDecimalToFloat	Converting a packed decimal to an MB float.
SL_PACKED.SapInt4ToInteger	Converting a packed integer to an MB integer.
SL_PACKED.SapIntegerToInt4	Converting an MB integer to a packed integer.
SL_PACKED.Version	Returning the version number for the module.

2.11.2 SL_PACKED.SapIntegerToInt4

Name

SL_PACKED.SapIntegerToInt4 – Function that converts an MB integer to a packed integer.

Syntax

SL_PACKED.SapIntegerToInt4 (*integer*)

Description

The `SL_PACKED.SapIntegerToInt4` function converts an MB integer type to a SAP RFC_INT4 type. The *integer* argument is the integer to convert. The returned result is of string type.

See also

`SL_PACKED.SapInt4ToInteger`.

2.11.3 SL_PACKED.SapInt4ToInteger

Name

`SL_PACKED.SapInt4ToInteger` – Function that converts a packed integer to an MB integer.

Syntax

`SL_PACKED.SapInt4ToInteger(packed)`

Description

The `SL_PACKED.SapInt4ToInteger` function converts a SAP RFC_INT4 type to an MB integer. The *packed* argument should be of string type and contain a packed integer on RFC_INT4 format. The MB integer value is returned.

See also

`SL_PACKED.SapIntegerToInt4`.

2.11.4 SL_PACKED.IbmFloatToPackedDecimal

Name

`SL_PACKED.IbmFloatToPackedDecimal` – Function that converts an MB float to a packed decimal.

Syntax

`SL_PACKED.IbmFloatToPackedDecimal(float, length, decimals
[, plus-nibble[, minus-nibble]])`

Description

The `SL_PACKED.IbmFloatToPackedDecimal` function converts an MB float value to an IBM RFC_BCD packed type.

The *float* argument is the MB float value to convert, the *length* argument is the length (in bytes) of the returned RFC_BCD encoded float value, and *decimals* is the number of digits after the implicit decimal point in the RFC_BCD type. The *plus-nibble* and *minus-nibble* arguments are used to indicate how to encode the sign of the floating point

value. The least significant nibble of the encoded value contains this information. Valid values for *plus-nibble* are the strings "a", "c", "e" or "f" and the default value is "c". For *minus-nibble*, the valid values are "b" or "d" and the default value is "d".

The return value is of string type.

Errors

The following values can be assigned to `$Error` (the error type is `SL_PACKED`):

- `SL_PACKED.$Error_IbmFloatToPackedDecimal_InvalidLength`
The *length* argument is too short. The floating point value does not fit into a buffer of the specified length.
- `SL_PACKED.$Error_IbmFloatToPackedDecimal_InvalidDecimalsCount`
The *decimals* argument is negative.
- `SL_PACKED.$Error_IbmFloatToPackedDecimal_InvalidPlusNibble`
The *plus-nibble* argument is invalid.
- `SL_PACKED.$Error_IbmFloatToPackedDecimal_InvalidMinusNibble`
The *minus-nibble* argument is invalid.

See also

`SL_PACKED.IbmPackedDecimalToFloat`.

2.11.5 SL_PACKED.IbmPackedDecimalToFloat

Name

`SL_PACKED.IbmPackedDecimalToFloat` – Function that converts a packed decimal to a float.

Syntax

```
SL_PACKED.IbmPackedDecimalToFloat(buffer, decimals  
[, plus-nibble[, minus-nibble]])
```

Description

The `SL_PACKED.IbmPackedDecimalToFloat` function converts an IBM RFC_BCD packed type to an MB float value.

The *buffer* argument is the RFC_BCD packed type to convert to an MB float value and *decimals* is the number of digits after the implicit decimal point in the RFC_BCD type. The *plus-nibble* and *minus-nibble* arguments are used to indicate how the sign of the floating point value is encoded. The least significant nibble of the encoded value contains this information. Valid values for *plus-nibble* are the strings "a", "c", "e" or "f" and the default value is "c". For *minus-nibble*, the valid values are "b" or "d" and the default value is "d".

The return value is of float type.

Errors

The following values can be assigned to `$Error` (the error type is `SL_PACKED`):

- `SL_PACKED.$Error_IbmPackedDecimalToFloat_InvalidDecimalsCount`
The *decimals* argument is negative.
- `SL_PACKED.$Error_IbmPackedDecimalToFloat_InvalidPlusNibble`
The *plus-nibble* argument is invalid.
- `SL_PACKED.$Error_IbmPackedDecimalToFloat_InvalidMinusNibble`
The *minus-nibble* argument is invalid.
- `SL_PACKED.$Error_IbmPackedDecimalToFloat_InvalidValue`
The *buffer* argument contains an invalid RFC_BCD packed decimal value.

See also

`SL_PACKED.IbmFloatToPackedDecimal`.

2.11.6 SL_PACKED.Version

Name

`SL_PACKED.Version`. – Function that returns the version number for the `SL_PACKED` module.

Syntax

`SL_PACKED.Version()`

Description

The `SL_PACKED.Version` function returns the version number for the `SL_PACKED` module as a string. The format of the string is *n1.n2.n3*.

2.12 The SL_PATH module

This chapter describes the `SL_PATH` module. The `SL_PATH` module contains path name handling routines.

All of the `SL_PATH` module functions and statements are declared in the library file *sl_path.s4m*.

Since the `SL_PATH` module uses the `MODULE_INIT` feature for initialization, no functions or statements from the `SL_PATH` module should be called from other `MODULE_INIT` statements.

2.12.1 Summary of functions and statements

Function/Statement	Used For
<code>SL_PATH.AppendSeparator</code>	Appending a platform-specific directory separator.
<code>SL_PATH.Compare</code>	Comparing two path names.

Function/Statement	Used For
SL_PATH.CompareFilter	Filtering and comparing two path names.
SL_PATH.DirectorySeparator	Obtaining the platform-specific directory separator.
SL_PATH.Filter	Filtering a path name.
SL_PATH.ReplaceSlashes	Replacing with platform-specific type of slash.
SL_PATH.Version	Returning the version number for the module.

2.12.2 SL_PATH.Filter

Name

SL_PATH.Filter – Function that filters a path name.

Syntax

SL_PATH.Filter(*path*)

Description

The SL_PATH.Filter function removes redundant sequences of characters from the *path* argument and returns the result.

The *path* arguments should be of string type and the returned value is a string.

Examples

The following program:

```
PRINT SL_PATH.Filter("abc/xyz/../rst/./file") & "\n";
```

outputs the string:

```
abc/rst/file
```

Notes

On a POSIX file system (UNIX and AS/400), forward slashes are expected to separate path name components. On Windows systems, backward slashes separate path name components.

2.12.3 SL_PATH.Compare

Name

SL_PATH.Compare – Function that compares two path names.

Syntax

SL_PATH.Compare(*path1*, *path2*)

Description

The `SL_PATH.Compare` function compares two path names. It returns non-zero if the path names are equal or zero if they are not. On UNIX and AS/400 machines, this function just do an ordinary string compare. On Windows, lower case letters in both path names are converted to uppercase letters before the path names are compared.

Both arguments should be of string type.

Examples

On a Windows machine the following program will output a non-zero value and on a UNIX or AS/400 machine, it will output zero:

```
PRINT SL_PATH.Compare("abc/file", "ABC/FILE") & "\n";
```

2.12.4 SL_PATH.CompareFilter

Name

`SL_PATH.CompareFilter` – Function that filters and compares two path names.

Syntax

`SL_PATH.CompareFilter(path1, path2)`

Description

Calling this function is equivalent to calling:

`SL_PATH.Compare(SL_PATH.Filter(path1), SL_PATH.Filter(path2))`

Examples

On a UNIX or AS/400 machine, this program outputs a non-zero value:

```
PRINT SL_PATH.CompareFilter("abc/xyz/../file", "abc/file") & "\n";
```

2.12.5 SL_PATH.AppendSeparator

Name

SL_PATH.AppendSeparator – Returns a path that ends in a separator, and which does not represent the file system root.

Syntax

SL_PATH.AppendSeparator(*path*)

Description

If *path* is empty `"/"` is returned. Otherwise, a slash (or backslash, on Windows) is appended to the end of *path*, if not already present.

Examples

On a UNIX or AS/400 machine, this one-line program:

```
PRINT SL_PATH.AppendSeparator("xx/yy") & "\n";
```

outputs to the standard output:

```
xx/yy/
```

2.12.6 SL_PATH.DirectorySeparator

Name

SL_PATH.DirectorySeparator – Returns the directory separator for this platform.

Syntax

SL_PATH.DirectorySeparator()

Description

On UNIX and OS/400, returns one forward slash. On Windows, returns one backslash.

2.12.7 SL_PATH.ReplaceSlashes

Name

SL_PATH.ReplaceSlashes – Replace the directory separators with the correct type of directory separators.

Syntax

SL_PATH.ReplaceSlashes(*path*)

Description

Returns a string which is identical to *path* except that it only contains slashes (on UNIX and OS/400) or backslashes (on Windows) as directory separators.

Examples

On a UNIX or AS/400 machine, this program:

```
PRINT SL_PATH.ReplaceSlashes("xx\\yy/zz") & "\n";
```

outputs to the standard output:

```
xx/yy/zz
```

2.12.8 SL_PATH.Version

Name

SL_PATH.Version. – Function that returns the version number for the SL_PATH module.

Syntax

SL_PATH.Version()

Description

The SL_PATH.Version function returns the version number for the SL_PATH module as a string. The format of the string is *n1.n2.n3*.

2.13 The SL_SEARCH module

This chapter describes the SL_SEARCH module. The SL_SEARCH module contains routines for searching arrays.

All of the SL_SEARCH module functions are declared in the library file *sl_search.s4m*.

2.13.1 Summary of functions

Function	Used For
SL_SEARCH.BinarySearch	Binary search on a sorted array.
SL_SEARCH.Version	Returning the version number for the module.

2.13.2 SL_SEARCH.BinarySearch

Name

SL_SEARCH.BinarySearch – Function that performs a binary search on a sorted array.

Syntax

SL_SEARCH.BinarySearch(*array*, *search*)

Description

The SL_SEARCH.BinarySearch function uses binary search to search the string array *array* for an occurrence of the string *search*. If *search* is found, the index for the matching element is returned. If *search* is not found, zero is returned.

For SL_SEARCH.BinarySearch to return a correct result, *array* must be sorted. The SL_SORT.QuickSort_String or SL_SORT.HeapSort_String statements can be used to do the sort.

The *array* argument should be a string array and *search* should be of string type.

Examples

The following example reads up to 10 strings from the input file into an array, sorts the array, and finally searches for the string *search* in the array:

```
DECLARE VARIABLE $array[] STRING;
DECLARE VARIABLE $index INTEGER;

FOR $i = 1 TO 10 {
    READ $array[$i] UNTIL "\n";
}
SL_SORT.QuickSort_String $array;
$index = SL_SEARCH.BinarySearch($array, "search");
IF $index = 0 {
    PRINT "Search string not found.\n";
}
ELSE {
    PRINT "Search string found at index " & $index & "\n";
}
```

See also

SL_SORT.QuickSort_String, SL_SORT.HeapSort_String.

2.13.3 SL_SEARCH.Version

Name

SL_SEARCH.Version. – Function that returns the version number for the SL_SEARCH module.

Syntax

SL_SEARCH.Version()

Description

The SL_SEARCH.Version function returns the version number for the SL_SEARCH module as a string. The format of the string is *n1.n2.n3*.

2.14 The SL_SORT module

This chapter describes the SL_SORT module. The SL_SORT module contains routines for sorting arrays.

All of the SL_SORT module functions and statements are declared in the library file *sl_sort.s4m*.

2.14.1 Summary of functions and statements

Function/Statement	Used For
SL_SORT.HeapSort_Any	Sorting an array using heap-sort.
SL_SORT.HeapSort_String	Sorting a string array using heap-sort.
SL_SORT.QuickSort_Integer	Sorting an array of integers.
SL_SORT.QuickSort_String	Sorting a string array using quick-sort.
SL_SORT.Version	Returning the version number for the module.

2.14.2 SL_SORT.HeapSort_Any

Name

SL_SORT.HeapSort_Any – Sort an array using heap-sort.

Syntax

SL_SORT.HeapSort_Any *array* COMPARE_IMPL *compare-function*;

Description

The SL_SORT.HeapSort_Any statement sorts an array using the heap-sort algorithm. The *array* parameter is the array to sort. The *compare-function* parameter is the name of the function used to compare two elements from *array*. It should take two arguments (the array elements to compare) and return minus one if the first element is less than the second, zero if the elements are equal, or plus one if the first element is greater than the second one. If the type of the elements to sort is a record of type *Elem*, the compare function might look as follows:

```
DECLARE FUNCTION
Compare($r1 RECORD Elem, $r2 RECORD Elem) PRIMITIVE
{
```

```

        IF $r1.$Name > $r2.$Name {
            RETURN 1;
        }
        IF $r1.$Name = $r2.$Name {
            RETURN 0;
        }
        RETURN -1;
    }
}

```

Examples

The following Sort statement sorts an array of *Elem* records:

```

DECLARE RECORD Elem {
    DECLARE FIELD $Name STRING;
    DECLARE FIELD $Other INTEGER;
}

DECLARE STATEMENT
Sort IN $r[] RECORD Elem
{
    SL_SORT.HeapSort_Any $r COMPARE_IMPL FIDENT(CompareElem);
    RETURN;
}

DECLARE FUNCTION
CompareElem($r1 RECORD Elem, $r2 RECORD Elem) PRIMITIVE
{
    IF $r1.$Name > $r2.$Name {
        RETURN 1;
    }
    IF $r1.$Name = $r2.$Name {
        RETURN 0;
    }
    RETURN -1;
}

```

2.14.3 SL_SORT.HeapSort_String

Name

SL_SORT.HeapSort_String – Sort a string array using heap-sort.

Syntax

SL_SORT.HeapSort_String *array*;

Description

The SL_SORT.HeapSort_String statement sorts an array of strings using the heap-sort algorithm. The *array* parameter is the array to sort. It should be a one-dimensional string array.

Examples

If the following program is executed:


```

DECLARE VARIABLE $array[] STRING;

$array[1] = "example string";
$array[2] = "another example string";
$array[3] = "etc";
$array[4] = "sort example";
$array[5] = "heap-sort";
SL_SORT.HeapSort_String $array;
FOR $i = 1 TO ARRAYSIZE[$array] {
    PRINT $array[$i] & "\n";
}

```

the following is written to the output file:

```

another example string
etc
example string
heap-sort
sort example

```

2.14.4 SL_SORT.QuickSort_String

Name

SL_SORT.QuickSort_String – Sort a string array using quick-sort.

Syntax

SL_SORT.QuickSort_String *array* [FROM *from*] [TO *to*];

Description

The SL_SORT.QuickSort_String statement sorts an array of strings, or a part of it, using the quick-sort algorithm.

The *array* parameter is the array to sort. It should be a one-dimensional string array. The *from* and *to* parameters are the indexes for the first and last elements, respectively, to include in the sort. They should be of integer type. The default for *from* is one and the default for *to* is the size of the array.

Examples

If the following program is executed:

```

DECLARE VARIABLE $array[] STRING;

$array[1] = "example string";
$array[2] = "another example string";
$array[3] = "etc";
$array[4] = "sort example";
$array[5] = "quick-sort";
SL_SORT.QuickSort_String $array;
FOR $i = 1 TO ARRAYSIZE[$array] {
    PRINT $array[$i] & "\n";
}

```

the following is written to the output file:

```

another example string
etc

```

```
example string
quick-sort
sort example
```

2.14.5 SL_SORT.QuickSort_Integer

Name

SL_SORT.QuickSort_Integer – Sort an integer array using quick-sort.

Syntax

SL_SORT.QuickSort_Integer *array* [FROM *from*] [TO *to*];

Description

The SL_SORT.QuickSort_Integer statement sorts an array of integers, or a part of it, using the quick-sort algorithm.

The *array* parameter is the array to sort. It should be a one-dimensional integer array. The *from* and *to* parameters are the indexes for the first and last elements, respectively, to include in the sort. They should be of integer type. The default for *from* is one and the default for *to* is the size of the array.

Examples

If the following program is executed:

```
DECLARE VARIABLE $array[] INTEGER;

$array[1] = 6;
$array[2] = 3;
$array[3] = 12;
$array[4] = 21;
$array[5] = 13;
SL_SORT.QuickSort_Integer $array;
FOR $i = 1 TO ARRAYSIZE[$array] {
    PRINT $array[$i] & "\n";
}
```

the following is written to the output file:

```
3
6
12
13
21
```

2.14.6 SL_SORT.Version

Name

SL_SORT.Version. – Function that returns the version number for the SL_SORT module.

Syntax

SL_SORT.Version()

Description

The SL_SORT.Version function returns the version number for the SL_SORT module as a string. The format of the string is *n1.n2.n3*.

2.15 The SL_STDLIB module

This chapter describes the SL_STDLIB module. The SL_STDLIB module contains routines that are common to the Standard Library.

All of the SL_STDLIB module functions are declared in the library file *sl_stdlib.s4m*.

2.15.1 Summary of functions

Function	Used For
SL_STDLIB.Version	Returning the version number for the Standard Library.

2.15.2 SL_STDLIB.Version

Name

SL_STDLIB.Version. – Function that returns the version number for the Standard Library.

Syntax

SL_STDLIB.Version()

Description

The SL_STDLIB.Version function returns the version number for the Standard Library as a string. The format of the string is *n1.n2.n3*.

Each module in the Standard Library has its own version number. There is also a version number that is common for the whole Standard Library. The SL_STDLIB.Version function returns this version number.

2.16 The SL_STRING module

This chapter describes the SL_STRING module. The SL_STRING module contains string-handling routines.

The error type generated by the SL_STRING module functions is SL_STRING. The following program fragment demonstrates how to handle SL_STRING errors:

```
INCLUDE "sl_string.s4h";
TRY {
    PRINT SL_STRING.FieldExtract("field A:field B:field C", 3, ":");
}
CATCH $exception
WHEN SL_STRING.$ExceptionName {
    PRINT "Failed to extract field\n";
}
```

All of the SL_STRING module functions and statements are declared in the library file *sl_string.s4m*. A number of constants (for example, SL_STRING.\$ExceptionName and error codes) that are useful when using the SL_STRING module can be found in the include file *sl_string.s4h*.

2.16.1 Summary of functions

Function	Used For
SL_STRING.Append	Appending to a string.
SL_STRING.AsciiToEbcDic	Converting a string of ASCII characters to EBCDIC.
SL_STRING.EbcDicToAscii	Converting a string of EBCDIC characters to ASCII.
SL_STRING.FieldCount	Counting the number of fields in a string.
SL_STRING.FieldExplode	Extracting all fields from a string.
SL_STRING.FieldExtract	Extracting a field from a string.
SL_STRING.FieldImplode	Concatenating an array of strings into one string.
SL_STRING.FindInSet	Finding first occurrence of character from set.
SL_STRING.FindNotInSet	Finding first occurrence of character not from set.
SL_STRING.Remove	Removing substrings from a string.
SL_STRING.Replace	Replacing substrings inside a string.
SL_STRING.Reverse	Reversing a string.
SL_STRING.ToLower	Converting all uppercase letters to lowercase.
SL_STRING.ToUpper	Converting all lowercase letters to uppercase.
SL_STRING.WildcardMatch	Matching wildcards in string.
SL_STRING.Version	Returning the version number for the module.

2.16.2 SL_STRING.AsciiToEbcDic

Name

SL_STRING.AsciiToEbcDic – Function that converts ASCII characters to EBCDIC.

Syntax

SL_STRING.AsciiToEbcDic(*string*)

Description

The `SL_STRING.AsciiToEbcDic` function converts all characters in the function argument from ASCII (actually ISO8859-1) characters to EBCDIC (code page 500) characters. The resulting EBCDIC string is returned.

The *string* argument is the string to convert to EBCDIC. On entry, it is assumed that this string contains ISO8859-1 characters.

See also

`SL_STRING.EbcDicToAscii`.

2.16.3 `SL_STRING.EbcDicToAscii`

Name

`SL_STRING.EbcDicToAscii` – Function that converts EBCDIC characters to ASCII.

Syntax

`SL_STRING.EbcDicToAscii(string)`

Description

The `SL_STRING.EbcDicToAscii` function converts all characters in the function argument from EBCDIC (code page 500) characters to ASCII (actually ISO8859-1) characters. The resulting ISO8859-1 string is returned.

The *string* argument is the string to convert to ISO8859-1. On entry, it is assumed that this string contains EBCDIC characters.

See also

`SL_STRING.AsciiToEbcDic`.

2.16.4 `SL_STRING.FieldCount`

Name

`SL_STRING.FieldCount` – Function that counts the number of fields in a string.

Syntax

`SL_STRING.FieldCount(string[, separators])`

Description

The `SL_STRING.FieldCount` function returns the number of fields in a string as an integer.

A field is all the characters between two separator characters, between the start of the string and the first separator character, or between the last separator character and the

end of the string. If there are no separator characters in the string, it contains one field (the string itself).

The string to count the number of fields in is *string*. The *separators* argument specifies all characters that act as field separators. The default separator character is a comma.

If the *string* argument is an empty string, zero is returned. Otherwise, the return value is always greater than zero.

Examples

The following program:

```
PRINT SL_STRING.FieldCount("field A:field B,field C:field D", ":",");
```

prints the following:

4

See also

SL_STRING.FieldExtract.

2.16.5 SL_STRING.FieldExtract

Name

SL_STRING.FieldExtract – Function that extracts a field from a string.

Syntax

SL_STRING.FieldExtract(*string*, *field-number*[, *separators*])

Description

The SL_STRING.FieldExtract function extracts and returns the specified field from a string.

A field is all the characters between two separator characters, between the start of the string and the first separator character, or between the last separator character and the end of the string. If there are no separator characters in the string, it contains one field (the string itself).

The string to extract the field from is *string*. The field to extract is specified by *field-number* (the first field has number 1). The *separators* argument specifies all characters that act as field separators. The default separator character is a comma.

Errors

The following values can be assigned to `$Error` (the error type is SL_STRING):

- SL_STRING.\$Error_FieldExtract_IllegalFieldNumber
Invalid field number, that is, the *field-number* argument is less than or equal to zero.
- SL_STRING.\$Error_FieldExtract_FieldNumberOutsideString
The *field-number* argument is greater than the number of fields in *string*.

Examples

The following program:

```
PRINT SL_STRING.FieldExtract("field A:field B,field C:field D",  
    3, ":",");
```

prints the following:

```
field C
```

See also

SL_STRING.FieldCount.

2.16.6 SL_STRING.FieldExplode

Name

SL_STRING.FieldExplode – Function that extracts all fields from a string.

Syntax

SL_STRING.FieldExplode(*string*[, *separator*])

Description

The SL_STRING.FieldExplode function extracts the fields in a string.

A field is all the characters between two separator characters, between the start of the string and the first separator character, or between the last separator character and the end of the string. If there are no separator characters in *string*, the string contains one field (the string itself).

A string array containing all the different fields is returned. The order of the fields in this array is the same as the original order in the string. If *string* is empty, an empty array is returned.

The *string* argument is the string to extract fields from. It should be of string type. The *separator* argument is the character that separates fields. It should be a one-character string. The default separator is a comma.

Examples

If the following program is executed:

```
DECLARE VARIABLE $array[] STRING;  
  
$array = SL_STRING.FieldExplode("field A:field B:field C:field D",  
    ":");  
FOR $i = 1 TO ARRAYSIZE($array) {  
    PRINT $array[$i] & "\n";  
}
```

the following is written to the output file:

```
field A
field B
field C
field D
```

See also

SL_STRING.FieldImplode.

2.16.7 SL_STRING.FieldImplode

Name

SL_STRING.FieldImplode – Function that concatenates the elements of an array to a string.

Syntax

SL_STRING.FieldImplode(*array*[, *separator*])

Description

The SL_STRING.FieldImplode function builds and returns a string whose fields are the elements of the *array* string array. The *separator* argument is the character that separates fields.

The *array* argument should be a string array and *separator* should be a one-character string. The default separator is a comma.

Examples

If the following program is executed:

```
DECLARE VARIABLE $array[] STRING;

$array[1] = "field A";
$array[2] = "field B";
$array[3] = "field C";
$array[4] = "field D";
PRINT SL_STRING.FieldImplode($array, ":");
```

the following is written to the output file:

```
field A:field B:field C:field D
```

See also

SL_STRING.FieldExplode.

2.16.8 SL_STRING.Append

Name

SL_STRING.Append – Append a string to another string

Syntax

SL_STRING.Append *dest* STRING *toBeAppended*;

Description

The SL_STRING.Append statement appends *toBeAppended* to *dest* in a way that is guaranteed to generate optimal code when compiled (unless compiled with optimizations disabled).

Examples

The following program:

```
DECLARE VARIABLE $string STRING;

$string = "abc";
SL_STRING.Append $string STRING "def";
PRINT $string & "\n";
```

Outputs the following:

```
abcdef
```

2.16.9 SL_STRING.Remove

Name

SL_STRING.Remove – Remove substrings from a string

Syntax

SL_STRING.Remove(*string*, *toBeRemoved*)

Description

The SL_STRING.Remove function returns the string *string* with all occurrences of *toBeRemoved* removed.

Examples

The following one-line program:

```
PRINT SL_STRING.Remove("123abc456abc", "abc") & "\n";
```

Outputs the following:

```
123456
```

2.16.10 SL_STRING.Replace

Name

SL_STRING.Replace – Replace substrings inside a string

Syntax

SL_STRING.Replace(*string*, *old*, *new*)

Description

The SL_STRING.Replace function returns the string *string* with all occurrences of *old* replaced with *new*.

Examples

The following one-line program:

```
PRINT SL_STRING.Remove("123abc456abc", "abc", "def") & "\n";
```

Outputs the following:

```
123def456def
```

2.16.11 SL_STRING.Reverse

Name

SL_STRING.Reverse – Reverse a string.

Syntax

SL_STRING.Reverse *string*;

Description

The SL_STRING.Reverse statement reverses the contents of the *string* parameter. On return the *string* parameter contains the reversed string.

Examples

The following program:

```
DECLARE VARIABLE $string STRING;  
  
$string = "reverse it";  
SL_STRING.Reverse $string;  
PRINT $string & "\n";
```

Outputs the following:

```
ti esrever
```

2.16.12 SL_STRING.ToLower

Name

SL_STRING.ToLower – Function that converts uppercase letters to lowercase.

Syntax

SL_STRING.ToLower(*string*)

Description

The SL_STRING.ToLower function converts all uppercase letters in *string* to lowercase. The converted string is returned.

Examples

The following one-line program:

```
PRINT SL_STRING.ToLower("STANDARD LIBRARY") & "\n";
```

outputs the following:

```
standard library
```

See also

SL_STRING.ToUpper.

2.16.13 SL_STRING.ToUpper

Name

SL_STRING.ToUpper – Function that converts lowercase letters to uppercase.

Syntax

SL_STRING.ToUpper(*string*)

Description

The SL_STRING.ToUpper function converts all lowercase letters in *string* to uppercase. The converted string is returned.

Examples

The following one-line program:

```
PRINT SL_STRING.ToUpper("standard library ") & "\n";
```

outputs the following:

```
STANDARD LIBRARY
```

See also

SL_STRING.ToLower.

2.16.14 SL_STRING.FindInSet

Name

SL_STRING.FindInSet – Function that finds the first character that matches a character in a specified set

Syntax

SL_STRING.FindInSet(*string*, *set*[, *start*])

Description

The position of the first character in *string* that is also in the string *set* is returned, or zero if no such character is found. The search starts from position *start*, if provided, else from the beginning of *string*.

Examples

The following one-line program:

```
PRINT SL_STRING.FindInSet("abcd", "gfd") & "\n";
```

outputs the following:

4

See also

SL_STRING.FindNotInSet.

2.16.15 SL_STRING.FindNotInSet

Name

SL_STRING.FindNotInSet – Function that finds the first character that does not match a character in a specified set

Syntax

SL_STRING.FindNotInSet(*string*, *set*[, *start*])

Description

The position of the first character in *string* that is not in the string *set* is returned, or zero if no such character is found. The search starts from position *start*, if provided, else from the beginning of *string*.

Examples

The following one-line program:

```
PRINT SL_STRING.FindNotInSet("dcba", "gfd") & "\n";
```

outputs the following:

See also

SL_STRING.FindInSet.

2.16.16 SL_STRING.WildcardMatch

Name

SL_STRING.WildcardMatch – Function that matches a wildcard pattern to a string.

Syntax

SL_STRING.WildcardMatch(*wildstring*, *string*)

Description

The SL_STRING.WildcardMatch function returns non-zero if the given *string* matches the given *wildstring*. *wildstring* can contain the * character, to match any number of characters, the ? character, which matches any single character. To match a literal \, * or ? character, use \\, * or, respectively, \?.

Examples

The following program:

```
IF SL_STRING.WildcardMatch("ab*d?f*i", "abcdefghi") {  
    PRINT "matches\n";  
}
```

outputs the following:

```
matches
```

2.16.17 SL_STRING.Version

Name

SL_STRING.Version. – Function that returns the version number for the SL_STRING module.

Syntax

SL_STRING.Version()

Description

The SL_STRING.Version function returns the version number for the SL_STRING module as a string. The format of the string is *n1.n2.n3*.

2.17 The SL_TAG module

This chapter describes the SL_TAG module. The SL_TAG module contains routines to read and write tags from and to files.

The error type generated by the SL_TAG module functions and statements is SL_TAG. The following program fragment demonstrates how to handle SL_TAG errors:

```
INCLUDE "sl_tag.s4h";
TRY {
    SL_TAG.Put "tagfile" TAG "TAG_A" VALUE "value A";
}
CATCH $exception
WHEN SL_TAG.$ExceptionName {
    PRINT "Failed to write tag\n";
}
```

All of the SL_TAG module functions and statements are declared in the library file *sl_tag.s4m*. A number of constants (for example, `SL_TAG.$ExceptionName` and error codes) that are useful when using the SL_TAG module can be found in the include file *sl_tag.s4h*.

Since the SL_TAG module uses the MODULE_INIT feature for initialization, no functions or statements from the SL_TAG module should be called from other MODULE_INIT statements.

2.17.1 Format of Tag files

The following information on the format of tag files is provided in case you want to create tag files using a third-party tool.

A tag file is a text file containing tags and values. A tag is a case-sensitive label used to identify a value. A value is any text. A tag and its value are separated by an equals character (=). All characters between the start of a line and the equals character are the tag name. The value is all the characters between the equals character and the end of the line. A value may continue on the next line if the last character of the line is a backslash character (\). The backslash and the new line characters are removed. The following special character sequences may be used in the value:

- \a Audible alarm (bell).
- \b Backspace.
- \f Form feed.
- \r Carriage return.
- \t Horizontal tab.
- \v Vertical tab.
- \xnn Hexadecimal value. nn is one or two hexadecimal digits.
- \0 Null byte.

Empty lines and lines starting with a semicolon character (;) are ignored.

2.17.2 Summary of functions and statements

Function/Statement	Used For
SL_TAG.Get	Reading a tag value from a file.
SL_TAG.List	Reading all tags and their values from a file.

Function/Statement	Used For
SL_TAG.Put	Writing a tag value to a file.
SL_TAG.Remove	Removing a tag and its value from a file.
SL_TAG.Version	Returning the version number for the module.

2.17.3 SL_TAG.Put

Name

SL_TAG.Put – Write a tag value to a file.

Syntax

SL_TAG.Put *file-name* TAG *tag-name* VALUE *value* [FORCE_READ *force-read*];

Description

The SL_TAG.Put statement writes a tag value to a file. If the specified tag already exists, the value of the tag is changed. Otherwise, the new tag is added to the file. If *file-name* does not exist, it is created.

The *file-name* parameter is the name of the file to add the tag to, *tag-name* is the name of the tag to add or modify, and *value* is the value.

If *force-read* is nonzero, the file is always read from disk. Otherwise, SL_TAG.Put may instead use a cached copy (and hence improve performance). The default value for *force-read* is zero.

The *file-name*, *tag-name* and *value* parameters should be of string type and *force-read* should be an integer.

Errors

The following values can be assigned to `$Error` (the error type is SL_TAG):

- SL_TAG.\$Error_Put_EmptyFilename
The *file-name* parameter is an empty string.
- SL_TAG.\$Error_Put_EmptyTagname
The *tag-name* parameter is an empty string.
- SL_TAG.\$Error_Put_ReadFail
Failed to read the contents of *file-name*.
- SL_TAG.\$Error_Put_OpenFail
Failed to open *file-name* for writing.
- SL_TAG.\$Error_Put_WriteFail
Failed to write the updated *file-name* file.

Examples

Refer to the example for the SL_TAG.Get statement.

2.17.4 SL_TAG.Get

Name

SL_TAG.Get – Function that reads a tag value from a file.

Syntax

SL_TAG.Get(file-name, tag-name[, force-read])

Description

The SL_TAG.Get function reads a tag value from a file. The tag value is returned.

The *file-name* argument is the name of the file to read the tag from and *tag-name* is the name of the tag.

If *force-read* is nonzero, the file is always read from disk. Otherwise, SL_TAG.Get may instead use a cached copy (and hence improve performance). The default value for *force-read* is zero.

The *file-name* and *tag-name* arguments should be of string type and *force-read* should be an integer.

Errors

The following values can be assigned to `$Error` (the error type is `SL_TAG`):

- `SL_TAG.$Error_Get_EmptyFilename`
The *file-name* argument is an empty string.
- `SL_TAG.$Error_Get_EmptyTagname`
The *tag-name* argument is an empty string.
- `SL_TAG.$Error_Get_OpenFail`
Failed to open the *file-name* file.
- `SL_TAG.$Error_Get_NoTag`
Could not find *tag-name* in *file-name*.
- `SL_TAG.$Error_Get_ReadFail`
Read error when reading the *file-name* file.

Examples

If the following program is executed:

```
SL_TAG.Put "tagfile" TAG "TAG_A" VALUE "value A";
SL_TAG.Put "tagfile" TAG "TAG_B" VALUE "value B";
SL_TAG.Put "tagfile" TAG "TAG_C" VALUE "value C";
SL_TAG.Put "tagfile" TAG "TAG_D" VALUE "value D";
SL_TAG.Put "tagfile" TAG "TAG_B" VALUE "new value B";
PRINT SL_TAG.Get("tagfile", "TAG_A") & "\n";
PRINT SL_TAG.Get("tagfile", "TAG_B") & "\n";
PRINT SL_TAG.Get("tagfile", "TAG_C") & "\n";
PRINT SL_TAG.Get("tagfile", "TAG_D") & "\n";
```

the following is written to the output file:

```
value A
new value B
value C
value D
```

2.17.5 SL_TAG.Remove

Name

`SL_TAG.Remove` – Remove a tag and its value from a file.

Syntax

`SL_TAG.Remove file-name TAG tag-name [FORCE_READ force-read];`

Description

The `SL_TAG.Remove` statement removes a tag and its value from a file.

The *file-name* parameter is the name of the file to remove the tag from and *tag-name* is the name of the tag to remove.

If *force-read* is nonzero, the file is always read from disk. Otherwise, `SL_TAG.Remove` may instead use a cached copy (and hence improve performance). The default value for *force-read* is zero.

The *file-name* and *tag-name* parameters should be of string type and *force-read* should be an integer.

Errors

The following values can be assigned to `$Error` (the error type is `SL_TAG`):

- `SL_TAG.$Error_Remove_EmptyFilename`
The *file-name* parameter is an empty string.
- `SL_TAG.$Error_Remove_EmptyTagname`
The *tag-name* parameter is an empty string.
- `SL_TAG.$Error_Remove_ReadFail`
Failed to read the contents of *file-name*.
- `SL_TAG.$Error_Remove_NoTag`
No tag named *tag-name* in *file-name*.
- `SL_TAG.$Error_Remove_OpenFail`
Failed to open *file-name* for writing.
- `SL_TAG.$Error_Remove_WriteFail`
Failed to write the updated *file-name* file.

Examples

The following program:

```
DECLARE VARIABLE $tags[] STRING;
DECLARE VARIABLE $values[] STRING;

SL_TAG.Put "tagfile" TAG "TAG_A" VALUE "value A";
SL_TAG.Put "tagfile" TAG "TAG_B" VALUE "value B";
SL_TAG.Put "tagfile" TAG "TAG_C" VALUE "value C";
SL_TAG.Put "tagfile" TAG "TAG_D" VALUE "value D";
SL_TAG.Put "tagfile" TAG "TAG_B" VALUE "new value B";
SL_TAG.Remove "tagfile" TAG "TAG_A";
SL_TAG.Remove "tagfile" TAG "TAG_C";
SL_TAG.List "tagfile" TAGS $tags VALUES $values;
FOR $i = 1 TO ARRAYSIZE($tags) {
    PRINT $tags[$i] & ":" & $values[$i] & "\n";
}
```

produces the following output (the order might be different):

```
TAG_B:new value B
TAG_D:value D
```

2.17.6 SL_TAG.List

Name

`SL_TAG.List` – Read all tags and their values from a file.

Syntax

`SL_TAG.List file-name [TAGS tags] [VALUES values] [FORCE_READ force-read];`

Description

The `SL_TAG.List` statement reads all the tags and their corresponding values from a file.

The *file-name* input parameter is the name of the file to read tags and values from and the *tags* and *values* output parameters are string arrays receiving the tags and values, respectively.

If *force-read* is nonzero, the file is always read from disk. Otherwise, `SL_TAG.List` may instead use a cached copy (and hence improve performance). The default value for *force-read* is zero.

The *file-name* parameter should be of string type and *force-read* should be an integer.

Errors

The following values can be assigned to `$Error` (the error type is `SL_TAG`):

- `SL_TAG.$Error_List_EmptyFilename`
The *file-name* parameter is an empty string.
- `SL_TAG.$Error_List_ReadFail`
Failed to read the contents of *file-name*.
- `SL_TAG.$Error_List_OpenFail`
Failed to open *file-name* for reading.

Examples

Refer to the example for the `SL_TAG.Remove` statement.

2.17.7 SL_TAG.Version

Name

`SL_TAG.Version.` – Function that returns the version number for the `SL_TAG` module.

Syntax

`SL_TAG.Version()`

Description

The `SL_TAG.Version` function returns the version number for the `SL_TAG` module as a string. The format of the string is *n1.n2.n3*.

3 AS/400 specific extensions

3.1 The AS400 extension

This section describes the Message Builder AS400 extension. The AS400 extension runs only the AS/400, and contains miscellaneous functions and statements specific to the AS/400.

The AS400 extension is implemented as a module. All of the AS400 extension functions and statements are declared in the library files `e_as400.s4` and `e_as400.t4`. A number of constants (for example, error codes) can be found in the include file `e_as400.s4h`.

3.2 Language reference

This section contains information about the functions and statements added by the AS400 extension.

3.2.1 AS400.BINDEC function

Name

AS400.BINDEC – This function converts numeric data into the AS/400 binary decimal format.

Syntax

buffer = AS400.BINDEC(*float*, *length*, *decimals*, *type*)

Description

The AS400.BINDEC function converts a numeric variable into a buffer that contains a number in AS/400 binary decimal format.

The *float* argument is the numeric data to be converted. This expression is converted to a floating point value.

The *length* argument is the number of digits of binary decimal data to be created. This expression is converted to an integer.

The *decimals* argument is the number of decimal digits in the binary decimal data. This expression is converted to an integer.

The *decimals* argument is used to multiply up the *float* argument before conversion.

If the result of this multiplication leaves additional decimal data, then this data will be lost.

If the result of this multiplication does not fit into the *length*, then the most significant data will be lost.

The *type* argument specifies whether the binary decimal data is signed or not. The symbolic variables *\$type_Signed_Binary* and *\$type_Unsigned_Binary* are defined in the header file, *e_as400.s4h*, for use with this argument.

The function returns a string type.

Errors

The following values can be assigned to *\$error*:

- *\$Error_AS400APIError*
Error in AS/400 API call
- *\$Error_BinDecIllegal*
An illegal argument was passed to the function.

Examples

The following statement converts a numeric value to a signed binary decimal of length 7 with 2 decimals, and writes it to a flat file with an associated tag string.

```
DECLARE $tagLength = 10 CONSTANT INTEGER;
DECLARE $binaryLength = 7 CONSTANT INTEGER;
DECLARE $binaryDecimals = 2 CONSTANT INTEGER;

DECLARE STATEMENT OUTPUTFILE IN $tag
                                VALUE IN $value
{
    PRINT LEFT($tag, $tagLength)
        & AS400.BINDEC($value,
                        $binaryLength,
                        $binaryDecimals,
                        $type_Signed_Binary)
        & "\n";

    RETURN;
}
```

See also

AS400.UNBDEC.

3.2.2 AS400.DIREINFO statement

Name

AS400.DIREINFO – This statement retrieves information from an AS/400 directory entry.

Syntax

AS400.DIREINFO *user* SYSTEM *system* [USRPRF *userProfile*]

Description

The AS400.DIREINFO statement retrieves information from an AS/400 directory entry.

The *user* and *system* arguments must be the names of the two parts of an existing AS/400 directory entry. These expressions are converted to strings.

The *userProfile* argument is the AS/400 user profile associated with the directory entry. The returned value is of string type.

Errors

The following values can be assigned to \$error:

- \$Error_AS400APIError
Error in AS/400 API call
 - \$Error_DireInfoNotFound
The directory entry was not found.
 - \$Error_DireInfoMultiple
More than one directory entry matched the specification of *user* and *system*.
-

Examples

The following processing checks if the Destination has an entry in the System Directory:

```
$Found = $TRUE;
TRY {
  AS400.DIREINFO $DestSub
    SYSTEM $Dest;
}
CATCH $exception
WHEN OTHERS {
  IF $error = $Error_DireInfoNotFound {
    $Found = $FALSE;
  }
}
```

Notes

This statement uses the QOKSCHD() AS/400 C library function.

3.2.3 AS400.FILEINFO statement

Name

AS400.FILEINFO – This statement retrieves information from an AS/400 stream file.

Syntax

AS400.FILEINFO *fileName* [CCSID *ccsid*]

Description

The AS400.FILEINFO statement retrieves information from an AS/400 stream file.

The *fileName* argument must be the name of an existing AS/400 stream file. This expression is converted to a string.

The *ccsid* argument is the AS/400 code character set identifier associated with the stream file. The returned value is of integer type. The values of code character set identifiers to be used should be determined by referring to the AS/400 system manuals.

Errors

The following values can be assigned to *\$error*:

- *\$Error_AS400APIError*
Error in AS/400 API call
 - *\$Error_FileInfoFail*
The file was not found.
-

Examples

The following user function checks the CCSID of a file, and converts the data if a particular CCSID is found.

```
DECLARE FUNCTION CONVERT_TO_MULTINATIONAL($InputString, $File)
{
  DECLARE $OutputString STRING;
  DECLARE $id INTEGER;

  AS400.FILEINFO $File CCSID $ccsid;
  IF $ccsid = 285 {
    $id = ICONV.COPEN(285, 500);
    $OutputString = ICONV.CONVERT($id, $InputString);
    ICONV.CCLOSE $id;
  }
  ELSE {
    $OutputString = $InputString;
  }

  RETURN $OutputString;
}
```

See also

ICONV.CCLOSE, ICONV.CONVERT, ICONV.OPEN.

Notes

This statement uses the *stat()* AS/400 C library function.

3.2.4 AS400.PCKDEC function

Name

AS400.PCKDEC – This function converts numeric data into the AS/400 packed decimal format.

Syntax

buffer = AS400.PCKDEC(*float*, *length*, *decimals*)

Description

The AS400.PCKDEC function converts a numeric variable into a buffer that contains a number in AS/400 packed decimal format.

The *float* argument is the numeric data to be converted. This expression is converted to a floating point value.

The *length* argument is the number of digits of packed decimal data to be created. Each digit in a packed decimal number is stored in half a byte, and the buffer is terminated with a half byte to denote the sign. The data will be padded with a leading half byte corresponding to a zero digit if the total number of half bytes does not add up to a whole number of bytes. So a number of 9 digits will be packed into a buffer of 5 bytes, 9 half bytes, plus the sign half byte. Similarly a number of 8 digits will also be packed into a buffer of 5 bytes, as a leading half byte is required to make a whole number of bytes. This expression is converted to an integer.

The *decimals* argument is the number of decimal digits in the packed decimal data. This expression is converted to an integer.

The *decimals* argument is used to multiply up the *float* argument before packing.

If the result of this multiplication leaves additional decimal data, then this data will be lost.

If the result of this multiplication does not fit into the *length*, then the most significant data will be lost.

The function returns a string type.

Errors

The following values can be assigned to `$error`:

- `$Error_AS400APIError`
Error in AS/400 API call
- `$Error_PckDecIllegal`
An illegal argument was passed to the function.

Examples

The following statement converts a numeric value to a packed decimal of length 7 with 2 decimals, and writes it to a flat file with an associated tag string.

```
DECLARE $tagLength = 10 CONSTANT INTEGER;  
DECLARE $packedLength = 7 CONSTANT INTEGER;
```



```

DECLARE $packedDecimals = 2 CONSTANT INTEGER;

DECLARE STATEMENT OUTPUTFILE IN $tag
        VALUE IN $value
{
    PRINT LEFT($tag, $tagLength)
        & AS400.PCKDEC($value,
            $packedLength,
            $packedDecimals)
        & "\n";

    RETURN;
}

```

See also

AS400.UNPDEC

Notes

This statement uses the *QXXDTOP()* AS/400 C library function.

3.2.5 AS400.RCVDTAQ statement

Name

AS400.RCVDTAQ – This statement receives data from an AS/400 data queue object.

Syntax

AS400.RCVDTAQ *dataQueueName* [WAIT *waitTime*] [KEYORDER *keyOrder*] [KEYDATA *keyData*] [DATA *data*] [JOBNAME *jobName*] [USER *user*] [JOBNUMBER *jobNumber*] [USERPROFILE *userProfile*]

Description

The AS400.RCVDTAQ statement receives data from an AS/400 data queue object.

The *dataQueueName* argument syntax is: "*library/dataQueue*" or "*dataQueue*". The *library* part of the argument must be the valid AS/400 name of an existing library, or one of the special values: "*LIBL" (the library list) or "*CURLIB" (the current library). If no *library* is specified, "*LIBL" is assumed. The *dataQueueName* argument must be the name of an existing data queue. It is an expression that is converted to a string.

The *waitTime* argument is the time in seconds to wait to receive a message. This expression is converted to an integer. If a negative value is specified for this argument, the wait time is infinite. If this argument is omitted, an infinite wait will be used as a default.

When an AS/400 data queue is created, it can be defined as keyed or unkeyed. If the data queue is keyed, then the *keyOrder* and *keyData* arguments may be used to provide keyed access to the data queue.

The *keyOrder* argument can take the following values:

- EQ equals
- GE greater than or equal
- GT greater than
- LE less than or equal
- LT less than
- NE not equal

It evaluates as an expression that is converted to a string.

The *keyData* argument is used with the *keyOrder* argument to provide keyed access to the data queue. It is an expression that is converted to a string. The length of the *keyData* argument must not be greater than the key length of the data queue, defined when it is created.

The *keyData* argument returns the value of the key of the data retrieved from the data queue.

If the data queue is unkeyed, then the *keyOrder* and *keyData* arguments must not be used.

The *data* argument is the data returned from the data queue. The returned value is of string type.

When an AS/400 data queue is created as keyed, it can be defined to retain information about the sender of data to the queue. If this is specified, the statement returns the *jobName*, *user*, *jobNumber*, and *userProfile*. These arguments are equivalent to the AS/400 Job Name, User, Job Number and User Profile of the job that sent the data to the data queue. The returned values are of string type.

If the wait time for a message is exceeded, the output arguments will return null or blank strings.

Errors

The following values can be assigned to *\$error*:

- *\$Error_AS400APIError*
Error in AS/400 API call
- *\$Error_SndDtaqNotKeyed*
An attempt has been made to access a non-keyed data queue using key information

Examples

The following example waits for an entry to appear at a data queue, and then processes it depending on the job it received the data from.

```
AS400.RCVDTAQ $DataQueue DATA $Data
      JOBNAM $JobName;
CASE $JobName
WHEN "MYAPPL" {
      ProcessData $Data;
}
WHEN OTHERS {
      /* Ignore */
}
```

```
        CONTINUE;  
    }
```

See also

S400.SNDDTAQ.

Notes

This statement uses the *QRCVDTAQ()* AS/400 C API function.

3.2.6 AS400.RCVMSG statement

Name

AS400.RCVMSG – This statement receives an AS/400 non-program message.

Syntax

```
AS400.RCVMSG msgQueueName [WAIT waitTime] [ACTION msgAction] [MSGID msgId]  
[MSGTYPE msgType] [MSGDATA msgData]
```

Description

The AS400.RCVMSG statement receives an AS/400 non-program message from a message queue.

The *msgQueueName* argument syntax is: "*library/msgQueue*" or "*msgQueue*". The *library* part of the argument must be the valid AS/400 name of an existing library, or one of the special values: "*LIBL" (the library list) or "*CURLIB" (the current library). If no *library* is specified, "*LIBL" is assumed. The *msgQueueName* argument must be the name of an existing message queue. It is usually associated with a particular user of the same name. It is an expression that is converted to a string.

The *waitTime* argument is the time in seconds to wait to receive a message. This expression is converted to an integer. If a value of -1 is specified for this argument the wait time is infinite. If this argument is omitted, an infinite wait will be used as a default.

The *msgAction* argument is the action to be taken when receiving the message. This expression is converted to a string. Valid values for this argument are "*REMOVE" (remove the message from the message queue), "*OLD" (mark the message as old, it will not be received again), and "*SAME" (leave the message). If this argument is omitted, the "*REMOVE" action will be used as a default.

The *msgId*, *msgType*, and *msgData* arguments are the Message ID, Message Type, and Message Data, respectively, of the returned message. The returned values are of string type. These arguments may be omitted. If the wait time for a message is exceeded, these arguments will return null or blank strings.

Errors

The following values can be assigned to `$error`:

```
$Error_AS400APIError
```

Examples

The following example waits for a message, and then processes it depending on the message received.

```
AS400.RCVMSG $MsgQueue[$i] WAIT $arg_WaitTime MSGID $MsgId
          MSGTYPE $MsgType MSGDATA $MsgData;

$MsgId = TRIMRIGHT($MsgId);

CASE $MsgId
WHEN "" {
    /* No more messages */
    CONTINUE;
}
WHEN "CPI8070" {
    TransferReportSENT $MsgData;
}
```

See also

AS400.SNDMSG.

Notes

This statement uses the *QMHRCVM* () AS/400 C API function.

3.2.7 AS400.RTVDTAARA statement

Name

AS400.RTVDTAARA – This statement retrieves the contents of an AS/400 data area object.

Syntax

AS400.RTVDTAARA *dtaaraName* [FROM *fromPosition*] [LENGTH *length*] [DATA *data*]

Description

The AS400.RTVDTAARA statement retrieves the contents of an AS/400 data area object.

The *dtaaraName* argument syntax is: "*library/dtaara*" or "*dtaara*". The *library* part of the argument must be the valid AS/400 name of an existing library, or one of the special values "*LIBL" (the library list) or "*CURLIB" (the current library). If no *library* is specified, "*LIBL" is assumed. The *dtaaraName* argument must be the name of an existing data area, or one of the special values "*LDA" (the local data area) or "*GDA" (the group data area). This expression is converted to a string.

The *fromPosition* argument is the position in the data area to read from. This expression is converted to an integer. This argument may be omitted when you want to read from the first position in the data area.

The *length* argument is the length of data to read from the data area. This expression is converted to an integer. This argument may be omitted when you want to read only a single character.

The *data* argument is the data returned from the read of the data area. The returned value is of string type. This argument may be omitted.

Errors

The following values can be assigned to `$error`:

```
$Error_AS400APIError
```

Error in AS/400 API call

Examples

The following user function retrieves a user name from the first ten characters of the LDA:

```
DECLARE FUNCTION RETRIEVE_USER()  
{  
  DECLARE $OutputString STRING;  
  
  AS400.RTVDTAARA "*LDA" LENGTH 10 DATA $OutputString;  
  
  RETURN $OutputString;  
}
```

Notes

This statement uses the `QWCRDTAA ()` AS/400 C API function.

3.2.8 AS400.SNDDTAQ function

Name

AS400.SNDDTAQ – This function sends data to an AS/400 data queue object.

Syntax

AS400.SNDDTAQ *dataQueueName* DATA *data* [KEYDATA *keyData*]

Description

The AS400.SNDDTAQ function sends data to an AS/400 data queue object.

The *dataQueueName* argument syntax is: "*library/dataQueue*" or "*dataQueue*". The *library* part of the argument must be the valid AS/400 name of an existing library, or one of the special values: "**LIBL*" (the library list) or "**CURLIB*" (the current library).

If no *library* is specified, *"*LIBL"* is assumed. The *dataQueueName* argument must be the name of an existing data queue. It is an expression that is converted to a string.

The *data* argument is the data to be sent to the data queue. This expression is converted to a string. If the length of the *data* argument exceeds the length of the data queue, then the data sent will be truncated.

When an AS/400 data queue is created, it can be defined as keyed or unkeyed. If the data queue is keyed, then the *keyData* argument may be used to provide keyed access to the data queue. It is an expression that is converted to a string. The length of the *keyData* argument must not be greater than the key length of the data queue, defined when it is created. If the data queue is unkeyed, then the *keyData* argument must not be used.

Errors

The following values can be assigned to *\$error*:

- *\$Error_AS400APIError*
Error in AS/400 API call
- *\$Error_SndDtaqNotKeyed*
An attempt has been made to access a non-keyed data queue using key information

Examples

The following example waits for an entry to appear at a data queue, and then processes it depending on the job it received the data from:

```
AS400.RCVDTAQ $DataQueue DATA $Data
      JOBNAME $JobName;

CASE $JobName
WHEN "MYAPPL" {
    ProcessData $Data;
}
WHEN OTHERS {
    /* Ignore */
    CONTINUE;
}
```

See also

AS400.SNDDTAQ.

Notes

This statement uses the *QRCVDTAQ()* AS/400 C API function.

3.2.9 AS400.SNDMSG statement

Name

AS400.SNDMSG – This statement sends an AS/400 non-program message.

Syntax

AS400.SNDMSG *msgQueueName* MESSAGE *msg*

Description

The AS400.SNDMSG statement sends an AS/400 non-program message of type `"*INFO"` to the specified message queue.

The *msgQueueName* argument syntax is: `"library/msgQueue"` or `"msgQueue"`. The *library* part of the argument must be the valid AS/400 name of an existing library, or one of the special values: `"*LIBL"` (the library list) or `"*CURLIB"` (the current library). If no *library* is specified, `"*LIBL"` is assumed. The *msgQueueName* argument must be the name of an existing message queue. It is usually associated with a particular user of the same name. This expression is converted to a string.

The *msg* argument is the message to be sent. This expression is converted to a string.

This statement will always send messages of type `"*INFO"` to the message queue specified.

Errors

The following values can be assigned to `$error`:

`$Error_AS400APIError`

Error in AS/400 API call

Examples

The following sends a message to the System Operator.

```
DECLARE STATEMENT SEND_MSG IN $Message
{
  AS400.SNDMSG "QSYSOPR" MESSAGE $Message;

  RETURN;
}
```

See also

AS400.RCVMSG.

Notes

This statement uses the *QMHSNDM* () AS/400 C API function.

3.2.10 AS400.UNBDEC function

Name

AS400.UNBDEC –This function converts AS/400 binary decimal data into numeric data.

Syntax

`float = AS400.UNBDEC(buffer, decimals, type)`

Description

The AS400.UNBDEC function converts an AS/400 binary decimal buffer into numeric data.

The *buffer* argument is the AS400 binary decimal data to be converted. This expression is converted to a string.

The *decimals* argument is the number of decimal digits in the binary decimal data. This expression is converted to an integer.

The *decimals* argument is used to divide into the result of the initial unpacking of the binary decimal buffer.

The *type* argument specifies whether the binary decimal data is signed or not. The symbolic variables *\$type_Signed_Binary* and *\$type_Unsigned_Binary* are defined in the header file, *e_as400.s4h*, for use with this argument.

The function returns a floating point value.

Errors

The following values can be assigned to *\$error*:

- *\$Error_AS400APIError*
Error in AS/400 API call
 - *\$Error_UnbDecIllegal*
An illegal argument was passed to the function.
-

Examples

The following statement converts a signed binary decimal value retrieved from a line of data into a numeric value:

```
DECLARE $tagLength = 10 CONSTANT INTEGER;
DECLARE $binaryLength = 7 CONSTANT INTEGER;
DECLARE $binaryDecimals = 2 CONSTANT INTEGER;

DECLARE STATEMENT RETRIEVEDATA IN $line
                                TAG    OUT $tag
                                VALUE  OUT $value
{
    $tag = STRMID($line, 1, $tagLength);
    $buffer = STRMID( $line,
                     $tagLength + 1,
                     $tagLength + $binaryLength);
    $value = AS400.UNBDEC($buffer,
                         $binaryDecimals,
                         $type_Signed_Binary);

    RETURN;
```


}

See also

AS400.BINDEC.

3.2.11 AS400.UNPDEC function

Name

AS400.UNPDEC –This function converts AS/400 packed decimal data into numeric data.

Syntax

float = AS400.UNPDEC(buffer, decimals)

Description

The AS400.UNPDEC function converts an AS/400 packed decimal buffer into numeric data.

The *buffer* argument is the AS400 packed decimal data to be converted. This expression is converted to a string.

The *decimals* argument is the number of decimal digits in the packed decimal data. This expression is converted to an integer.

The *decimals* argument is used to divide into the result of the initial unpacking of the packed decimal buffer.

The function returns a floating point value.

Errors

The following values can be assigned to \$error:

- \$Error_AS400APIError
Error in AS/400 API call
- \$Error_UnpDecIllegal
An illegal argument was passed to the function.

Examples

The following statement converts a packed decimal value retrieved from a line of data into a numeric value.

```
DECLARE $tagLength = 10 CONSTANT INTEGER;
DECLARE $packedLength = 7 CONSTANT INTEGER;
DECLARE $packedDecimals = 2 CONSTANT INTEGER;

DECLARE STATEMENT RETRIEVEDATA IN $line
                                TAG    OUT $tag
                                VALUE  OUT $value
{
    $tag = STRMID($line, 1, $tagLength);
```

```

    $buffer = STRMID( $line,
                      $tagLength + 1,
                      $tagLength + $packedlength);
    $value = AS400.UNPDEC($buffer, $packedDecimals);

    RETURN;
}

```

See also

AS400.PCKDEC.

Notes

This statement uses the *QXXPTOD()* AS/400 C library function.

3.2.12 AS400.UNZDEC function

Name

AS400.UNZDEC –This function converts AS/400 zoned decimal data into numeric data.

Syntax

float = AS400.UNZDEC(buffer, decimals)

Description

The AS400.UNZDEC function converts an AS/400 zoned decimal buffer into numeric data.

The *buffer* argument is the AS400 zoned decimal data to be converted. This expression is converted to a string.

The *decimals* argument is the number of decimal digits in the zoned decimal data. This expression is converted to an integer.

The *decimals* argument is used to divide into the result of the initial unpacking of the zoned decimal buffer.

The function returns a floating point value.

Errors

The following values can be assigned to *\$error*:

- *\$Error_AS400APIError*
Error in AS/400 API call
- *\$Error_UnzDecIllegal*
An illegal argument was passed to the function.

Examples

The following statement converts a zoned decimal value retrieved from a line of data into a numeric value.

```
DECLARE $tagLength = 10 CONSTANT INTEGER;
DECLARE $zonedLength = 7 CONSTANT INTEGER;
DECLARE $zonedDecimals = 2 CONSTANT INTEGER;

DECLARE STATEMENT RETRIEVEDATA IN $line
        TAG OUT $tag
        VALUE OUT $value

{
    $tag = STRMID($line, 1, $tagLength);
    $buffer = STRMID( $line,
        $tagLength + 1,
        $tagLength + $zonedlength);
    $value = AS400.UNZDEC($buffer, $zonedDecimals);

    RETURN;
}
```

See also

AS400.ZNDDEC.

Notes

This statement uses the *QXXZTOD()* AS/400 C library function.

3.2.13 AS400.USERINFO statement

Name

AS400.USERINFO – This statement retrieves information for an AS/400 user profile.

Syntax

AS400.USERINFO userProfile [MSGQ msgQueue] [MSGQLIB msgQueueLibrary]

Description

The AS400.USERINFO statement retrieves information for an AS/400 user profile.

The *userProfile* argument must be the name of an existing AS/400 user profile. This expression is converted to a string.

The *msgQueue* and *msgQueueLibrary* arguments are the Message Queue name and library name associated with the user profile. The returned values are of string type.

Errors

The following values can be assigned to \$error:

```
$Error_AS400APIError
```

Error in AS/400 API call

Examples

The following user function retrieves the message queue for a user and retrieves the last message from the queue.

```
DECLARE FUNCTION GET_MESSAGE($UserProfile)
{
  DECLARE $MessageQueue STRING;
  DECLARE $MessageQueueLibrary STRING;
  DECLARE $Msgq STRING;
  DECLARE $Message STRING;

  AS400.USERINFO $UserProfile
    MSGQ $MessageQueue
    MSGQLIB $MessageQueueLibrary;

  $msgq = $MessageQueueLibrary & "/" & $MessageQueue;

  AS400.RCVMSG $msgq MESSAGE $OutputString;

  RETURN $OutputString;
}
```

See also

AS400.RCVMSG, AS400.SNDMSG.

Notes

This statement uses the *QSYRUSRI* () AS/400 C API function.

3.2.14 AS400.ZNDDEC function

Name

AS400.ZNDDEC – This function converts numeric data into the AS/400 zoned decimal format.

Syntax

buffer = AS400.ZNDDEC(float, length, decimals)

Description

The AS400.ZNDDEC function converts a numeric variable into a buffer that contains a number in AS/400 zoned decimal format.

The *float* argument is the numeric data to be converted. This expression is converted to a floating point value.

The *length* argument is the number of digits of zoned decimal data to be created. Each digit in a zoned decimal number is stored in half a byte, and the buffer is terminated with a half byte to denote the sign. The data will be padded with a leading half byte corresponding to a zero digit if the total number of half bytes does not add up to a whole number of bytes. So a number of 9 digits will be zoned into a buffer of 5 bytes, 9 half bytes plus the sign half byte. Similarly a number of 8 digits will also be zoned into a buffer of 5 bytes, as a leading half byte is required to make a whole number of bytes. This expression is converted to an integer.

The *decimals* argument is the number of decimal digits in the zoned decimal data. This expression is converted to an integer.

The *decimals* argument is used to multiply up the *float* argument before packing.

If the result of this multiplication leaves additional decimal data, then this data will be lost.

If the result of this multiplication does not fit into the *length*, then the most significant data will be lost.

The function returns a string type.

Errors

The following values can be assigned to \$error:

- \$Error_AS400APIError
Error in AS/400 API call
- \$Error_ZndDecIllegal
An illegal argument was passed to the function.

Examples

The following statement converts a numeric value to a zoned decimal of length 7 with 2 decimals, and writes it to a flat file with an associated tag string.

```
DECLARE $tagLength = 10 CONSTANT INTEGER;
DECLARE $zonedLength = 7 CONSTANT INTEGER;
DECLARE $zonedDecimals = 2 CONSTANT INTEGER;

DECLARE STATEMENT OUTPUTFILE IN $tag
                                VALUE IN $value
{
    PRINT LEFT($tag, $tagLength)
      & AS400.ZNDDEC($value,
                    $zonedLength,
                    $zonedDecimals)
      & "\n";
```

```
    RETURN;  
}
```

See also

AS400.UNZDEC.

Notes

This statement uses the *QXXDTOZ()* AS/400 C library function.

3.3 The ICONV extension

This appendix describes the Message Builder ICONV extension. The ICONV extension runs only on the AS/400. It contains functions and statements for the conversion of data between different AS/400 character sets.

The ICONV extension is implemented as a module. All of the ICONV extension functions and statements are declared in the library files *e_iconv.s4* and *e_iconv.t4*. A number of constants (for example, error codes) can be found in the include file *e_iconv.s4h*.

3.4 Language reference

This section contains information about the functions and statements added by the ICONV extension.

3.4.1 ICONV.CCLOSE statement

Name

ICONV.CCLOSE – This statement closes a conversion handle.

Syntax

ICONV.CCLOSE *convId*

Description

The ICONV.CCLOSE statement closes a conversion handle specified by the *convId*. The *convId* must be an open conversion handle previously returned by the ICONV.COPEN function. This expression is converted to an integer.

Errors

The following values can be assigned to *\$error*:

```
$Error_IconvCloseFail
```

Failed to close conversion handle

Examples

The following user function converts a string from the AS/400 Swedish character set (CCSID=285) to the AS/400 Multinational character set (CCSID=500).

```
DECLARE FUNCTION CONVERT_TO_MULTINATIONAL($InputString)
{
  DECLARE $OutputString STRING;
  DECLARE $id INTEGER;

  $id = ICONV.COPEN(285, 500);
  $OutputString = ICONV.CONVERT($id, $InputString);
  ICONV.CCLOSE $id;

  RETURN $OutputString;
}
```

See also

ICONV.CONVERT, ICONV.COPEN.

Notes

This statement uses the *iconv_close()* AS/400 C library function.

3.4.2 ICONV.CONVERT function

Name

ICONV.CONVERT – This function converts a string from one AS/400 character set to another.

Syntax

ICONV.CONVERT(convId, inputString)

Description

The ICONV.CONVERT function converts a string of data from one AS/400 character set to another. The *convId* must be an open conversion handle previously returned by the ICONV.COPEN function. This expression is converted to an integer. The code character set identifiers, to convert from and to, are defined in the ICONV.COPEN function that returned the conversion handle. The *inputString* is an expression converted to a string.

This function returns a string containing the converted data.

Errors

The following values can be assigned to *\$error*:

\$Error_IconvConvertFail

Failed to convert data

Examples

The following user function converts a string from the AS/400 Swedish character set (CCSID=285) to the AS/400 Multinational character set (CCSID=500).

```
DECLARE FUNCTION CONVERT_TO_MULTINATIONAL($InputString)
{
  DECLARE $OutputString STRING;
  DECLARE $id INTEGER;

  $id = ICONV.COPEN(285, 500);
  $OutputString = ICONV.CONVERT($id, $InputString);
  ICONV.CCLOSE $id;

  RETURN $OutputString;
}
```

See also

ICONV.CCLOSE, ICONV.COPEN.

Notes

This function uses the *iconv()* AS/400 C library function.

3.4.3 ICONV.COPEN function

Name

ICONV.COPEN – This function opens a conversion handle.

Syntax

ICONV.COPEN(*fromccsid*, *toccsid*)

Description

The ICONV.COPEN function opens a conversion handle. The *fromccsid* and *toccsid* are the AS/400 code character set identifiers for the conversion process. Both expressions are converted to integers. The values of code character set identifiers to be used should be determined by referring to the AS/400 system manuals.

This function returns an integer value that should be used to refer to the handle when performing other operations with it.

Note that for optimum performance conversion handles should be opened at the beginning of a program and left open until the end of the program when possible.

Errors

The following values can be assigned to *\$error*:

```
$Error_IconvOpenFail
```


Failed to open conversion handle.

Examples

The following user function converts a string from the AS/400 Swedish character set (CCSID=285) to the AS/400 Multinational character set (CCSID=500).

```
DECLARE FUNCTION CONVERT_TO_MULTINATIONAL($InputString)
{
  DECLARE $OutputString STRING;
  DECLARE $id INTEGER;

  $id = ICONV.COPEN(285, 500);
  $OutputString = ICONV.CONVERT($id, $InputString);
  ICONV.CCLOSE $id;

  RETURN $OutputString;
}
```

See also

ICONV.CCLOSE, ICONV.CONVERT.

Notes

This function uses the *iconv_open()* AS/400 C library function.

3.5 The RFILE extension

This appendix describes the Message Builder RFILE extension. The RFILE extension runs only on the AS/400, and it contains functions and statements to read, write, delete, and update records from record files on the AS/400.

The RFILE extension is implemented as a module. All of the RFILE extension functions and statements are declared in the library files *e_rfile.s4* and *e_rfile.t4*. A number of constants (for example, error codes) can be found in the include file *e_rfile.s4h*.

3.6 Language reference

This section contains information about the functions and statements added by the RFILE extension.

3.6.1 RFILE.COMMIT statement

Name

RFILE.COMMIT – This statement commits pending database updates.

Syntax

RFILE.COMMIT

Description

The RFILE.COMMIT statement commits any database updates pending from the last commitment boundary. The statement creates a new commitment boundary.

This statement is only valid if commitment control is active. An exception, CPF8350, will be thrown if this is not the case.

To turn on commitment control you use the RFILE.STARTCOMMITSESSION Message Builder statement, or the STRCMTCTL AS/400 command.

Errors

The following values can be assigned to \$error:

```
$Error_RfileCommitFail
```

Failed to commit.

Examples

The following user function writes a record to the record file TEMPFILE in the library QTEMP.

```
DECLARE FUNCTION WRITE_RECORD()  
{  
  DECLARE $Record STRING;  
  DECLARE $id INTEGER;  
  
  RFILE.STARTCOMMITSESSION;  
  $id = RFILE.ROPEN("QTEMP/TEMPFILE", "ar+",  
                   $Rfile_OpenCommitTrue);  
  $Record = "NewData";  
  RFILE.RWRITE $id DATA $Record;  
  RFILE.COMMIT;  
  RFILE.RCLOSE $id;  
  RFILE.ENDCOMMITSESSION;  
  
  RETURN $Record;  
}
```

See also

RFILE.ENDCOMMITSESSION, RFILE.ROLLBACK, RFILE.ROPEN, RFILE.STARTCOMMITSESSION.

Notes

This statement uses the `_Rcommit ()` AS/400 C library function.

3.6.2 RFILE.ENDCOMMITSESSION statement

Name

RFILE.ENDCOMMITSESSION – This statement ends a commit session for the current job.

Syntax

RFILE.ENDCOMMITSESSION

Description

The RFILE.ENDCOMMITSESSION statement ends a commit session for the current job. The statement deactivates commitment control.

This statement is only valid if commitment control is active. An exception, RFILE, will be thrown if this is not the case.

Errors

The following values can be assigned to \$error:

`$Error_RfileEndCommitSessionFail`

Failed to end commitment control.

Examples

The following user function writes a record to the record file TEMPFILE in the library QTEMP.

```
DECLARE FUNCTION WRITE_RECORD()  
{  
  DECLARE $Record STRING;  
  DECLARE $id INTEGER;  
  
  RFILE.STARTCOMMITSESSION;  
  $id = RFILE.ROPEN("QTEMP/TEMPFILE", "a+",  
                   $Rfile_OpenCommitTrue);  
  $Record = "NewData";  
  RFILE.RWRITE $id DATA $Record;  
  RFILE.COMMIT;  
  RFILE.RCLOSE $id;  
  RFILE.ENDCOMMITSESSION;  
  
  RETURN $Record;  
}
```

See also

RFILE.RCOMMIT, RFILE.ROLLBACK, RFILE.ROPEN, RFILE.STARTCOMMITSESSION.

Notes

This statement uses the *ENDCMTCTL* AS/400 CL command.

3.6.3 RFILE.FORMAT statement

Name

RFILE.FORMAT – This statement sets the record format to use in a record file.

Syntax

RFILE.FORMAT *fileId* NAME *formatName*

Description

The RFILE.FORMAT statement sets the record format to use in the open file specified by the *fileId*.

The *fileId* must be an open file ID previously returned by the RFILE.ROPEN function. This expression is converted to an integer.

The *formatName* argument specifies the name of the record format to change to. The argument is an expression converted to a string and must be in upper case.

If a *formatName* argument is used which does not exist for the open file, then no error will occur, and the record format will not be changed.

On opening a file, the default record format will be used.

It should be noted that only one file pointer is maintained for each file opened using the RFILE.OPEN statement. If the file is positioned using one record format, and the record format is changed, then that file position is lost. This could cause a problem if you are using a multi-format file, and you need to use file positioning on one of the formats in this file. The only safe way to program this is to open the file once for each format to be used, and then use RFILE.FORMAT once for each open file to set the appropriate record format. A file pointer will now be maintained for each file opened, which is only used for a single record format.

Errors

The following values can be assigned to *\$error*:

```
$Error_RfileFormatFail
Failed to change the record format
```

Examples

The following user function reads the first record from the CUSTOMER record format in a record file TEMPFILE in the library QTEMP:

```
DECLARE FUNCTION READ_FIRST_RECORD()
{
  DECLARE $Record STRING;
  DECLARE $id INTEGER;
```

```

$id = RFILE.ROPEN("QTEMP/TEMPFILE", "rr");
RFILE.FORMAT $id NAME "CUSTOMER";
$Record = RFILE.RREAD($id);
RFILE.RCLOSE $id;

RETURN $Record;
}

```

See also

RFILE.ROPEN.

Notes

This statement uses the `_Rformat ()` AS/400 C library function.

3.6.4 RFILE.LOCATE statement

Name

RFILE.LOCATE – This statement positions the file pointer of a record file.

Syntax

RFILE.LOCATE *fileId* [KEY *keyValue*] [TYPE *positionType*]

Description

The RFILE.LOCATE statement positions the file pointer of open file specified by the *fileId*.

The *fileId* must be an open file ID previously returned by the RFILE.ROPEN function. This expression is converted to an integer.

The *positionType* argument is the type of file pointer positioning that is requested. This expression is converted to an integer. The following values are supported which may be implemented using symbolic variables defined in the file *e_rfile.s4h*.

Values	Meaning
\$RFile_LocateEnd	Positions to just after the last record in a file. There is no record associated with this position. The <i>keyValue</i> parameter is ignored.
\$RFile_LocateFirst	Positions to the first record in the access path that is currently being used by the open file. The <i>keyValue</i> argument is ignored.
\$RFile_LocateEQ	Positions to the first record with the value specified by the <i>keyValue</i> argument.
\$RFile_LocateGE	Positions to the first record that has a key greater than or equal to the value specified by the <i>keyValue</i> argument.
\$RFile_LocateGT	Positions to the first record that has a key greater than the value specified by the <i>keyValue</i> argument.
\$RFile_LocateLE	Positions to the first record that has a key less than or equal to the value specified by the <i>keyValue</i> argument.
\$RFile_LocateLT	Positions to the first record that has a key less than the value specified by the <i>keyValue</i> argument.

<code>\$RFile_LocateLast</code>	Positions to the last record in the access path that is currently being used by the open file. The <code>keyValue</code> argument is ignored.
<code>\$RFile_LocateNext</code>	Positions to the next record in the access path that is currently being used by open file. The <code>keyValue</code> argument is ignored.
<code>\$RFile_LocatePrevious</code>	Positions to the previous record in the access path that is currently being used by open file. The <code>keyValue</code> argument is ignored.
<code>\$RFile_LocateStart</code>	Positions to just before the first record in the file. There is no record associated with this position. The <code>keyValue</code> parameter is ignored.

Note: `$RFile_LocateEnd` and `$RFile_LocateStart` do not position to a record, so an attempt to use `RFILE.RREAD` using the type `$Rfile_ReadCurrent` will fail.

The *keyValue* argument is the key value used to position the file pointer. This expression is converted to a string. The default value of this argument is a null string. This argument has no meaning when used with the following values for the *positionType* argument:

`$RFile_LocateEnd`, `$RFile_LocateFirst`, `$RFile_LocateLast`, `$RFile_LocateNext`, `$RFile_LocatePrevious` and `$RFile_LocateStart`

If a key value that is shorter than the key length is used in the *keyValue* argument, only the partial key will be used to position the file.

If this statement fails to execute an exception, `RFILE`, will be thrown. Typical examples of failures would be:

- Attempting to position a file to a record using `$RFile_LocateEQ` for a key value that does not exist
- Attempting to position a file using a key value that is larger than the key length
- Attempting to position a file which does not have a key

If this statement fails with a non-recoverable I/O error, then the file must be closed and re-opened.

Errors

The following values can be assigned to `$error`:

- `$Error_RfileLocateFail`
Failed to position file cursor
- `$Error_RfileLocateTypeUndefined`
Position type undefined

Examples

The following user function reads the first record from a record file `TEMPFILE` in the library `QTEMP`.

```
DECLARE FUNCTION READ_FIRST_RECORD()
{
  DECLARE $Record STRING;
  DECLARE $id INTEGER;

  $id = RFILE.ROPEN("QTEMP/TEMPFILE", "rr");
  RFILE.LOCATE $id TYPE $RFile_LocateFirst;
  $Record = RFILE.RREAD($id);
  RFILE.RCLOSE $id;
```

```
    RETURN $Record;
}
```

See also

RFILE.ROPEN, RFILE.RREAD.

Notes

This statement uses the `_Rlocate ()` AS/400 C library function.

3.6.5 RFILE.RCLOSE statement

Name

RFILE.RCLOSE – This statement closes a record file.

Syntax

RFILE.RCLOSE *fileId*

Description

The RFILE.RCLOSE statement closes the open file specified by the *fileId*. The *fileId* must be an open file ID previously returned by the RFILE.ROPEN function. This expression is converted to an integer.

Errors

The following values can be assigned to `$error`:

```
$Error_RfileCloseFail
```

Failed to close the file

Examples

The following user function reads the first record from a record file TEMPFILE in the library QTEMP.

```
DECLARE FUNCTION READ_FIRST_RECORD()
{
  DECLARE $Record STRING;
  DECLARE $id INTEGER;

  $id = RFILE.ROPEN("QTEMP/TEMPFILE", "rr");
  $Record = RFILE.RREAD($id);
  RFILE.RCLOSE $id;

  RETURN $Record;
}
```

See also

RFILE.RDELETE, RFILE.ROPEN, RFILE.RREAD, RFILE.RUPDATE, RFILE.RWRITE.

Notes

This statement uses the `_Rclose ()` AS/400 C library function.

3.6.6 RFILE.RDELETE statement

Name

RFILE.RDELETE – This statement deletes a record from a record file.

Syntax

RFILE.RDELETE *fileId*

Description

The RFILE.RDELETE statement deletes the current record from a record file. The *fileId* must be an open record file previously returned by the RFILE.ROPEN function. This expression is converted to an integer. The file must be open for update. The RFILE.RDELETE statement must be preceded by an RFILE.RREAD function.

Note that the file pointer position after running RFILE.RDELETE is deleted.

Errors

The following values can be assigned to `$error`:

`$Error_RfileDeleteFail`

Failed to delete record

Examples

The following user function deletes the first record from a record file TEMPFIL in the library QTEMP.

```
DECLARE FUNCTION DELETE_FIRST_RECORD()
{
  DECLARE $Record STRING;
  DECLARE $id INTEGER;

  $id = RFILE.ROPEN("QTEMP/TEMPFIL", "rr+");
  $Record = RFILE.RREAD($id);
  RFILE.RDELETE $id;
  RFILE.RCLOSE $id;

  RETURN $Record;
}
```

See also

RFILE.RCLOSE, RFILE.ROPEN, RFILE.RREAD, RFILE.RUPDATE, RFILE.RWRITE.

Notes

This statement uses the `_Rdelete()` AS/400 C library function.

3.6.7 RFILE.ROLLBACK statement

Name

RFILE.ROLLBACK – This statement rolls back pending database updates.

Syntax

RFILE.ROLLBACK

Description

The RFILE.ROLLBACK statement rolls back any database updates pending from the last commitment boundary. The statement creates a new commitment boundary.

This statement is only valid if commitment control is active. An exception, CPF8350, will be thrown if this is not the case.

To turn on commitment control you use the RFILE.STARTCOMMITSESSION Message Builder statement, or the STRCMTCTL AS/400 command.

Errors

The following values can be assigned to `$error`:

`$Error_RfileRollbackFail`

Failed to rollback.

Examples

The following user function writes records to the record file TEMPFIL in the library QTEMP from a flat file.

```
DECLARE STATEMENT WRITE_RECORDS IN $flatFile
{
  DECLARE $Record STRING;
  DECLARE $id INTEGER;
  DECLARE $errorFlag = 0 INTEGER;

  RFILE.STARTCOMMITSESSION;
  $id = RFILE.ROPEN("QTEMP/TEMPFIL", "ar+",
                  $Rfile_OpenCommitTrue);

  WHILE 1 {
    TRY {
      $Record = ReadLine($flatFile);
    }
    CATCH $exception
```

```

WHEN OTHERS {
    LOG "Error reading file" TYPE "ERROR";
    $errorFlag = 1;
    BREAK;
}

    IF $Record = "" {
        BREAK;
    }

RFILE.RWRITE $id DATA $Record;
}

IF $errorFlag {
    RFILE.ROLLBACK;
}
ELSE {
    RFILE.COMMIT;
}

RFILE.RCLOSE $id;
RFILE.ENDCOMMITSESSION;

RETURN;
}

```

See also

RFILE.COMMIT, RFILE.ENDCOMMITSESSION, RFILE.ROPEN, RFILE.STARTCOMMITSESSION.

Notes

This statement uses the `_Rrollbck ()` AS/400 C library function.

3.6.8 RFILE.ROPEN function

Name

RFILE.ROPEN – This function opens a record file.

Syntax

RFILE.ROPEN(*fileName* [, *openMode*] [, *commitFlag*])

Description

The RFILE.ROPEN function opens a record file for reading, writing, or both.

The *fileName* argument is the name of the file to open. The argument is an expression converted to a string.

The *openMode* argument specifies the mode of open. The argument is an expression converted to a string.

The *commitFlag* argument specifies whether the file is opened with commitment control or not. The argument is an expression converted to an integer.

The *fileName* argument should have the syntax "*library/file(member)*", "*library/file*", "*file(member)*" or "*file*". The *library* part of the argument must be a valid AS/400 name of an existing library, or one of the special values: "*LIBL" (the library list) or "*CURLIB" (the current library). If no *library* is specified, "*LIBL" is assumed. The *file* part of the argument must be a valid AS/400 name of an existing file. The *member* part of the argument must be a valid AS/400 name of an existing member of the file *file*. If no *member* is specified, then the default member, "*FIRST", is assumed.

The *openMode* argument must have one of the following values:

Values	Meaning
rr	Open an existing file for reading records.
wr	Open an existing file for writing records. If the file contains data, the content is cleared unless the file is a logical file.
ar	Open an existing file for writing records to the end of the file (append).
rr+	Open an existing file for reading, writing, or updating records.
wr+	Open an existing file for reading, writing, or updating records. If the file contains data, the content is cleared unless the file is a logical file.
ar+	Open an existing file for reading and writing records. All data is written to the end of the file.

If *openMode* is not specified, the file is opened for reading.

The *commitFlag* argument must have one of the following values, which are supported through the use of symbolic variables defined in the file *e_rfile.s4h*:

Values	Meaning
<i>\$RFile_OpenCommitFalse</i>	Open the file without commitment control.
<i>\$RFile_OpenCommitTrue</i>	Open the file with commitment control.

A file can only be opened with commitment control if:

- The file is journalled
- Commitment control is turned on

To turn on commitment control you use the RFILE.STARTCOMMITSESSION Message Builder statement, or the STRCMTCTL AS/400 command.

If *commitFlag* is not specified, the file is opened without commitment control.

This function returns an integer value that should be used to refer to the handle when performing other operations with it.

Errors

The following values can be assigned to *\$error*:

```
$Error_RfileOpenFail
```

Failed to open file

Examples

The following user function reads the first record from a record file TEMPFIL in the library QTEMP.

```
DECLARE FUNCTION READ_FIRST_RECORD()
{
  DECLARE $Record STRING;
```

```

DECLARE $id INTEGER;

$id = RFILE.ROPEN("QTEMP/TEMPFILE", "rr");
$Record = RFILE.RREAD($id);
RFILE.RCLOSE $id;

RETURN $Record;
}

```

See also

RFILE.COMMIT, RFILE.ENDCOMMITSESSION, RFILE.RCLOSE, RFILE.RDELETE, RFILE.ROLLBACK, RFILE.RREAD, RFILE.RUPDATE, RFILE.RWRITE, RFILE.STARTCOMMITSESSION.

Notes

This function uses the `_Ropen()` AS/400 C library function.

3.6.9 RFILE.RREAD function

Name

RFILE.RREAD – This function reads the next record from a record file.

Syntax

RFILE.RREAD(*fileId*[, *type*])

Description

The RFILE.RREAD function reads a record from a record file.

The *fileId* must be an open record file previously returned by the RFILE.ROPEN function. This expression is converted to an integer.

The *type* is the type of read requested. The supported values may be implemented using symbolic variables defined in the file `e_rfile.s4h`.

Values	Meaning
<code>\$Rfile_ReadNext</code>	Read the next record.
<code>\$Rfile_ReadPrevious</code>	Read the previous record.
<code>\$Rfile_ReadFirst</code>	Read the first record in the file.
<code>\$Rfile_ReadLast</code>	Read the last record in the file.
<code>\$Rfile_ReadCurrent</code>	Read the current record in the file.

If no type is specified, `$Rfile_ReadNext` is the default.

Records are read from the file using the access path used when the file was created. When the end of file is reached, an exception is thrown. The exception string is "EOF".

Note that if `$RFile_LocateEnd` and `$RFile_LocateStart` are used to position the file using RFILE.LOCATE, the file pointer is not positioned to a record, so an attempt to use RFILE.RREAD using the type `$Rfile_ReadCurrent` will fail.

A string containing the record data is returned. Note that the data corresponds to a byte-by-byte copy of the record in the file, with no delimiters for fields in the record. Field data should be extracted from the record using the STRMID Message Builder operation code. Conversion may be required for fields that are not character-based. For example, packed decimal fields can be converted from the AS/400 packed format to the Message Builder Integer or Float values using the AS400.UNPDEC Message Builder function.

Errors

The following values can be assigned to `$error`:

- `$Error_RfileReadFail`
Failed to read a record
- `$Error_RfileReadUnknownType`
An unknown type was specified to the READ function

Examples

The following user function reads the first record from a record file TEMPFIL in the library QTEMP.

```
INCLUDE "e_rfile.s4h";
DECLARE FUNCTION READ_FIRST_RECORD()
{
  DECLARE $Record STRING;
  DECLARE $id INTEGER;

  $id = RFILE.ROPEN("QTEMP/TEMPFIL", "rr");
  $Record = RFILE.RREAD($id, $Rfile_ReadNext);
  RFILE.RCLOSE $id;

  RETURN $Record;
}
```

See also

RFILE.LOCATE, RFILE.RCLOSE, RFILE.RDELETE, RFILE.ROPEN, RFILE.RUPDATE, RFILE.RWRITE.

Notes

This function uses the `_Rreadf()`, `_Rreadl()`, `_Rreadn()`, `_Rreadp()` and `_Rreadc()` AS/400 C library functions.

3.6.10 RFILE.RUPDATE statement

Name

RFILE.RUPDATE – This statement updates a record in a record file.

Syntax

RFILE.RUPDATE *fileId* DATA *data*

Description

The RFILE.RUPDATE statement updates the current record in a record file. The *fileId* must be an open record file previously returned by the RFILE.ROPEN function. This expression is converted to an integer. The file must be open for update. The RFILE.RUPDATE statement must be preceded by an RFILE.RREAD function.

Errors

The following values can be assigned to \$error:

`$Error_RfileUpdateFail`

Failed to update record

Examples

The following user function updates the first record from a record file TEMPFILE in the library QTEMP.

```
DECLARE FUNCTION UPDATE_FIRST_RECORD()
{
  DECLARE $Record STRING;
  DECLARE $id INTEGER;

  $id = RFILE.ROPEN("QTEMP/TEMPFILE", "rr+");
  $Record = RFILE.RREAD($id);
  $Record = "NewData";
  RFILE.RUPDATE $id DATA $Record;
  RFILE.RCLOSE $id;

  RETURN $Record;
}
```

See also

RFILE.RCLOSE, RFILE.ROPEN, RFILE.RDELETE, RFILE.RREAD, RFILE.RWRITE.

Notes

This statement uses the `_Rupdate()` AS/400 C library function.

3.6.11 RFILE.RWRITE statement

Name

RFILE.RWRITE – This statement writes a record to a record file.

Syntax

RFILE.RWRITE *fileId* DATA *data*

Description

The RFILE.RWRITE statement writes a record to a record file. The *fileId* must be an open record file previously returned by the RFILE.ROPEN function. This expression is converted to an integer. The file must be open for writing. The record is appended to the end of the file.

Errors

The following values can be assigned to \$error:

```
$Error_RfileWriteFail
```

Failed to write record

Examples

The following user function writes a record to the record file TEMPFILE in the library QTEMP.

```
DECLARE FUNCTION WRITE_RECORD()  
{  
  DECLARE $Record STRING;  
  DECLARE $id INTEGER;  
  
  $id = RFILE.ROPEN("QTEMP/TEMPFILE", "a+");  
  $Record = "NewData";  
  RFILE.RWRITE $id DATA $Record;  
  RFILE.RCLOSE $id;  
  
  RETURN $Record;  
}
```

See also

RFILE.RCLOSE, RFILE.ROPEN, RFILE.RDELETE, RFILE.RREAD, RFILE.RUPDATE.

Notes

This statement uses the `_Rwrite()` AS/400 C library function.

3.6.12 RFILE.STARTCOMMITSESSION statement

Name

RFILE.STARTCOMMITSESSION – This statement starts a commit session for the current job.

Syntax

RFILE.STARTCOMMITSESSION

Description

The RFILE.STARTCOMMITSESSION statement starts a commit session for the current job. The statement activates commitment control and creates a new commitment boundary.

This statement is only valid if commitment control is not active. An exception, RFILE, will be thrown if this is not the case.

Errors

The following values can be assigned to \$error:

```
$Error_RfileStartCommitSessionFail
```

Failed to start commitment control.

Examples

The following user function writes a record to the record file TEMPFILE in the library QTEMP.

```
DECLARE FUNCTION WRITE_RECORD()  
{  
  DECLARE $Record STRING;  
  DECLARE $id INTEGER;  
  
  RFILE.STARTCOMMITSESSION;  
  $id = RFILE.ROPEN("QTEMP/TEMPFILE", "ar+",  
                   $Rfile_OpenCommitTrue);  
  $Record = "NewData";  
  RFILE.RWRITE $id DATA $Record;  
  RFILE.COMMIT;  
  RFILE.RCLOSE $id;  
  RFILE.ENDCOMMITSESSION;  
  
  RETURN $Record;  
}
```

See also

RFILE.RCOMMIT, RFILE.ROLLBACK, RFILE.ROPEN, RFILE.ENDCOMMITSESSION.

Notes

This statement uses the *STRCMTCTL* AS/400 CL command.

3.7 The ASEXEC file extension

This section describes the Message Builder ASEXEC extension. The ASEXEC extension runs only on the AS/400. It contains functions and statements that enable the dynamic calling of programs on the AS/400.

The ASEXEC extension is implemented as a module. All of the ASEXEC extension functions and statements are declared in the library files *e_asexec.s4* and *e_asexec.t4*. A number of constants (for example, error codes) can be found in the include file *e_asexec.s4h*.

3.8 Example implementation

The following example shows how the ASEXEC extension can be used to execute a dynamic program call:

```
INCLUDE "e_asexec.s4h";

DECLARE $parameters[] INTEGER;

$parameters[1] = ASEXEC.AddParameter($dataType_String, 15, 0);
$parameters[2] = ASEXEC.AddParameter($dataType_PackedDecimal, 6, 2);

ASEXEC.SetParameter $parameters[1]
                     DATA "          ";
ASEXEC.SetParameter $parameters[2]
                     DATA AS400.PckDec(0, 6, 2);

ASEXEC.ExecuteProgram "TESTPGM"
                     PARAMETERS $parameters;

PRINT "Output string parameter is <" &
ASEXEC.GetParameter($parameters[1]) & ">\n";
PRINT "Output packed parameter is <" &
AS400.UnpDec(ASEXEC.GetParameter($parameters[2]), 2) & ">\n";

ASEXEC.RemoveParameter $parameters[1];
ASEXEC.RemoveParameter $parameters[2];
```

Note especially that:

- When using ASEXEC.AddParameter, the parameters must be created with the correct data type and size specification to match the parameters of the program to be executed.
- Each parameter must be initialized before the program is executed.
- The correct encoding or decoding of data must be done when using ASEXEC.SetParameter and ASEXEC.GetParameter.

3.9 Language Reference

This section contains information about the functions and statements added by the ASEXEC extension.

3.9.1 ASEXEC.AddParameter function

Name

ASEXEC.AddParameter – This function creates a parameter for use when executing a program.

Syntax

ASEXEC.AddParameter(*dataType*[, *length*][, *decimals*])

Description

The ASEXEC.AddParameter function creates a parameter.

The *dataType* argument is the data type of the parameter. This argument is an expression that is converted to an integer. The valid values are the symbolic variables \$dataType_String, \$dataType_PackedDecimal and \$dataType_ZonedDecimal which can be found in the file *e_asexec.s4h*.

The *length* argument is the size of the data that should be defined for the parameter, and is mandatory for data types that can have a variable length. On the AS/400, all parameters have a fixed length, but may contain data that is of shorter length. For a string parameter the length will be the number of characters of the parameter. For a packed decimal parameter it will be the number of digit places of the parameter, including decimals. For example, a packed decimal of length 5 and decimals 2 can have the value +999.99 to -999.99

This argument defaults to 0, and should be left empty for parameter data types of fixed length. This argument is an expression that is converted to an integer.

The *decimals* argument is the number of decimal positions that are defined for a numeric parameter of variable length. This argument defaults to 0, and should be left empty for parameter data types of fixed length and for string parameters.

This argument is an expression that is converted to an integer.

This function returns an integer value that should be used to refer to the parameter when performing other operations with it.

Errors

The following values can be assigned to \$Error:

- \$Error_AsexecAddParameterIllegalDataType
An invalid data type was used for the parameter
- \$Error_AsexecAddParameterLengthMandatory
The length argument was not included for a data type of variable length

3.9.2 ASEXEC.ExecuteProgram statement

Name

ASEXEC.ExecuteProgram – This statement executes a program.

Syntax

ASEXEC.ExecuteProgram *programName* [PARAMETERS *parameterIds*]

Description

The ASEXEC.ExecuteProgram executes a program.

The *programName* argument is the name of the program to be executed. The program must exist in the library list of the process executing this MessageBuilder code. It is an expression that is converted to a string.

The *parameterIds* argument is an array of open parameter identifiers previously returned by the ASEXEC.AddParameter function. Each parameter identifier should have been set to a value or initialized using the ASEXEC.SetParameter statement. Each element of the array is an expression that is converted to an integer.

Errors

The following values can be assigned to \$Error:

- \$Error_AsexecExecuteProgramResolveFail
Failed to find the program to be executed
- \$Error_AsexecExecuteProgramExecuteFail
An error occurred when executing the program
- \$Error_AsexecExecuteProgramIllegalParameters
An invalid number of parameters were used.
- \$Error_ParameterHandleNotFound
One of the parameter identifiers is invalid

3.9.3 ASEXEC.GetParameter function

Name

ASEXEC.GetParameter – This function returns the value from a parameter.

Syntax

ASEXEC.GetParameter(*parameterId*)

Description

The ASEXEC.GetParameter function returns the value from a parameter.

The *parameterId* argument is a parameter identifier previously returned by the ASEXEC.AddParameter function.

This function returns a byte string that contains the value of the parameter. This value may be encoded, depending on the data type of the parameter. The following functions can be used to decode these encoded values:

Data Type	Function
Packed Decimal	AS400.UnpDec
Zoned Decimal	AS400.UnzDec

Errors

The following values can be assigned to `$Error`:

`$Error_ParameterHandleNotFound`

The parameter identifier is invalid.

3.9.4 ASEXEC.RemoveParameter statement

Name

ASEXEC.RemoveParameter – This statement destroys a parameter.

Syntax

ASEXEC.RemoveParameter *parameterId*

Description

The ASEXEC.RemoveParameter statement destroys a parameter.

The *parameterId* argument is a parameter identifier previously returned by the ASEXEC.AddParameter function.

Errors

The following values can be assigned to `$Error`:

- `$Error_AsexecRemoveParameterIdListFail`
An error occurred while removing the parameter
- `$Error_ParameterHandleNotFound`
The parameter identifier is invalid

3.9.5 ASEXEC.SetParameter statement

Name

ASEXEC.SetParameter – This function sets the value of a parameter.

Syntax

ASEXEC.SetParameter *parameterId* DATA *data*

Description

The `ASEXEC.SetParameter` function sets the value of a parameter.

The *parameterId* argument is a parameter identifier previously returned by the `ASEXEC.AddParameter` function.

The *data* argument is a byte string that contains the value of the parameter to be set. This value may be encoded, depending on the data type of the parameter. The following functions can be used to encode these values:

Data Type Function

Packed Decimal `AS400.PckDec`

Zoned Decimal `AS400.ZndDec`

Errors

The following values can be assigned to `$Error`:

`$Error_ParameterHandleNotFound`

The parameter identifier is invalid.